

COMPUTER ORGANISATION & ARCHITECTRE

Handwritten Notes

VIVEKANAND VERNEKAR

GateWallah Parakram C 2024

(Hinglish Weekday)

Other Notes Uploaded Below

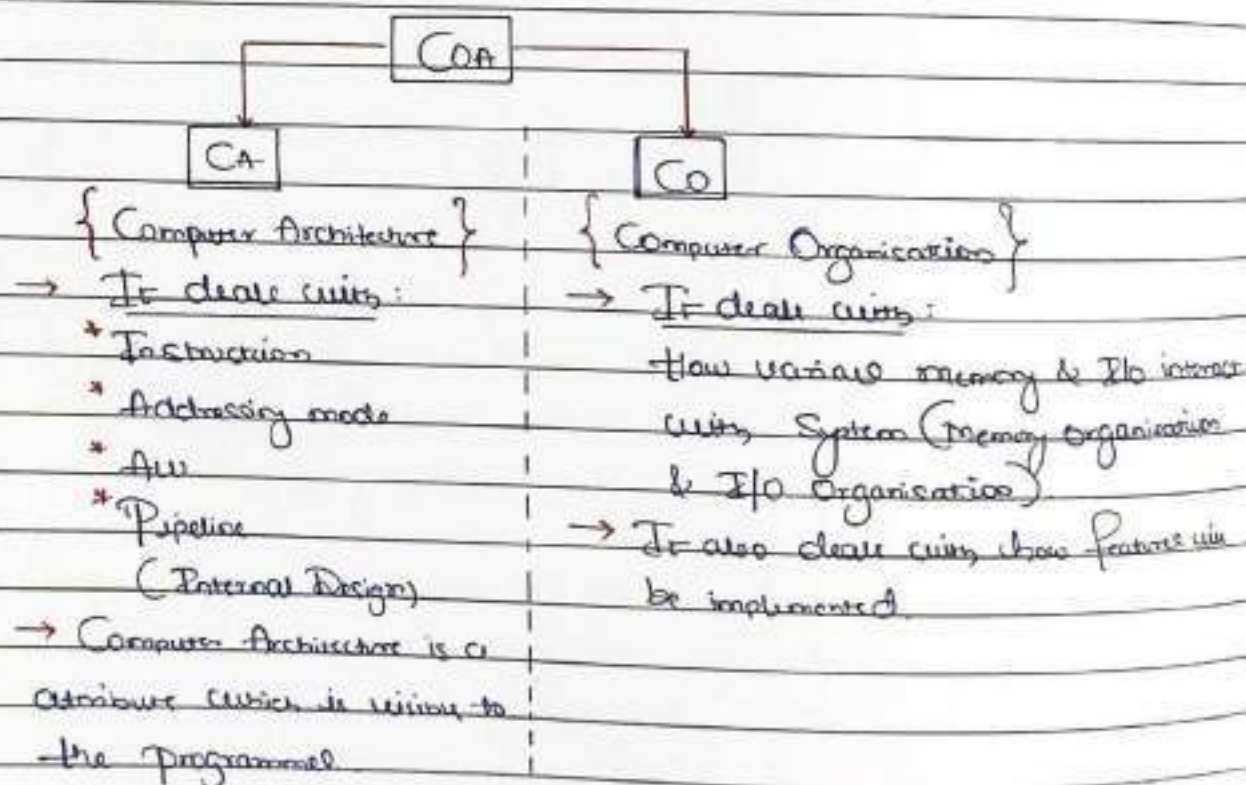
<https://t.me/pwgatenotes>

COMPUTER ORGANIZATION & ARCHITECTURE.

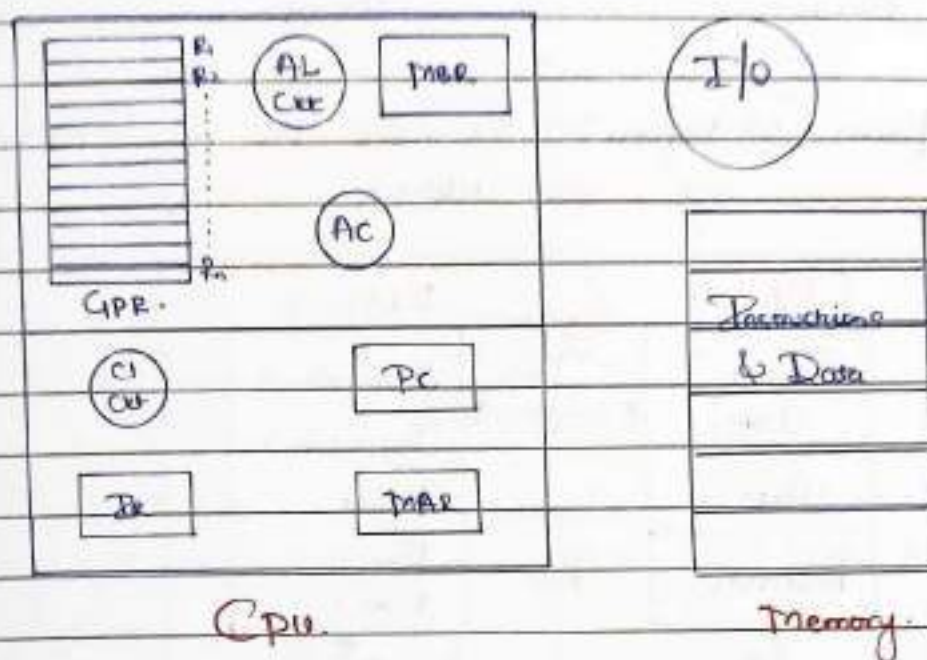
Computer Generation:

	1 st	2 nd	3 rd	4 th and 5 th
	1942-1955	1955-1964	1965-1974	1974-Present
Components	Vacuum Tube	Transistor	IC (Integrated Chip)	VLSI & ULSI
Language	Machine Language	Assembly language	HLL, COBOL, FORTRAN	COBOL, FORTRAN, JAVA, C++

* The First Digital Computer was "ENIAC" in 1943 (Electronic Numerical and Integrator Computer).



Note: Intel x86 has same architecture but different organization.

Components of a Computer:

- ① Program Counter [PC]: It contains starting address of the next instruction to be executed (fetched).
- ② Memory Address Register [MAR]: MAR contains the memory address used for either read or write operations.
- ③ DR/MDB/MBR (Memory Buffer Register): Holds the instructions or data.
- ④ IR (Instruction Register): It contains the instructions which are currently executed by the CPU.

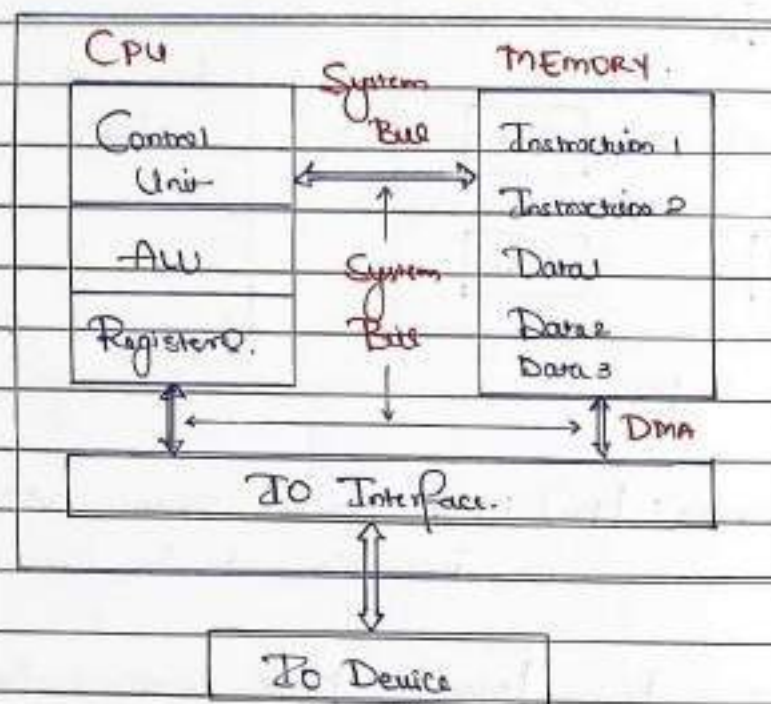
Note:

* Instruction is stored in IR because instruction's format is predefined in IR.

- ⑤ AC (Accumulator): It contains the temporary result of all operations or first operand (data) of all operations.
- ⑦ GPR (General Purpose Register): It is used for processing the data.

⑧ PSW (Program Status Register) / Flag Register : It store the status of the last result.

⑨ Stack Pointer (SP Register) : Contains the Top (Top of the Stack) Address



Instruction Cycle:

* The process required for each instruction execution

or

* The instruction cycle describe the execution sequence of the instructions.

Instruction Cycle contains two sub cycle:

① Fetch Cycle : To fetch (to bring) the instructions from memory to CPU and at the end of fetch cycle PC (Program Counter) is incremented, now PC will denote next instructions starting address.

② Execute Cycle : To process (to execute) the fetched instruction.

① Decode

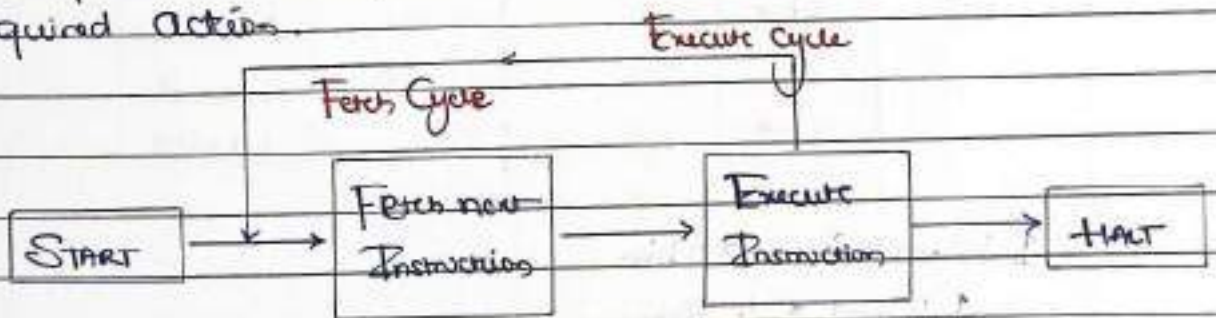
② Execute

Steps in Instruction Cycle:

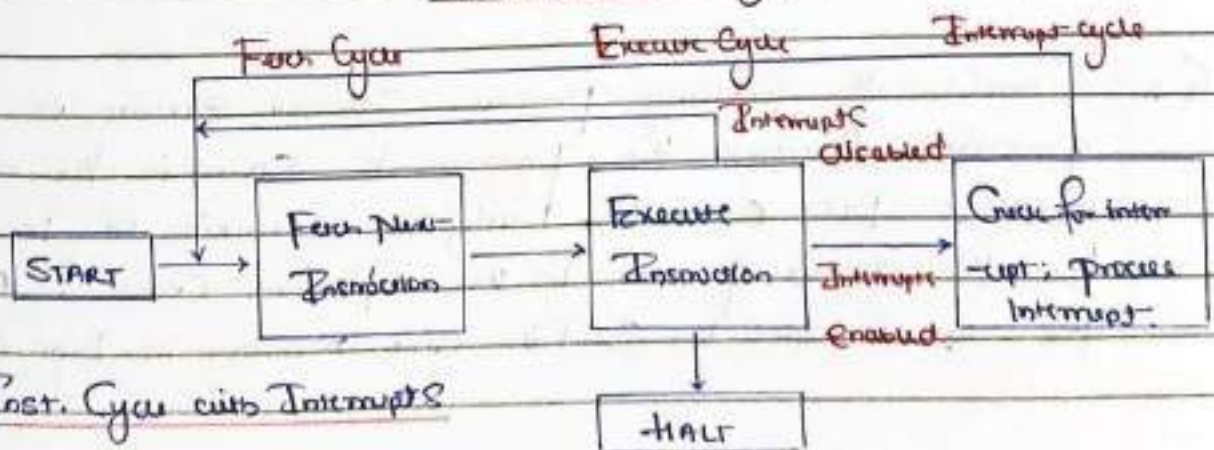
1. IAC (Instruction Address Calculation) { Fetch Cycle
2. IF (Instruction Fetch)
3. Decoding (Analysis of Instruction)
4. OAC (Operand address Calculation)
5. OF (Operand Fetch)
6. DP (Data Processing)
7. Result Storage

Fetus Cycle:

- * Fetch Cycle:
 - * At the beginning of each instruction cycle the processor fetches an instruction from memory.
 - * The Program Counter holds the address of the instruction to be fetched next.
 - * The processor increments the PC after each instruction fetch so that it will fetch the next instruction in sequence.
 - * The fetched instruction is loaded into the instruction register.
 - * The processor interprets the instruction and performs the required action.



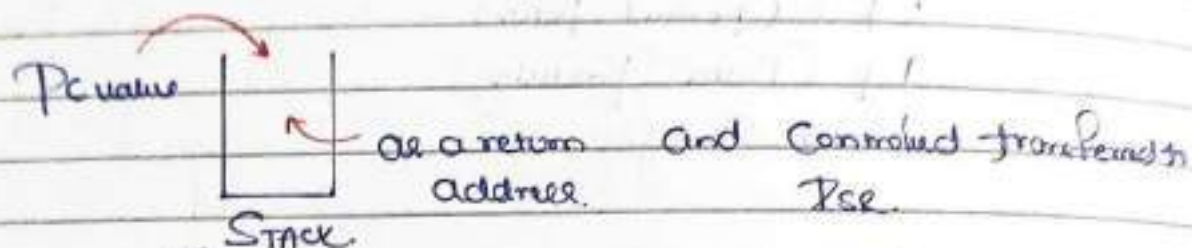
Basic Instruction Cycle



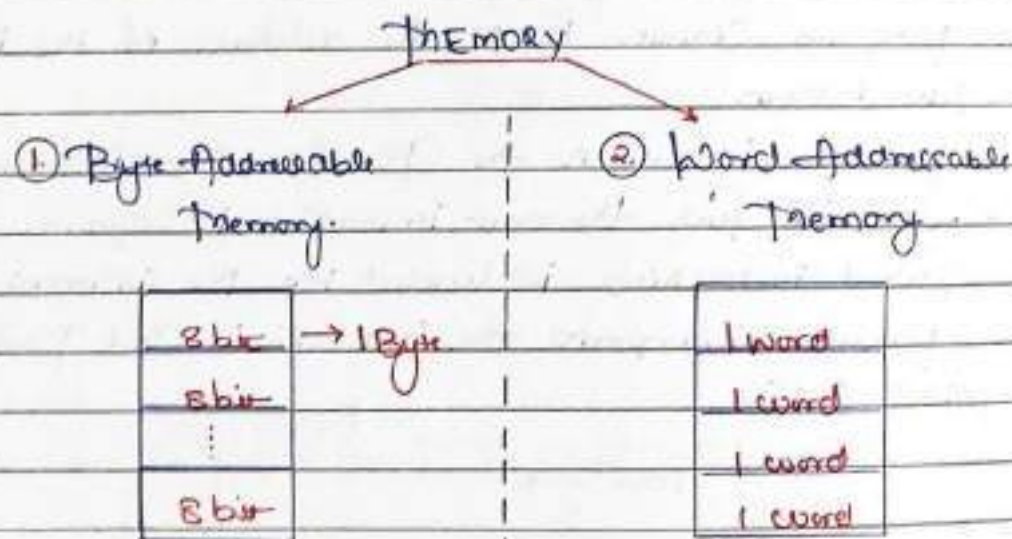
Post Cycle with Interrupts

Note: When interrupt occurs then after completion of current instructions execution, interrupt can be serviced.

* When interrupt occurs, it pushes the PC value into the stack as a return address and control is transferred to ISR (Interrupt Sub Routine).



Stack is used because of "Last in First Out" nature.



1 Word Size = 32 bit

1 Word = 4 Byte

$\therefore$ 1 word = 4 Byte

Q1. Consider the following Program Segment execute on hypothetical processor. Assume the Program is stored in the memory address 1000 onwards. During the execution of it what could be the value present in the Program Counter? Assume the word size is 32 bit and memory is Byte addressable?

Ans.

Instruction	Size (words)
I ₁	2
I ₂	1
I ₃	1
I ₄	3
I ₅	1
I ₆	2
I ₇	1

1 word = 32 bit

= 4 byte

2 words = 2 × 4 = 8 byte

1000 - 1007

1008 - 1011

1012 - 1015

1016 - 1027

1028 - 1037

I₆ - 1032 - 1039

1040 - 1043

Stored in PC during exe. I₆

* Value present in PC during execution of I₆ = 1040

Byte Addressable

Q2.

Consider the following program segment execute on hypothetical processor. Assume that word size is 32 bit and memory is word addressable. The program is stored in the memory at address 1000 onwards. During the execution of I₅ how could be the value present in the program counter?

Ans.

Instruction	Size (words)
I ₁	2
I ₂	1
I ₃	1
I ₄	3
I ₅	1
I ₆	2
I ₇	1

1000 - 1001

1002

1003

1004, 1005, 1006

I₅ - 1007

1008 - 1009

1010

Word Addressable

→ PC will store 1008 (address of new instruction i.e. I₆)

PC will store I₆ address during execution of I₅.

* If interrupt occurs during execution of I₅ or after execution of I₅ then PC will push I₆ value (1008) into stack as return address.

Q3. Consider the following program segment for a hypothetical CPU having three user registers R_1 , R_2 and R_3 .

	Instruction	Operation	Instruction (words)
I_1	MOV $R_1, 5000$	$R_1 \leftarrow \text{Memory}[5000]$	2 1000 - 1001
I_2	MOV $R_2, (R_1)$	$R_2 \leftarrow \text{Memory}[(R_1)]$	1 1002
I_3	ADD R_2, R_3	$R_2 \leftarrow R_2 + R_3$	1 1003
I_4	MOV 6000, R_2	$\text{Memory}[6000] \leftarrow R_2$	2 1004 - 1005
I_5	HALT	Machine Halt	1 1006

Consider that the memory is word addressable with size 32 bits and the program has been loaded starting from memory location 1000. If an interrupt occurs during ADD instruction, what will be the return address pushed into stack?

Ans. Interrupt occurs at ADD (I_3), $\therefore$ PC will store I_4^{th} address $\Rightarrow$ 1004

Q4. Consider the following program segment for a hypothetical CPU having three user registers R_1 , R_2 , R_3 .

	Instruction	Operation	Instruction (words)
I_1	MOV $R_1, 5000$	$R_1 \leftarrow \text{Memory}[5000]$	2 1000 - 1001
I_2	MOV $R_2, (R_1)$	$R_2 \leftarrow \text{Memory}[(R_1)]$	1 1002 - 1003
I_3	ADD R_2, R_3	$R_2 \leftarrow R_2 + R_3$	1 1004 - 1005
I_4	MOV 6000, R_2	$\text{Memory}[6000] \leftarrow R_2$	2 1006 - 1007
I_5	HALT	Machine Halt	1 1008 - 1009

Consider that the memory is Byte addressable with size 32 bits and the program has been loaded starting from memory location 1000. If an interrupt occurs while the CPU has been halted after the executing "MOV 6000, R_2 " instruction, the return address saved in the stack will be

Ans. MOV 6000, R_2 = I_4 instruction interrupted, so I_5^{th} instruction address will be stored in stack $\Rightarrow$ 1008

Q5. Consider the following Program Segment for hypothetical Cu

Instruction	Meaning	Instruction Size (words)	
I ₁ mov r ₀ , 300	$r_0 \leftarrow M[300]$	3	$3 \times 3 + 4 = 13$
I ₂ mov r ₁ , 300	$r_1 \leftarrow M[300]$	3	$3 \times 2 + 4 = 10$
I ₃ mul r ₀ , r ₁	$r_0 \leftarrow r_0 \times r_1$	1	$1 \times 3 + 6 = 9$
I ₄ mov 600, r ₀	$M[600] \leftarrow r_0$	3	$3 \times 2 + 4 = 10$
I ₅ HALT	Machine halt	1	$1 \times 3 = 3$
			<u>51 cycles</u>

Let the clock cycle required for various operations be as follows: Instruction Fetch & Decode: 3 clock cycle per word
True with both operand and stored in register: 6 clock cycle
Register to/from memory transfer: 4 clock cycle
The total number of clock cycle required to execute the program is 51 cycle.

Q6. Consider the following Program Segment for a hypothetical Op having three users registers R₁, R₂ and R₃

Instruction	Operation	Instruction Size	
mov R ₁ , 500	$R_1 \leftarrow \text{Memory}[500]$	2	$2 \times 2 + 3 = 7$
mov R ₂ , (R ₁)	$R_2 \leftarrow \text{Memory}[R_1]$	1	$1 \times 2 + 3 = 5$
add R ₂ , R ₃	$R_2 \leftarrow R_2 + R_3$	1	$1 \times 2 + 1 = 3$
mov 600, R ₂	$\text{Memory}[600] \leftarrow R_2$	2	$2 \times 2 + 3 = 7$
HALT	Machine halt	1	$1 \times 2 + 0 = 2$
			<u>24 cycles</u>

Let the clock cycle required for various operations be as follows:
Register to/from memory transfer: 3 clock cycle
Add with both operand in register: 1 clock cycle
Instruction fetch and decode: 2 clock cycle per word
Total no of clock cycle required to execute this program 24

System Bus:

System bus is a collection of lines which are used to provide the communication between major components of the computer (Memory, I/O, CPU)

System Bus Contains 3 type of bus/lines

- ① Address line / Address bus
- ② Data line / Data bus
- ③ Control line / Control bus

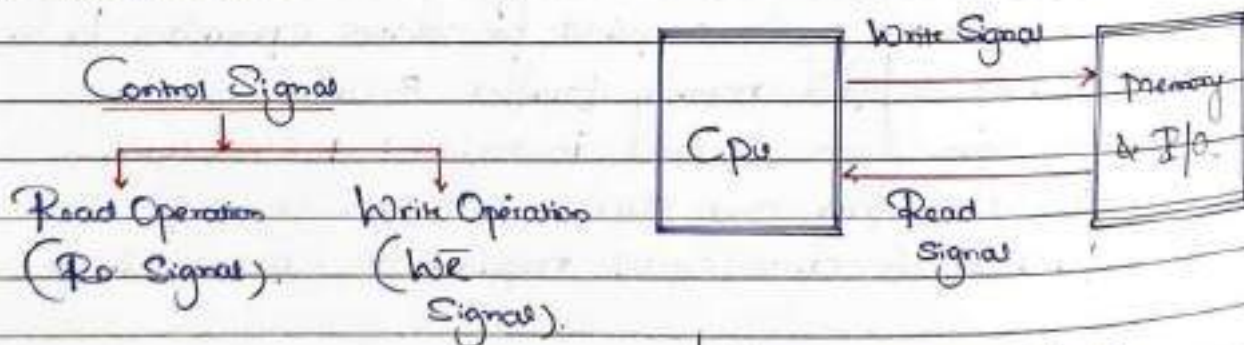
- ① Address Line: Address line are used to carry the address towards memory and I/O.
 * Address line are unidirectional.



- ② Data Line: Data line are used to carry the data (Binary Sequence). Data line are bi-directional.



- ③ Control Line: Control line are used to control the carry signal. Control lines are individually unidirectional and collectively bidirectional.

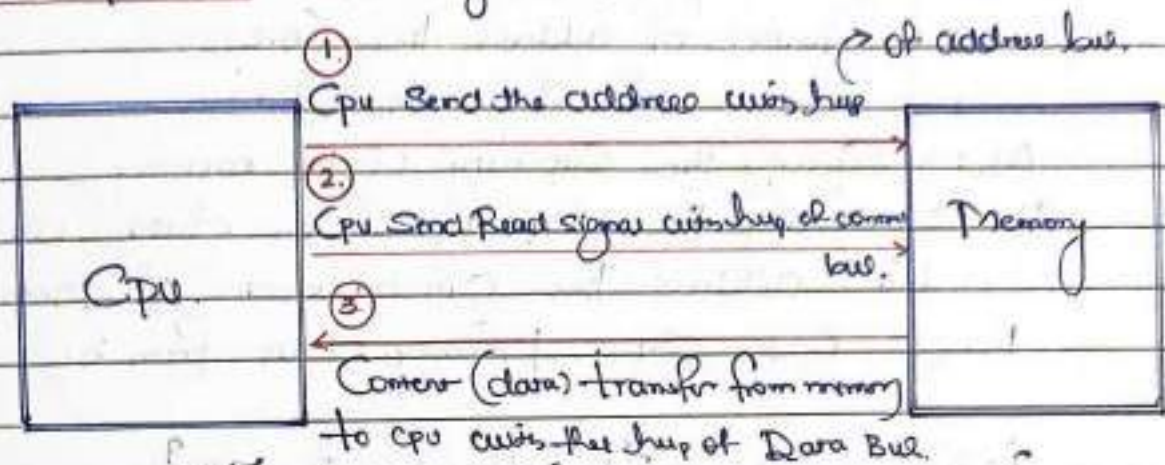
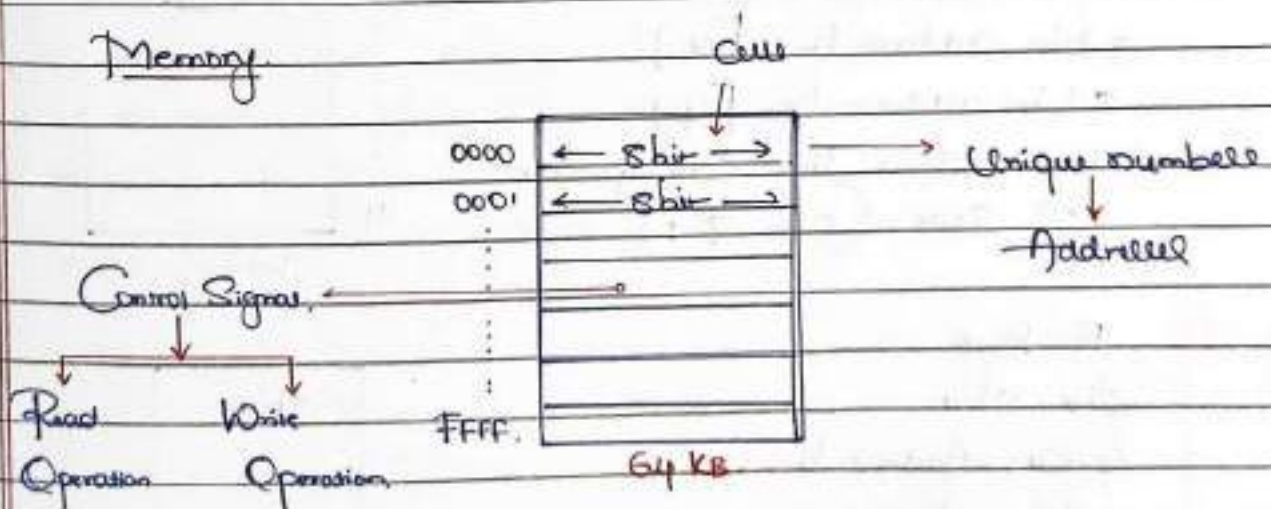
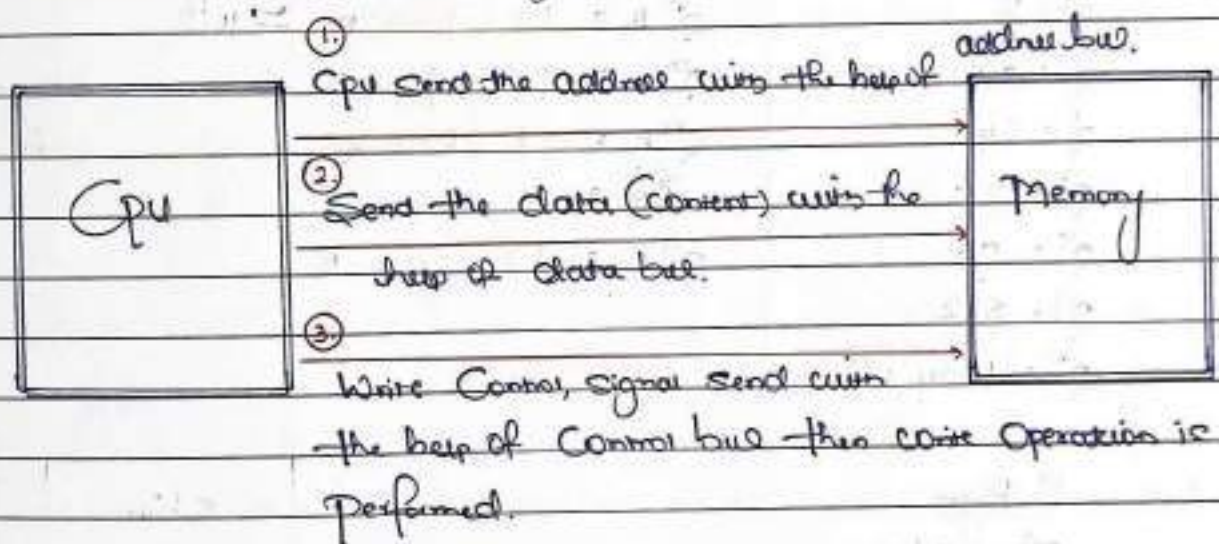


Memory Read: Memory to Cpu

Memory Write: Cpu to Memory

LOAD: Memory Read
(LD)

STORE: Memory Write
(ST)

Read Operation: (Memory Read)Write Operation: (Memory Write)

Memory is organized into equal parts, each part is called cell. Each cell is identified by a unique number called as address.

→ Memory is represented as 2^m

n : number of address line (A.L.)

m : number of data line (D.L.)

AL → Specify the Capacity of the memory

DL → Specify the Capacity of the data (cell size)

* n bit address line can represent 2^n Memory cells

Range: 0 to $2^n - 1$. Memory starts from 0.

$$2^1 = 2$$

$$2^2 = 4$$

$$2^3 = 8$$

$$2^4 = 16$$

$$2^5 = 32$$

$$2^6 = 64$$

$$2^7 = 128$$

$$2^8 = 256$$

$$2^9 = 512$$

$$2^{10} = 1024 [1K]$$

$$2^{10} B = 1K (kilo) B \approx 10^3$$

$$2^{20} B = 1M (mega) B \approx 10^6$$

$$2^{30} B = 1G (giga) B \approx 10^9$$

$$2^{40} B = 1T (Tera) B \approx 10^{12}$$

$$2^{50} B = 1P (Peta) B$$

$$2^{60} B = 1E (Era) B$$

$$2^{70} B = 1Z (Zetta) B$$

$$2^{80} B = 1Y (Yona) B$$

1 Byte = 8 bit

1 Nibble = 4 bit

eg. ①

8 Byte

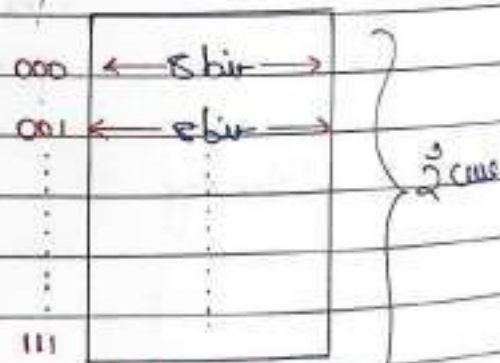
$$2^3 \times 8 \text{ bit}$$

3 bit address line [AL]

8 bit address line [DL]

3 bit address line can represent

2^3 Cells (0 to $2^3 - 1$)



eg. ②

16 Byte

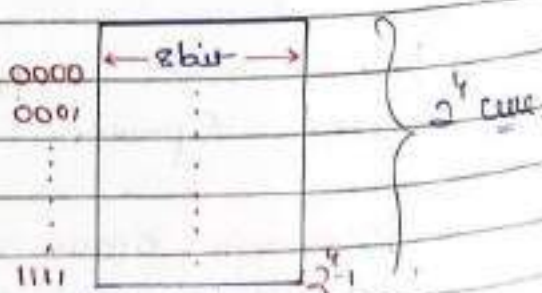
$$2^4 \times 8 \text{ bit}$$

4 bit address line

8 bit data line

4 bit A.L. can represent

2^4 memory cells (0 to $2^4 - 1$)



Q1. The capacity of a memory unit is defined by the number of word multiplied by number of bit/word. How many separate address and data lines needed for memory of $64K \times 16$?

Ans. $64K \times 16$
 $2^6 \cdot 2^{10} \times 16 = 2^{16}$
 $\therefore 16 \text{ Bit-Addressed and } 16 \text{ Bit-data lines}$

Q2. Consider a system which has $1024K$ words. Each word has a size of 32 bits then what is the Capacity of memory in Tera (Mega Byte) — 4 MB

$1024K \text{ words}$ $1 \text{ Word} = 32 \text{ bits}$
 $\Rightarrow 4 \text{ Bytes}$
 $2^0 \times 2^{10} \times 2^2 \text{ Byte}$
 $2^{23} \text{ Byte} = 2^2 \times 2^{20}$
 $= 4 \text{ Mega Byte}$

→ Based on the Cell Size, memory configuration is divided into 2 types:

- (1) Byte addressable memory
- (2) Word addressable memory

Byte-Addressable Memory: When the cell size is '8 bit' then corresponding address is called Byte addressable memory.

(2¹) 4×8 : 2 bit address

(2²) 16×8 : 4 bit address

(2³) 64×8 : 6 bit address

→ $2^n \times 8$: 'n' bit address. (Cell size is 8 bit)

Word-Addressable Memory: When the cell size is given in the form of words (word length) then corresponding address is word addressable memory.

$2^n \times W$: n bit address (each cell size must be a word)

Microprocessor	Byte-Addressable [B]	Word-Addressable	
8 bit processor	8 bit	8 bit	
16 bit processor	8 bit	16 bit	1 Word = word length
32 bit processor	8 bit	32 bit	
...	...	...	
n bit processor	n bit	n bit	
	Unique meaning ↓ No ambiguity	multiple meaning ↓ Ambiguity	

Note:

* Default Configuration in memory is Byte addressable so in the memory data always stored Byte wise.

* Default data type in the CPU is word so operations are performed on a CPU based on word format.

* So to synchronise the memory data type (Byte) with a CPU data type (word), memory interfacing will be adjusted by the designer according to the word length of the CPU to access the data from memory to CPU in the form of words.

eg: 16 bit processor

Word length = 16 bit

Operation are performed on 16 bit data

Q1. The Capacity of a memory unit is defined by the number of word multiplied by the number of bit/word. How many separate addresses and data lines are needed for memory of 64Kx16?

Ans $2^6 \times 2^{10} = 64K \times 16$

$2^6 \times 16$

16 bit address line

16 bit data line

Q2. The Capacity of a memory unit is defined by the number of

word multiplied by the number of bit/word. How many separate addresses and data lines are needed for a memory of $4K \times 16$?

Ans $4K \times 16 \Rightarrow 2^{12} \times 16$
 $\therefore$ 12 bit address line
 16 bit data line.

Q3. A processor can support a maximum of 4GB memory, where the memory is word addressable (a word consists of 2 bytes). The size of the address bus of the processor is at least 31 bits.

Ans

$4GB$
 $\swarrow \quad \searrow$
 $2^9 \times 2 \text{ Byte} \quad \quad \quad \frac{4GB \text{ words}}{2}$

2^9 Words
 $2^9 \times 2^{30} \text{ words}$
 $2^{31} \text{ Word} \Rightarrow \underline{31 \text{ bit}}$

Memory: word addressable
 1 Word: 2 Byte
 2 Byte $\Rightarrow$ 1 Word

Q4. Consider a 32 bit hypothetical processor which support 128 MByte memory. System is enhanced (new design) with a word addressable memory. Then how many address lines are required in the new system?

Ans $128 \text{ MB} \Rightarrow 32m \times 4 \text{ Byte}$
 $32m \text{ Words}$
 $2^{25} \text{ Word} \Rightarrow \underline{25 \text{ bit Address line}}$

Byte Addressable	Word Addressable
$2^{27} \text{ Byte (128 MB)}$ $\downarrow$ $2^{25} \times 4 \text{ Byte}$ 2^{25} Words or $32m \text{ Words}$	$2^{25} \text{ Words (32 mWords)}$ $1 \text{ Word} = 4 \text{ Byte}$ $2^{25} \times 4 \text{ Byte}$ 2^{27} Byte 128 MByte

Q5. Consider a 64 bit hypothetical processor which support 2G word memory. System is enhanced (new design) with a Byte addressable memory. Then how many extra address lines are required in the new system?

Ans	<u>Old design:</u> 2G Words $2^4 \cdot 2^{30}$ Words 2^{34} Words <u>Address = 34 bit</u>	<u>New design: Byte addressable</u> 1 Word = 64 bit = 8 Byte 2G Words $\rightarrow 2G \times 8$ Byte 16G Byte = $2^4 \cdot 2^{30}$ = 2^{34} Byte <u>Address = $\frac{34}{8}$ bit</u>
-----	---	--

$$\therefore \text{Extra} = 34 - 31 \\ = \underline{\underline{3 \text{ bit}}}$$

Pins: Processor contain set of hardware pins to perform operations

1. Active low pin
2. Active high pin
3. Dual pin
4. Time multiplexed pin

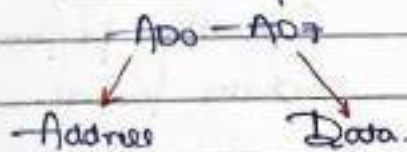
① Active pin: This pin is enabled when the input is '0' or clock pulse is in low state. It is denoted as pin name
eg: $\overline{RD}$, $\overline{WR}$ etc.

② Active high pin: This pin is enabled when the input is '1' or clock pulse is in high state.
eg: $\overline{JMTA}$, $\overline{HIOA}$, $\overline{ALE}$

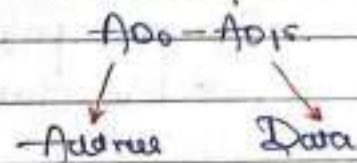
③ Dual pins: eg: $M/\overline{IO}$
 1: then memory operation (Read or write)
 0: then IO operation (IO Read or IO write)

4. Time Multiplexed Pins: This pin carries the multiple meaning but one only at a time. Address pins are time multiplexed with data pins to carry the data and address (But only one at a time).

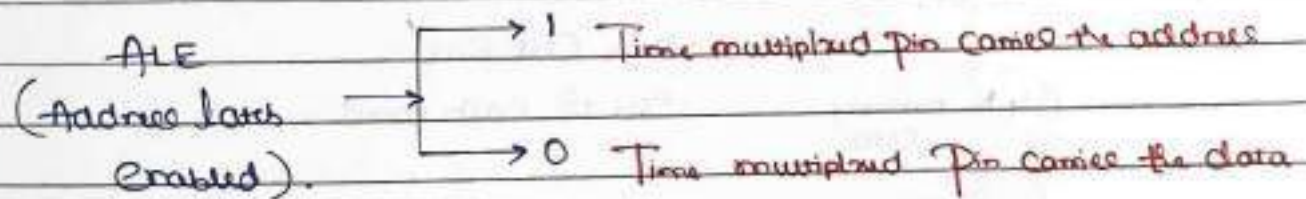
eg: In 8085 Processor



eg: In 8086 Processor



In 8085 A0-A7: Address pins are time multiplexed with data pins to carry the data but to only once at a time.



* The advantage is number of hardware pins reduced.

System Bus:

1. Address Line: Address lines are used to carry the address towards memory and I/O (Unidirectional).

Based on the address line we can determine capacity of the memory.

eg: In 8085 processor

A0-A7 and A8-A15

Address = 16 bit.

= 2^{16} cells, 64K cells

= 64KB

eg: In 8086 processor

A0-A15 & A16-A19

Address = 20 bit.

= 2^{20} cells

= 1M cells

2. Data line: Data lines are used to carry the data (bidirectional). Based on the data line we can determine the word length of the CPU (processor).

The performance of the processor is measured by word length of the CPU.

eg: In 8085

- AD₀ - AD₇

Word length = 8 bit

Operation performed on 8 bit data format.

eg: In 8086

- AD₀ - AD₁₅

Word length = 16 bit

Operation performed on 16 bit data format.

3. Control lines: → Carries the Control Signal
→ Unidirectional (Individually)

Q1.

64K × 8

↓
64K memory cells

→ each cell size/
Size of each word

(2¹⁶ cells)

∴ 16 bit required to represent any of the cells

Width of address bus = $\lceil \log_2 \text{memory size} \rceil = \lceil \log_2 64K \rceil = 16 \text{ bit}$

Width of data bus = 8 bit.

Q2.

4G × 32

Width of address bus = 32 bit

Width of data bus = 32 bit

32 bit processor

Data line = 32 bit

Data bus = 32 bit

Word size = 32 bit

ALU = 32 bit

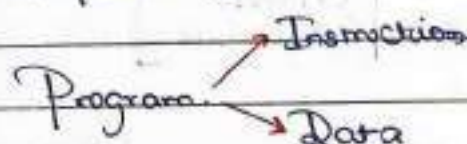
AC = 32 bit

Index/MoE = 32 bit

Registers = 32 bit

Computer: Computer is a Computational device used to process the data under the control of program.

Program: Sequence (Set) of Instructions along with data.



Instruction: It is a binary sequence (code) which is designed inside the processor to perform the actions/operations.

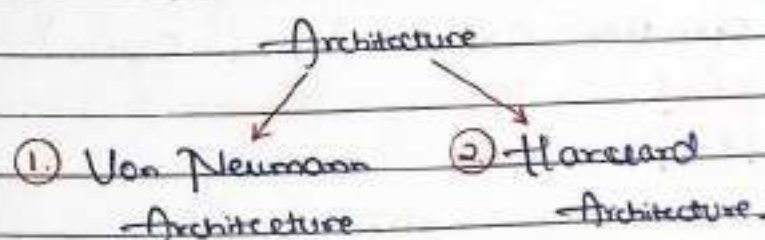
Instruction is a set (Sequence) of binary bits to instruct the computer to perform the operations.

eg: Assume in processor

0110 → Add

1011 → mul

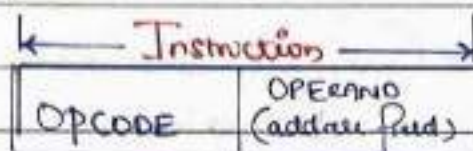
Data: Data is a binary sequence which carries the value (data format: BCD, Hex, etc).



* Computer works on "Stored Program Concept".

Von Neumann Architecture (Stored Program Concept)

- * Main memory contains the instructions and data.
- * All operating on binary data.
- * Control unit interpreting the instructions from memory and executing.
- * Input/Output equipment operated by control unit.

MACHINE INSTRUCTION & ADDRESSING MODESInstruction Format

OPCODE: Operational Code
 ↳ Type of Operation

OPERAND: Data
 or
 Address of Operand

eg: * 2 bit OPCODE Can perform $2^2 = 4$ operations.
 3 bit OPCODE Can perform $2^3 = 8$ operations

n bit OPCODE Can perform 2^n operations.

If memory size is given then we can calculate address field size.

eg: if memory = 1M Byte $\rightarrow$ 20 bit Address Line.

Q. If instruction size is 16 bits & memory is 1K Byte then how many number of operations supported by the system?

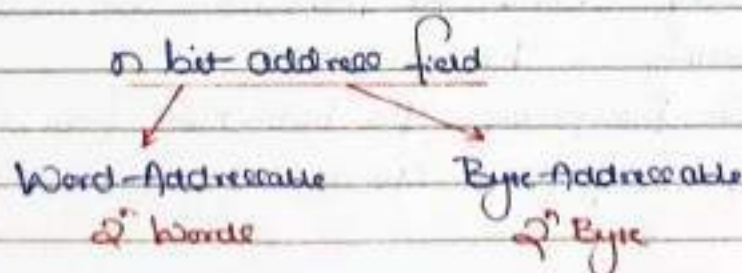
Ans. Instruction: 16 bit Memory = 1K Byte

OPCODE	A.F
--------	-----

 6 bit 10 bit = 2^{10} Byte
 - AF = 10 bit

If opcode = 6 bit

then total # operations = $2^6 = 64$ operations.



Important Points:

- * If n bit Opcode can perform 2^n operations.
- * If N operations then Opcode = $\lceil \log_2 N \rceil$ bit.
- * If n bit address line can represent memory of capacity 2^n .
- * If memory capacity of m B/w then address field = $\lceil \log_2 m \rceil$ bit.

Q1. If opcode is 6 bit then total # Operations?

Ans. Opcode = 6 bit, total # of operation = $2^6 = 64$.

Q2. If 100 operations supported by the system then size of opcode?

Ans. 100 operation $\approx 2^7$.
Opcode = 7 bit.

Q3. If memory is 256 KB then size of the address field?

Ans. 256 KB = 2^{18} Byte $\therefore$ Address = 18 bit.

Q4. If A.F is 29 bit then size of memory?

Ans. AF = 29 bit. Memory = 2^{29} Byte = 512 MB.

Instruction Representation:

- * Within the computer each instruction is represented by a sequence of bits.
- * The instruction is divided into fields, corresponding to the constituent elements of the instruction.



- * Operands are represented by abbreviations called mnemonics.
eg: Add = Add
Sub = Subtract.

MUL — Multiply

DIV — Divide

LOAD — Load from memory the data.

STORE — Store data to memory.

- * Operands are also represented symbolically.
- * Each symbolic opcode has a fixed binary representation.

Machine Instruction Characteristics:

- * The operation of the processor is determined by the instructions it executes, referred to as machine instructions or computer instructions.
- * The collection of different instructions that the processor can execute is referred to as the processor's instruction set.

Elements of a Machine Instruction:

Operation Code (Opcode): Specifies the operation to be performed. The operation is specified by a binary code, known as the operation code, or opcode.

Source Operand Reference: The operation may involve one or more source operands, that is, operands are inputs for the operation.

Result Operand Reference: The operation may produce a result.

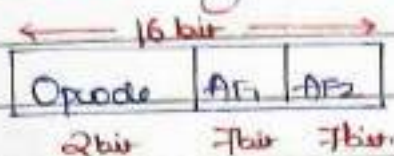
Next-Instruction Reference: This tells the processor where to fetch the next instruction after the execution of this instruction is complete.

- Q1. Consider a hypothetical processor which supports 128 byte memory and instruction length is 16 bit.
- (i) If 2AF is used then how many total number of operations supported?

Ans

128 Byte $\Rightarrow 2^7$ Bytes $\Rightarrow$ AF = 7 bit

Instruction length = 16 bit



if Opcode = 2 bit

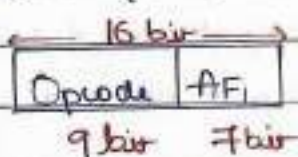
Total # operations = 2^2

= 4 operations

(i) If IAF (address field) is used then how many total number of operations support formulated?

Ans

IAF/IAE

Total # of operations = 2^7

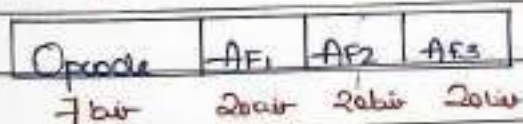
= 128 operations

Q2 A Hypothetical processor support 100 different operations and 3 address memory field (same size). Instruction is stored in 1MB memory. Then what is the length of the instruction?

Ans

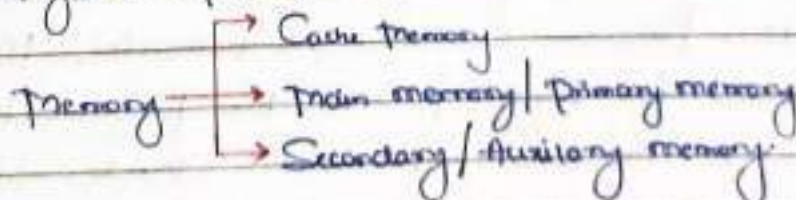
Memory = 1MB (2^{20} B), AF = 20 bit

100 Operations - Opcode = 7 bit

or $\lceil \log_2 100 \rceil = 7 = 100$ Instruction length = $7 + 20 \times 3$
= 67 bit

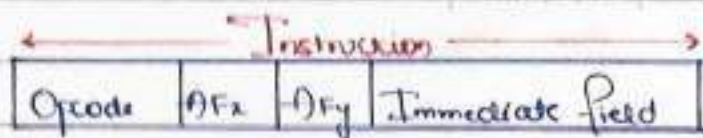
Storage

Registers (Fast)

Fastest: Register $\rightarrow$ C.M. $\rightarrow$ M.M. $\rightarrow$ S.M.Slow: S.M. $\rightarrow$ M.M. $\rightarrow$ C.M. $\rightarrow$ Register.Register $\rightarrow$ Reg. A.F.Memory $\rightarrow$ Mem. A.F.

Q1. Memory 4MB = 2^{20} Byte $\Rightarrow$ Memory-AR = 22 bit.

Q2. 50 Registers = $2^6 \Rightarrow$ Reg-AR = 6 bit.



Sometimes instruction having immediate field also along with RF.

$\rightarrow$ If immediate field = n bit

n bit immediate unsigned range = 0 to $2^n - 1$

n bit immediate signed range = -2^{n-1} to $+(2^{n-1} - 1)$

eg: Assume, if immediate field is 7 bit.

Unsigned = 0 to $2^7 - 1 = 0$ to $2^7 - 1 = 0$ to 127

Signed = $-(2^{7-1})$ to $+(2^{7-1} - 1) = -64$ to 127.

Instruction Set Size of 10

10 Different type of Operation/Instruction performed

Opcode = 4 bit

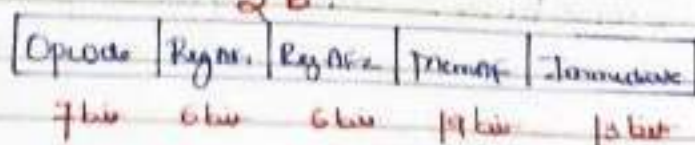
Q1. Consider a hypothetical Cpu which supports 110 instruction so register and 512 KB memory space. Instructions contain 2 register Operands, memory Operands, and 13 bit immediate Constant fields. Program contains 300 instruction. Memory Storage Space required in Bytes to store the Program is 2100 Byte.

Ans.

110 Instructions/Operations = Opcode = 7 bit

50 Register = Reg-AR = 6 bit

512 KB Memory = Mem-AR = 19 bit



$$\text{Instruction Size (Length)} = 7 + 6 + 6 + 19 + 13 = 51 \text{ bit}$$

$$\text{Instruction Size} = \left\lceil \frac{51 \text{ bit}}{8 \text{ bit}} \right\rceil \text{ Byte} = 7 \text{ Byte}$$

Program Contains 300 instructions

$$\text{Program Size} = 300 \times 7 \text{ Byte} = 2100 \text{ Byte}$$

Q2. Consider a processor which contains the following pin structure
 $A_{00} - A_{023}$, $A_{24} - A_{39}$

Processor Contains 250 Registers. Instruction is designed with 4 fields
 i.e. Opcode, register address, memory address and 16 bit
 immediate field

Processor Support 180 instructions (Operations). A program Contains
 400 instructions then how much space is required for the program

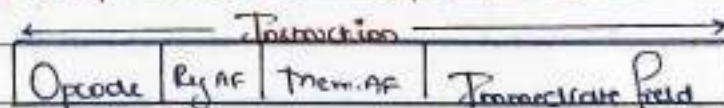
(i) in Byte?

(ii) in Words?

$$\begin{array}{ll} A_{00} - A_{23}, A_{24} - A_{39} = \text{Memory address} = 40 \text{ bit} & \begin{array}{l} A_{00} - A_{39} \text{ Data line} \\ A_{00} - A_{23} = 24 \text{ bit} \end{array} \end{array}$$

$$250 \text{ Register} = \text{Reg. A} = 8 \text{ bit}$$

$$180 \text{ Operation/Instructions} \rightarrow \text{Opcode} = 8 \text{ bit}$$



$$\text{Instruction length (Size)} = 8 + 8 + 40 + 16 = 72 \text{ bit}$$

$$1 \text{ Instruction Size} = \frac{72 \text{ bit}}{8 \text{ bit}} = 9 \text{ Byte}$$

Program Contains = 400 Instructions

$$\text{So Program Size} = 9 \times 400 = 3600 \text{ Byte}$$

Program Size in Word:

$$1 \text{ Word Size} = 24 \text{ bit} = 3 \text{ B}$$

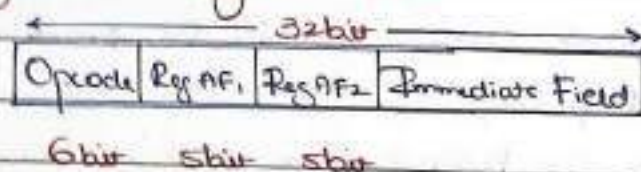
$$\text{Program Size} = 3600 \text{ Byte, in Word} = \frac{3600 \text{ Byte}}{3 \text{ Byte}} \text{ Word}$$

$$= 1200 \text{ Words}$$

Q3. A processor has 40 distinct instructions and 24 general purpose registers. A 32 bit instruction word has an Opcode, two register Operands and an immediate Operand. The number of bits available for the immediate Operand field is

Ans. 40 Instructions $\rightarrow$ Opcode = 6 bit

24 Register $\rightarrow$ Reg-Of = 5 bit



Immediate field = 32 - (6 + 5 + 5)
= 16 bit

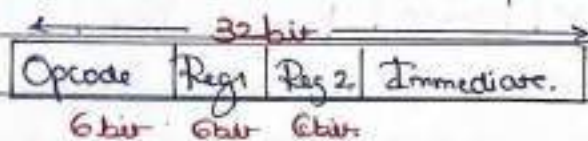
Q4. A machine has a 32 bit architecture, with 1 word long instructions. It has 64 registers, each of which is 32 bit long. It needs to support 45 instructions, which have an immediate Operand in addition to two register Operands. Assuming that the immediate Operand is an unsigned integer the maximum value of the immediate Operand is

Ans. 1 Word long instructions

$\therefore$ 1 Instruction size = 32 bit

64 Register = 6 Reg-Of = 6 bit

45 Instructions/Operations $\rightarrow$ Opcode = 6 bit



Unsigned Immediate field
Maximum = 0 to $2^4 - 1$
= 0 to 16383

$\therefore$ 16383

Immediate field = 32 - (6 + 6 + 6)
= 14 bit

Q5. Consider a processor with 64 registers and instruction set of size twelve. Each instruction has five distinct features fields, namely Opcode, two source register identifiers, one destination register identifier, and a twelve bit immediate value. Each instruction must be stored in memory in a byte-aligned fashion. If a program has 100 instructions, the amount of memory (in bytes)

Consumed by the program - i.e. is

Opcode	Reg1	Reg1/2	Reg1/2	Imm.F
--------	------	--------	--------	-------

4 bit 6 bit 6 bit 6 bit 12 bit

Instruction Size = $4 + 6 + 6 + 6 + 12 = 34 \text{ bits}$

1 Instruction Size = 5 Byte

Program Contains = 100 Inst.

Program Size = $100 \times 5 \text{ B}$
= 500 Byte

Instruction Set Architecture Classification:

1. Stack Organisation
2. Single accumulator
3. General register Organisation.

→ CPU Organisation is classified into 3 types based on the availability of All Operand (Data). (AF: Address Fields or AP: Address Instruction).

1. Stack CPU (OAF)
2. Accumulator CPU (IAF)
3. General purpose Register Organisation
 - (i) Reg-Memory reference CPU (2AF)
 - (ii) Reg-Reg reference CPU (3AF)

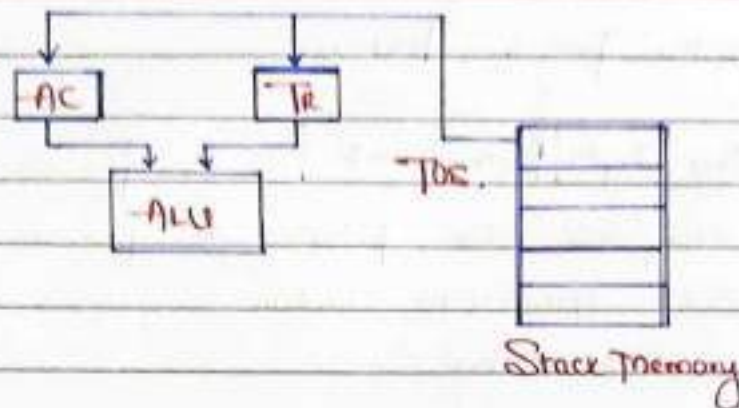
Stack Organisation: [Stack Cpu]

* In the stack based organisation All operations are performed only on stack data.

* So both the Operand (Data) must be required in the stack and after processing result after processing is also stored in the stack.

* Stack is part of memory which is initialised by (Stack pointer) Sp register.

* In stack insert and delete operations are performed at the same low/end called Top (Top of Stack) so it became default location.



Push $\rightarrow$ Insert } Stack Specific
 Pop $\rightarrow$ Delete }

Load $\rightarrow$ Memory Read } Memory Specific
 Store $\rightarrow$ Memory Write }

In $\rightarrow$ I/O Read } I/O Specific
 Out $\rightarrow$ I/O Write }

Mov $\rightarrow$ General with all combination.

eg: $(x+y)+2$. How many machine instructions are required to execute using stack based organisation (stack-cpu)?

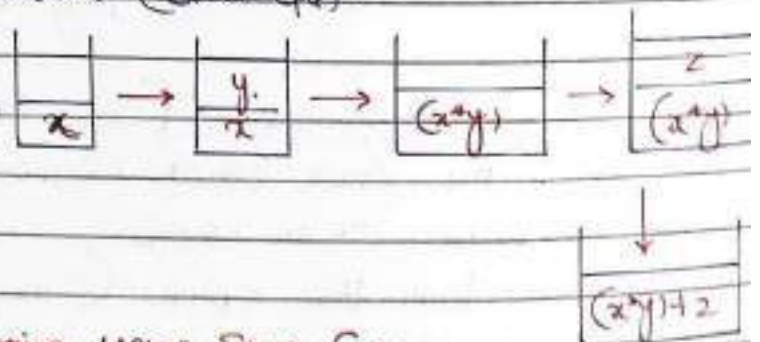
$\rightarrow$ I₁: Push x

I₂: Push y

I₃: Mul

I₄: Push 2

I₅: Add.

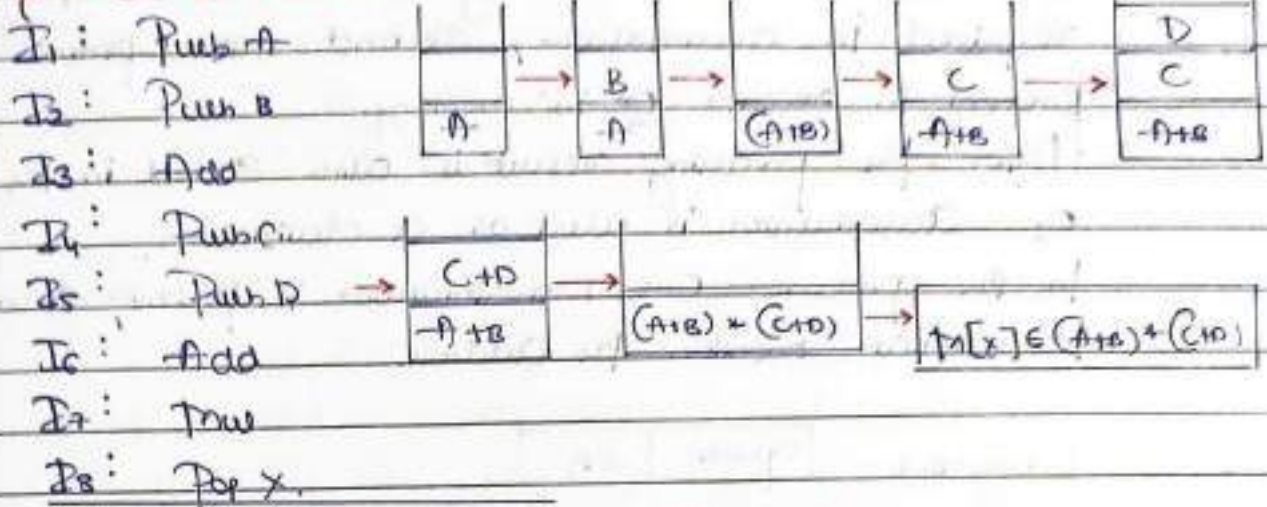


5 Machine Instructions using Stack Cpu.

Note: In the stack-cpu, only ALU operations and data transfer instructions (push & pop) are not OI₁ (zero address instructions).

Q1. How many machine instructions are required using Stack-CPU?

$$X = (A+B) * (C+D)$$



8 Machine Instructions using Stack-CPU.

Q2. Consider a 32-bit hypothetical processor which uses stack CPU. Which support 1 word Opcode and 24-bit address. Following statement is executed on a stack-CPU

$$X = (A+B) * (C+D)$$

Ans. Word size = 32-bit

Opcode = 1 word = 32-bit $\approx$ 4 Byte

Address = 24-bit $\approx$ 3 Byte

Q3. How much memory is required for the program in Byte?

$$X = (A+B) * (C+D)$$

I1: Push A $\rightarrow 4B + 3B = 7$ Byte

I2: Push B $\rightarrow 4B + 3B = 7$ Byte

I3: Add $\rightarrow 4B = 4$ Byte

I4: Push C $\rightarrow 4B + 3B = 7$ Byte

I5: Push D $\rightarrow 4B + 3B = 7$ Byte

I6: Add $\rightarrow 4B = 4$ Byte

I7: True $\rightarrow 4B = 4$ Byte

I8: Pop X $\rightarrow 4B + 3B = 7$ Byte

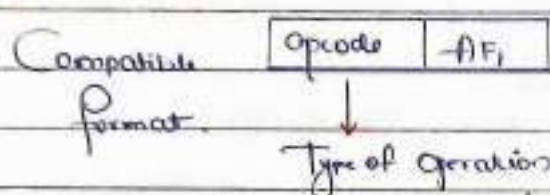
47 Byte

Q. What is the status at the end of the program execution?

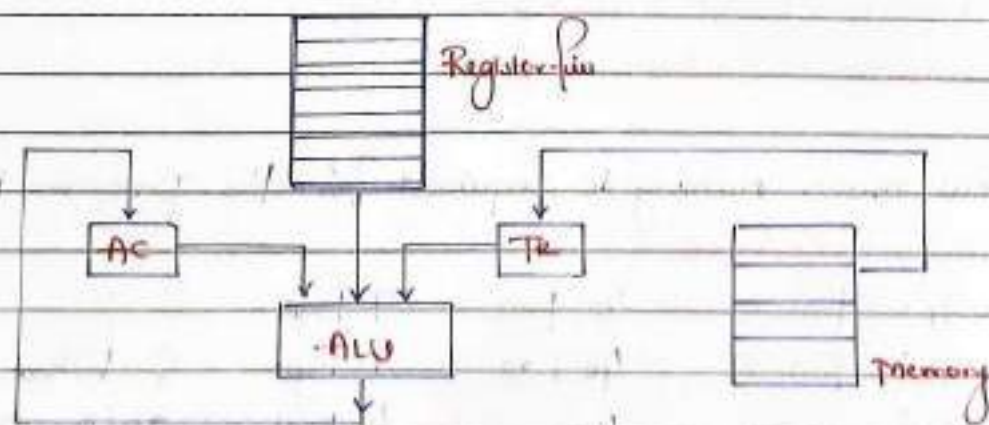
$\rightarrow$ Empty, because result is stored in $Mem[X]$

Single Accumulator Organisation:

- In Accumulator based Organisation first ALU operand are required in accumulator, second ALU operand (Data) either present in register or in memory.
- After the processing result is also stored in the accumulator. Accumulator is used as a destination.
- In the processor only 1 accumulator is present, so not needed to explicit mention the address.



	Destination	Source	Source 2
(i) ALU Operation	AC	AC	Reg/Memory
(ii) Data transfer Operation	AC	Memory	(Memory Read)
	Memory	AC	(Memory Write)



Register File is The number of registers supported by the processor is denoted (called) register file.
 eg: Register File 16
 Contains 16 register file.

Q1. $(A+B)$ - A and B are the variable in the memory. How many machine instructions are required to execute using Ac-Cpu?

Ans.

I₁: Load A; $Ac \leftarrow M[A]$

I₂: Add B; $Ac \leftarrow Ac + M[B]$

2 machine instructions using Ac-Cpu.

Q2. $(x*y)+2$ - How many machine instructions required using Ac-Cpu?

Ans.

I₁: Load x; $Ac \leftarrow M[x]$

I₂: mul y; $Ac \leftarrow Ac * M[y]$ $Ac \leftarrow x*y$

I₃: Add z; $Ac \leftarrow Ac + M[z]$ $Ac \leftarrow [x*y]+2$

3 Machine instructions using Ac-Cpu.

Q3. $x = (A+B) * (C+D)$ - A, B, C, D and x are variables in the memory.

(i) How many machine instructions required to execute using Ac-Cpu? $\rightarrow 7$

(ii) How many Spill (memory Spill) are required? $\rightarrow 1$

Ans.

I₁: Load A; $Ac \leftarrow M[A]$

I₂: Add B; $Ac \leftarrow Ac + M[B]$

I₃: Store T; $M[T] \leftarrow Ac$

I₄: Load C; $Ac \leftarrow M[C]$

I₅: Add D; $Ac \leftarrow Ac + M[D]$

I₆: mul T; $Ac \leftarrow Ac * M[T]$

I₇: Store x; $M[x] \leftarrow Ac$

$M[T] = (Ans)$

Memory Spill = 1

7 m/c instructions using Ac-Cpu.

General Register Organisation:

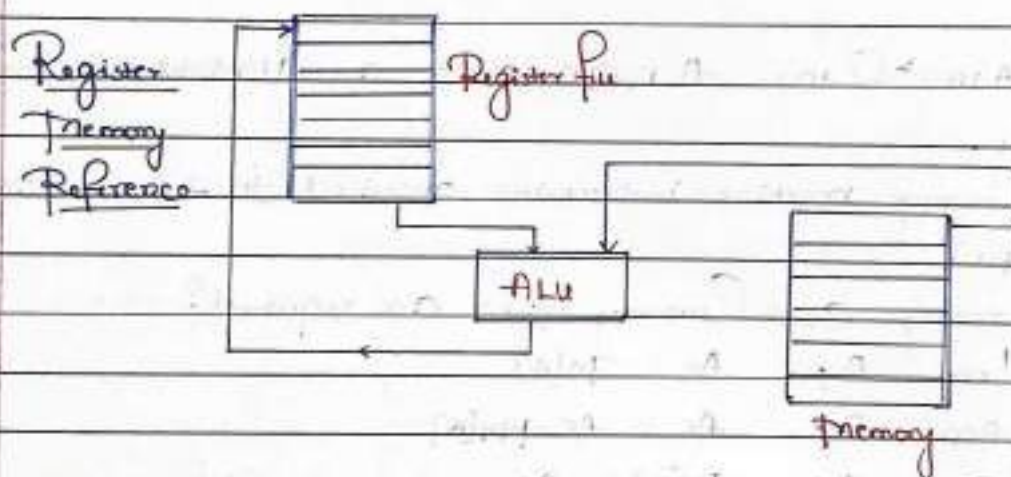
Based on the number of registers (Register file size) supported by the processor this architecture is divided into two types:

① Register-Memory Reference (Reg-Memory Cpu)

② Register-Memory Register Reference (Reg-Reg Cpu)

- ① Register - Memory Reference: This architecture supports less number of registers so register file size is small.

Compatible format:	OpCode	AF ₁	AF ₂
	Destination	Source ₁	Source ₂
ALU Operation	Reg(SI)	Reg	Reg/Memory
Data Transfer Operation	Reg Memory Reg	Mem Reg Reg	} Mov



eg: (4+6) using Register memory reference: How many r/mc instructions are required?

I₁: Mov r0 A: r0 ← m[A]

I₂: Add r0 B: r0 ← r0 + m[B]

2 Machine Instructions (using Register Memory Reference)

eg: (x*y)+2: using Register-Memory reference Cpu?

I₁: Mov r0 x: r0 ← m[x]

I₂: Mul r0 y: r0 ← r0 * m[y]

I₃: Add r0 2: r0 ← r0 + m[2]

3 Machine Instructions

eg: $x = (A+B) * (C+D)$ - A, B, C, D, E are variable in memory, then how many machine instructions are required using Reg-memory reference?

I₁: MOV r0 A;

I₂: ADD r0 B;

I₃: MOV r1 C;

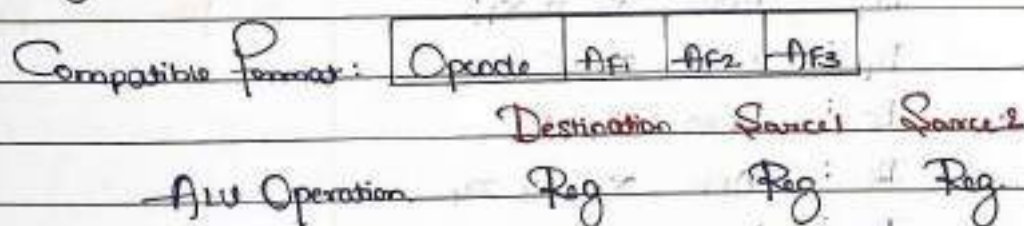
I₄: ADD r1 D;

I₅: MOV r0 r1;

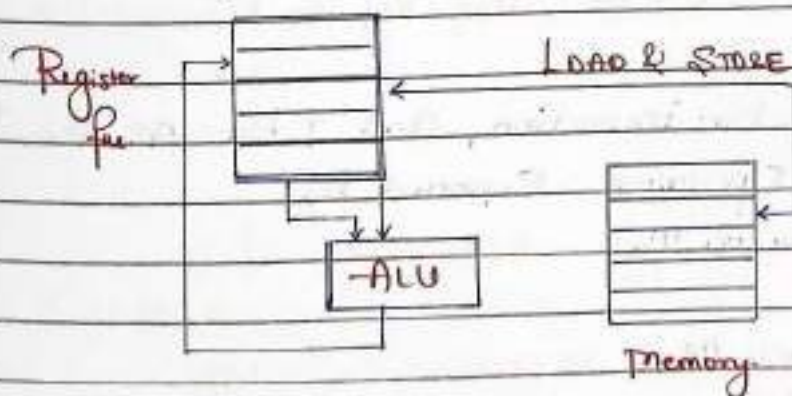
I₆: MOV x r0;

6 m/c Instructions using Reg-memory reference.

(2) Register-Register Reference: This architecture: Supports more number of registers.



* In this organisation All operand always required in registers.



eg: (A+B) using Reg-Reg reference Cpu?

I₁: LOAD r0 A; $r0 \leftarrow M[A]$

I₂: LOAD r1 B; $r1 \leftarrow M[B]$

I₃: ADD r2 r0 r1; $r2 \leftarrow r0 + r1$

3 Machine Instructions using Reg-Reg Reference.

eg: $(x * y) + z$ Using reg-reg reference?

I₁: Load r0 x;

I₂: Load r1 y;

I₃: MUL r2 r0 r1;

I₄: Load r3 z;

I₅: ADD r4 r2 r3;

5 m/c Instructions using Reg-Reg reference

eg: $x = (A * B) * (C * D)$ - Here A, B, C, D only are variable in memory
How many machine instructions are required using Reg-Reg
Cpu?

I₁: Load r0 A;

I₂: Load r1 B;

I₃: ADD r2 r0 r1;

I₄: Load r3 C;

I₅: Load r4 D;

I₆: ADD r5 r3 r4;

I₇: MUL r6 r2 r5;

I₈: Store x r6

How many minimum no. of registers required?

3 minimum Reg required.

8 m/c Instructions (Using Reg-Reg Reference)

Q1. Consider a 16 bit instruction, and 7 bit R. How many
- total no. of operations supported by:

2AF $\rightarrow$ $\left[\begin{array}{c|c|c} \text{Opcode} & R_1 & R_2 \\ \hline \text{7bit} & \text{7bit} & \text{7bit} \end{array} \right]$ # operation = $2^2 = 4$

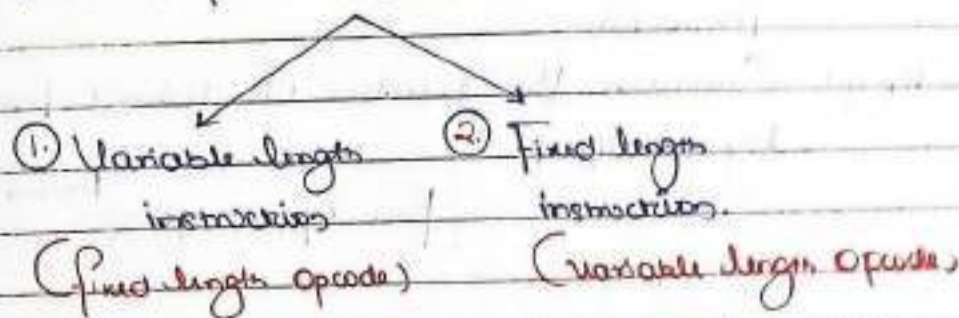
1AF $\rightarrow$ $\left[\begin{array}{c|c} \text{Opcode} & R_1 \\ \hline \text{7bit} & \text{7bit} \end{array} \right]$ # operation = $2^1 = 2$

0AF $\rightarrow$ $\left[\begin{array}{c} \text{Opcode} \\ \hline \text{16bit} \end{array} \right]$ # operation = $2^6 = 64$

3AF/3A2 $\left[\begin{array}{c|c|c|c} \text{Opcode} & R_1 & R_2 & R_3 \\ \hline \end{array} \right]$ $\rightarrow$ (It can support upto 3AF/3A2
it can support 2AF, 1AF, 0AF)

Expand Opcode Technique:

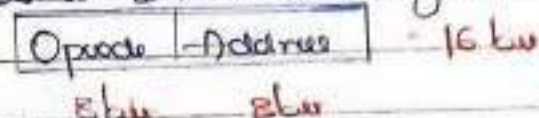
Expand opcode length is required in the fixed length instructions. Supported Cpu design to implement the various instructions with different formulae (formats).

Variable Length Instruction Supported Cpu Design

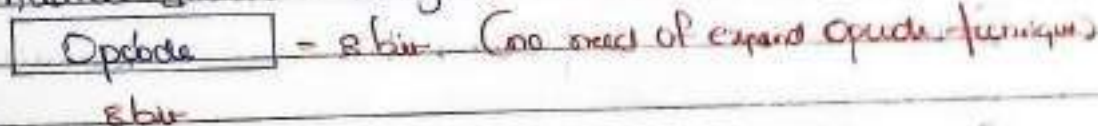
Opcode = 8 bit

Address field = 8 bit

(i) 1 Address Instruction Design:



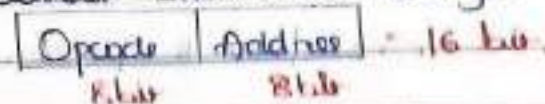
(ii) 0 Address Instruction Design:

Fixed length Instruction Cpu Supported Cpu Design

Opcode = 8 bit

AF = 8 bit

(i) 1 Address Instruction Design:



(ii) 0 Address Instruction Design:



Expand Opcode Technique: We start from primitive instructions.

- ① Primitive Instruction (lowest bit in opcode)
- ② Derived Instruction (higher opcode bit)
- ③ Further derived instruction (highest opcode bit).

Expand Opcode Technique:

Step 1: Identify the primitive instruction in the Cpu.

Step 2: Calculate the total number of possible operation.

Step 3: Identify the free opcode after allocating the existing instruction.

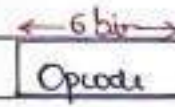
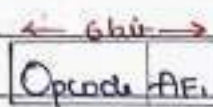
Step 4: Calculate the number of desired instruction possible by multiply

Free opcode $\times 2^{\text{Increment bit in opcode}}$

Q1. Consider a processor which support 6 bit instruction and 4 bit address field. If there exist 2 one address instruction then how many 0 address instruction can be formulated?

Ans. Instruction length = 6 bit

AF = 4 bit



Step 1:

Primitive Inst.

Desired Inst.

Step 2: Total no. of operations/instruction in IAF = $2^4 = 16$

Step 3: Number of free opcode after allocating IAF = $16 - 2 = 14$ Free

Step 4: Total # operation in OAF = Free opcode $\times 2^{\text{Inc. bit in opcode}}$
 $= 14 \times 2^2$
 $= 28$

Q2. Consider a processor which contains 8 bit word and 256 word memory. It support 3 word instruction. If there exist 24 2-address instruction and 256 1-address instruction then how many 0-address instruction can be formulated?

Ans. 1 word = 8 bit = 1 Byte

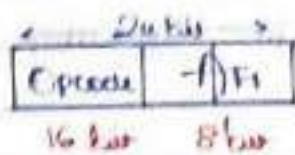
Memory = 256 word = $256 \times 1 \text{ Byte} = 256 \text{ Byte}$

Memory = 256 Page (2B) $\rightarrow$ 2B = 2Kb

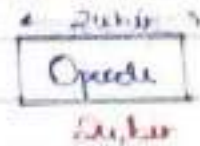
Instructions size = 3 word $\rightarrow$ $3 \times 8\text{Kb} = 24\text{Kb}$



Primitive Inst.



Derived Inst.



Further Derived Inst.

(I) Primitive:

Total no. of operation in 2AF (primitive inst.) = $2^8 = 256$ Oper. ins.

Given $2AF/2AF = 254$

free Op code after allocating 2AF = $256 - 254 = 2$

(II) Derived:

Total # operations = Free op code $\times 2$ (Increment bit in op code)
 In 1AF = $2 \times 2^{16-8}$
 = 512 Operations

Given 1AF (AF) = 256

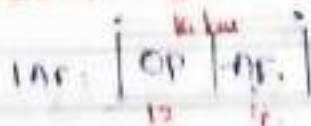
free Op code after allocating 1AF = $512 - 256 = 256$

(III) Further Derived:

Total # Operations = Free op code $\times 2$ (Increment bit in op code)
 in OAF = $256 \times 2^{24-16}$
 = 256×2^8
 = 2^{16} or 64K

Consider a processor with 16 Kb instructions. Processor has 12 registers and support 2 address instruction and 1 address instructions. If processor support 256 1 address instruction then number of 2 address instructions are

16 Register $\rightarrow$ Register = 4 Kb, Instructions size = 16 Kb



Primitive: Total no. of operation in 2AF = 2^8

Assume 2AI/2AF Instruction/Operation = x

Free Opcode after allocating 2AI = $[2^8 - x]$

Operation in

Increment bin in opcode

$$1AF/1AI = \text{Free Opcode} \times 2$$

$$= (2^8 - x) \times 2^{12-8}$$

$$2^8 = (2^8 - x) \times 2^4 \Rightarrow 2^8 / 2^4 = 2^8 - x$$

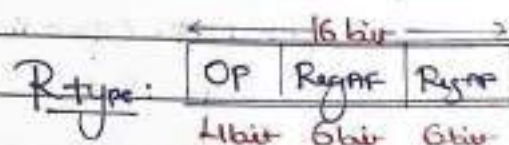
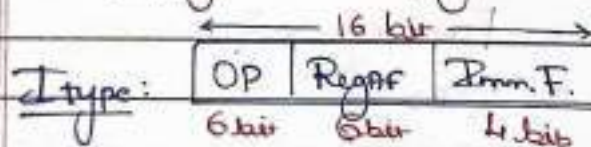
$$2^4 = 2^8 - x$$

$$16 = 256 - x$$

$$x = 256 - 16 \Rightarrow \underline{240}$$

Q4. A processor has 64 registers and uses 16 bit instructions. It has two types of instructions: I-type and R-type. Each I-type instruction contains an opcode, a register name, and a 4 bit immediate value. Each R-type instruction contains an opcode and two register names. If there are 8 distinct I-type opcodes, then the maximum number of distinct R-type opcodes is 14

Ans 64 Register $\Rightarrow$ Reg AF = 6 bit Instruction Size = 16 bit



$$16 - (6+4) = \underline{6 \text{ bit}}$$

$$16 - (6+6) = \underline{4 \text{ bit}}$$

Total # Operation in R-type = 2^4

Assume Rtype instruction = x

Number of Free Opcode after allocating R type = $2^4 - x$

$$\# \text{ operations in I-type} = (2^4 - x) \times 2^{6-4} \Rightarrow 8 = (2^4 - x) \times 2^2$$

$$8 = (2^4 - x) \times 4 \Rightarrow x = 16 - 2 \Rightarrow \underline{14}$$

Q5. A processor has 16 registers ($R_0, R_1, \dots, R_{15}$) and 64 floating point registers ($F_0, \dots, F_{63}$). It uses a 2 Byte instruction format. There are four categories of instructions: Type-1, Type-2, 3, 4.

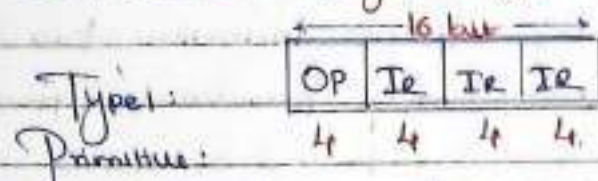
Type-1 Category consists of 4 instructions, each with three integer operands (3R). Type-2 Category consists of eight instructions, each with two floating point register operands (2F). Type-3 Category consists of fourteen instructions, each with one integer operand and one floating point register operand (1R+1F). Type-4 Category consists of N instructions, each with a floating point register operand (F). The maximum value of N is _____.

Ans

16 Registers $\Rightarrow$ R Registers = 16 bit

64 floating registers: F = 6 bit

Instructions = 2 Byte = $2 \times 8 = 16$ bit

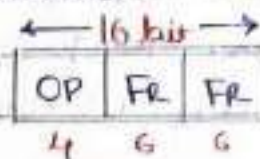


$$\text{Total \# operations} = 2^4 = 16$$

$$\text{Free in Type-1} = 16 - 4 = 12$$

Type-2:

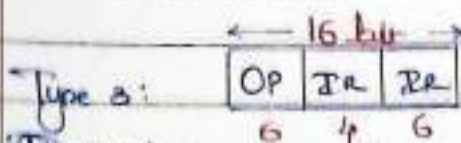
Primitive:



$$\text{Total \# operations} = 2 \times 2^{4-4} = 12$$

Given = 8

$$\text{Free} = 12 - 8 = 4$$



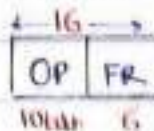
Derived:

$$\begin{aligned} \text{\# operations} &= 4 \times 2^{6-4} \\ &= 4 \times 2^2 = 16 \end{aligned}$$

Given = 14

$$\text{Free} = 16 - 14 = 02$$

Type-4: Further Derived:

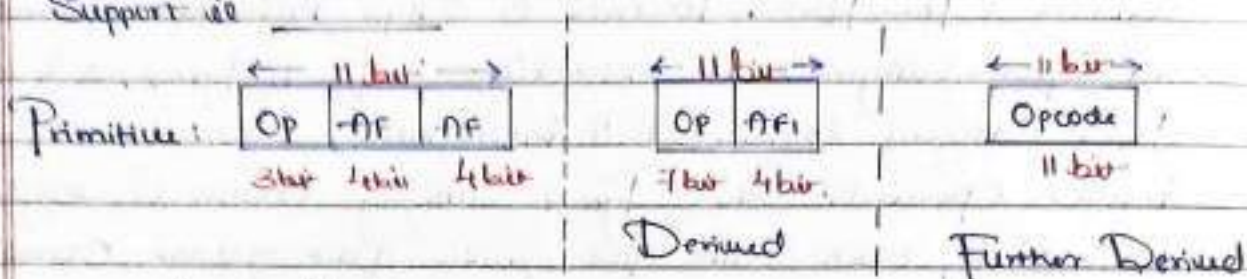


$$\begin{aligned} \text{\# operations} &= 2 \times 2^{16-6} = 2 \times 2^4 \\ &= 32 \end{aligned}$$

Q6. Consider a processor with 16 instructions. The size of address field is 4 bit. The compiler uses expanding opcode technique and has '5' two (2) address instructions and '32' one (1) address

instruction. Then the number of zero address instruction it can support is

Ans



$$Q^1 = 5 \times Q^0 \times Q^0 + 32 \times Q^0 + 'x'$$

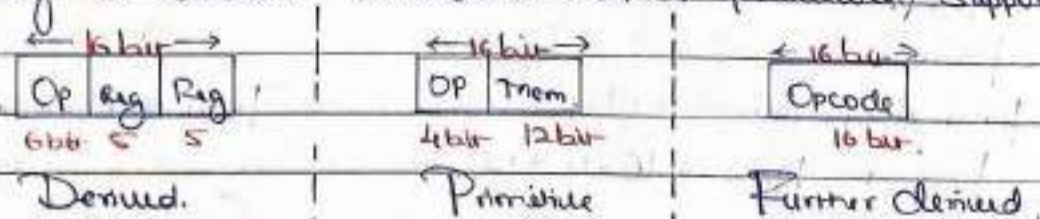
$$Q^0 = 5 \times Q^0 + Q^0 + 'x'$$

$$Q^0 = Q^0 (5+1) + 'x'$$

$$x = \underline{256}$$

Q7. Consider a 16 bit hypothetical processor which support 1 word long instruction. Processor has 32 registers and 4Kb of memory size. If there exist '2' address register reference instruction and '10' 1 address memory reference instruction. Then how many '0' address instructions can be formulated/supported?

Ans



Primitive: Total # Operations = $2^4 = 16$

Given memory reference = 10

$$\text{Free} = 16 - 10 = 6$$

Derived:

$$\text{Total \# Operations} = 6 \times 2^{6-4} = 6 \times 2^2 = 24$$

Given = 11

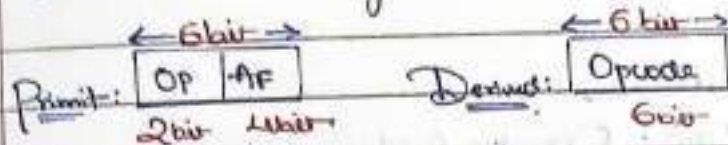
$$\text{Free Opcode} = 24 - 11 = 13$$

$$\text{OAF} = 13 \times 2^{6-6}$$

$$= 13 \times 2^0 = \underline{13}$$

Q. Consider a processor which support 6 bit instructions and 4 bit address field. If there exist 2 one address instructions then how many 0 address instructions can be formulated?

Ans.



operation in 1AF/1AI = $2^2 = 4$

Given 1AI = 2

Free = $4 - 2 = 2$

Total # operations in OAI

= $2 \times 2^{6-2}$

= $2 \times 2^4 = 32$

Addressing Modes

* Addressing is a technique used to calculate the effective address (EA)
or

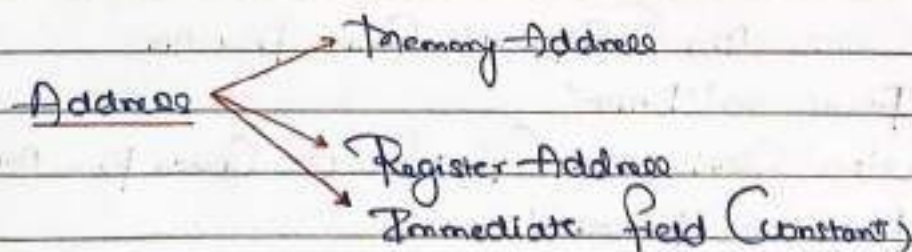
* Addressing mode shows the way where the required object is present
or

* Addressing mode shows the way how to get operand.

* Effective Address [EA] is the actual address of the object.

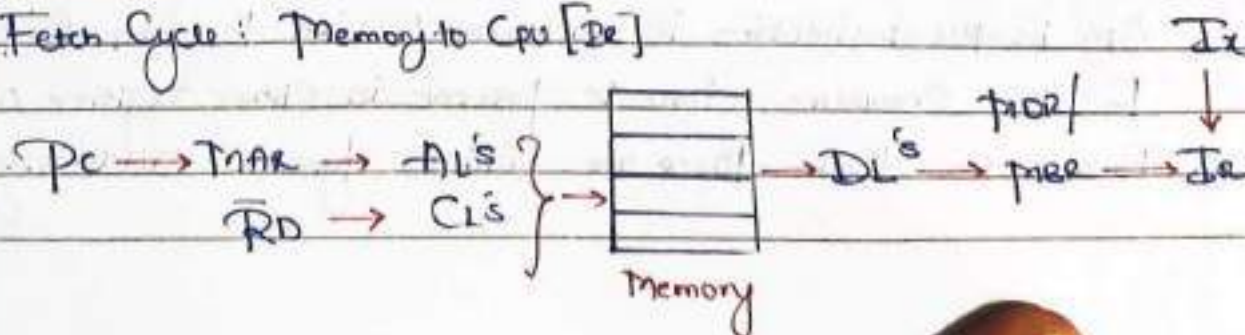
* Object may be instructions or data.

* The output of addressing mode is effective address.



When Am?

Fetch Cycle: Memory to CPU [De]



Fetch Cycle:

IR [Opcode | mode | AF]

Execute Cycle

- Decode
- Execute

Decodes

OpC: Operator Address Calculation

OpF: Operand Fetch

DP & RS: Data Processing & result Storage

Why - Am?

Opcode | Address

- Register (Direct or Indirect)
- Memory (Direct or Indirect)
- Constant (value)

Opcode | mode field | AF

* Mode field helps you to get operand.

- Immediate Constant
- Register (Direct/Indirect)
- Memory (Direct/Indirect)

Addressing Modes: Addressing mode shows the way where the required object is present or location of required object.

① Data Centric - Am (Sequential Control Flow Am)

→ Focus on 'Data'

② Instruction Centric - Am (Transfer of Control Flow Am)

→ Focus on 'Instructions'

- * Am in the instruction is implemented with the help of 'mode field'.
- * In the computer, data is present in either register or memory. Based on this there are various type in addressing mode.

1. Immediate - Am
2. Direct / Absolute - Am
3. Memory Indirect - Am
4. Register Direct - Am
5. Register Indirect - Am
6. PC-Relative - Am
7. Based Register - Am
8. Indexed Register - Am
9. Implied / Implicit - Am
10. Auto Decrement - Am
11. Auto Increment - Am
- 12.

Symbols of Am.

I or # $\longrightarrow$ Immediate Am

[] $\longrightarrow$ Direct Am

[()] or @ $\longrightarrow$ Indirect Am

Reg name $\longrightarrow$ Register Am

Index Reg Name $\longrightarrow$ Indexed Reg. Am

- ① Immediate Am: In this addressing mode Operand are present in the address field of the instruction or In this Am operand are present in the instructions itself.

Ex:

Opcode	Address
--------	---------

 $\xrightarrow{\text{Data}}$ Immediate Am are used to access the Constant and initialise the register with Constant value.

Immediate Am has Some Limitations:

Immediate Am can never be used as a destination address. It can be used only as source address, because destination must be required storage and constant does not have any storage.

eg: $\text{ADDI R}_1, \text{R}_1, 500$ ✓
 $\text{R}_1 \leftarrow \text{R}_1 + 500$

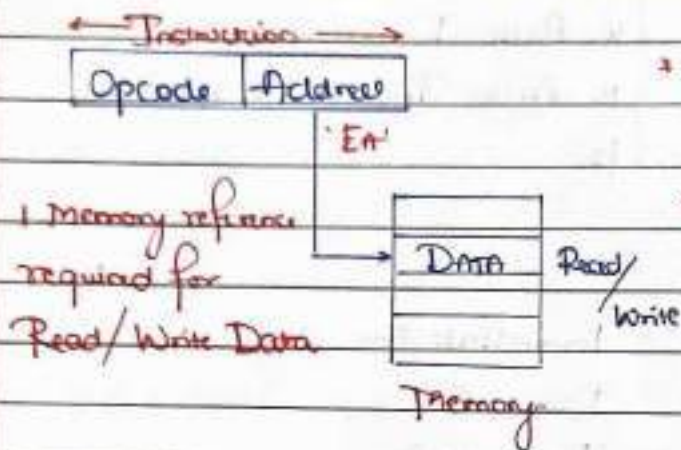
eg: ADDI 500, R_1 ✗
 $500 \leftarrow 500 + \text{R}_1$

The range of constant is limited by address field.

In this immediate field Unsigned range = 0 to $2^n - 1$

Signed range = -2^{n-1} to $+2^{n-1} - 1$.

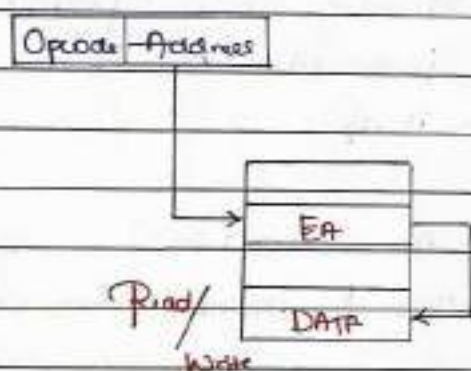
- ② Memory Direct or Absolute Am : In this addressing mode operand are present in the memory and address field of the instruction contain the effective address.



* This Am is used to access the variable

* 1 Memory reference required for read/write data

- ③ Memory Indirect Am : In this Am operand are present in the memory, effective address is also present in memory, instruction contain address of effective address.



* Indirect Am are used to access the pointers

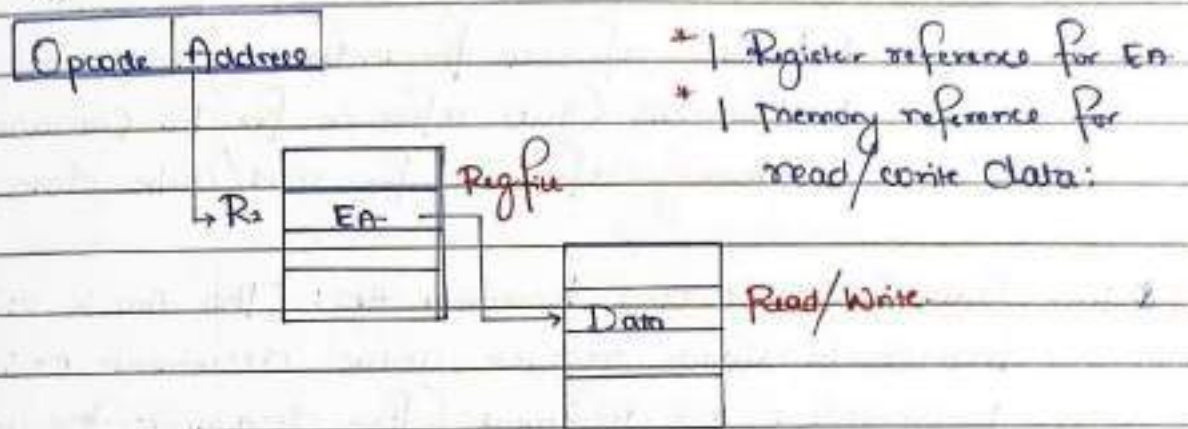
* 2 Memory reference is required in memory indirect Am

- ④ Register Direct Am : This Am is similar to memory direct Am but in this Am operand are present in the registers instead of memory.

Or
In this Am operand are present in the register that register address (register name) maintained in the address field of the instruction.

* 1 Register reference required for read/write data

- (5) Register Indirect - Am: In this -Am operand are present in the memory and effective address is stored in register.



Q. Why register indirect -Am used instead of memory indirect -Am?

Ans. * To shorten the instructions length (size)
 * Accessing the register is very fast compared to memory.
 eg: IP memory = 16GB then memory -Ar = 34 bit then instruction size large but in the processor we have 32 or 64 register so we need 5 or 6 bit respectively.
 So instruction size reduced and faster access.

Displacement Am.

⑥ PC Relative Am.

$$EA = \text{Current PC value} + \text{Ar (offset)}$$

or

$$EA = \text{Current PC value} + \text{Relative value}$$

⑦ Base Register Am.

$$EA = \text{Base Register value} + \text{Ar (offset)}$$

⑧ Index Register Am.

$$EA = \text{Index reg value} + \text{offset (Ar)}$$

↓
Array implementation

Index - Am $\rightarrow$ (Array implementation)

$$EA = \text{Index Reg value} + \Delta E (\text{offset})$$

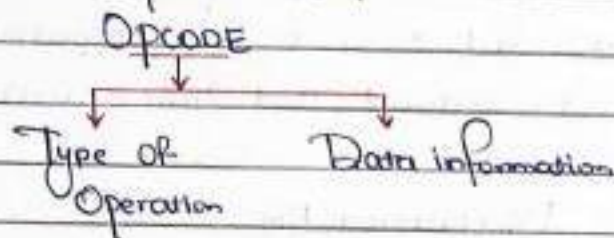
- * 1 Register reference for index register value
- * 1 Arithmetic (AU) reference for EA calculation
- * 1 memory reference for read/write data.

⑨, ⑩ Auto Decrement and Increment Am: This Am is similar to register indirect in which register value decrement or increment.

Decrement: Pre decrement (first decrement the value (in register), then access the data)

Increment: Post increment (first access the value/data then increment in register value)

⑪ Implied / Implicit Am: In this Am operand (data info) are present in the opcode itself.



* Stack based organization used in implicit/implicit Am

Note:

- Constant $\rightarrow$ Immediate Am
- Variable $\rightarrow$ Direct Am
- Pointer $\rightarrow$ Indirect Am
- Array $\rightarrow$ Index Am
- Reallocation $\rightarrow$ Base Reg Am

Q1. In which of the following addressing mode, operand is not a part of the instruction?

Ans Direct
Indirect
Register

Eight addressing modes for the load instructions:

Mode	Assembly Convention	Register Transfer
Direct address	LD ADR	$AC \leftarrow M[ADR]$
Indirect address	LD @ADR	$AC \leftarrow M[M[ADR]]$
Relative address	LD #ADR	$AC \leftarrow M[PC + ADR]$
Immediate operand	LD #NBR	$AC \leftarrow NBR$
Index addressing	LD ADR(R)	$AC \leftarrow M[ADR + R]$
Register	LD R1	$AC \leftarrow R1$
Register indirect	LD (R1)	$AC \leftarrow M[R1]$
Auto increment	LD (R1)+	$AC \leftarrow M[R1] \quad R1 \leftarrow R1 + 1$

Q2. Consider a 16 bit hypothetical processor which support 1 address instructions design with various addressing modes. It contains 8 bit opcode. It supports 1 word instructions stored in the memory with a starting address of $(745)_{10}$ onwards. Address field value of the instruction is 222. Register R1 contains memory content of $[222]$ is 155. Which of the following is/are correct about effective address of various Addressing Mode (Am)?

I In the immediate Am effective address is 745

II In the immediate Am effective address is 746

III In the memory indirect Am effective address is 155

IV In the index Am effective address is 333 (R1 as index register).

Ans. 16 bit processor

Instruction Size = 1 word = 16 bits

Register (R1) = 111

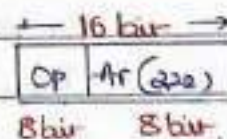
Immediate Am: EA = 746

Memory indirect

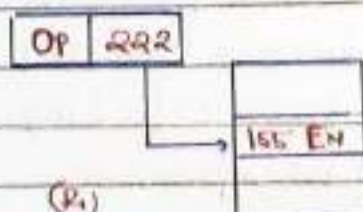
EA = 155

Index Am

$$EA = Index R1 + AF \\ = 111 + 222 = 333$$



745	Opcode
746	Ar = 222
PC: 745	Next Inst.
222	155



Q3. Match the following pairs:

- | | | |
|------------------------------|---|--------------|
| 1. Indirect addressing | → | 1. Loops |
| 2. Immediate addressing | → | 2. Pointers |
| 3. Auto decrement addressing | → | 3. Constants |

Q4. If we use internal data forwarding to speed up the performance of a CPU (R_1, R_2 and R_3 are registers and $m[100]$ is a memory reference), then the sequence of operations.

$R_1 \rightarrow m[100]$

$m[100] \rightarrow R_2$

$m[100] \rightarrow R_3$ Can be replaced by

Ans $R_1 \rightarrow R_2$ R_2, R_3 and $m[100]$ contain updation by R_2 content

$R_1 \rightarrow R_3$

$R_1 \rightarrow m[100]$

Q5. Match the following:

- | | | |
|---------------------|---|------------------------|
| A. $A[i] = B[j];$ | → | 1. Indirect addressing |
| B. $while (*A++);$ | → | 2. Indirect addressing |
| C. $int temp = *x;$ | → | 3. Auto increment |

Q6. The memory locations 1000, 1001, and 1020 have data values 18, 1 and 16 respectively. Before the following program is executed.

MOV	R_1	Move immediate
LOAD	$Rd, 100(R_1)$	Load from memory
ADDI	$Rd, 100$	Add immediate
STOREI	$0(Rd), R_1$	Store immediate

Ans

$MOV[1000] = 18$

$MOV[1001] = 1$

$MOV[1020] = 16$

$I_1 = MOV R_1, 1$

$R_1 = 1$

$I_2 = LOAD Rd, 100(R_1)$

$Rd \leftarrow m[1000 + R_1]$

$Rd \leftarrow m[1000 + 1]$

$Rd = 1$

$I_3 = ADDI Rd, 100$

$Rd \leftarrow Rd + 100$

$Rd = 100$

$I_4 = \text{Store } I, 0(\text{rd}), 20$

$M[0+Rd] \leftarrow 20$

$M[1001] \leftarrow 20$

$\therefore$ Memory location 1001 has value 20

Q3 The absolute addressing mode:

Ans The address of the operand is inside the instruction.

Q4 A CPU has 24-bit instructions. A program starts at address 300 (in decimal). Which one of the following is a legal program counter?

Ans 24 bit instruction = 3 Byte

$$300 + 3x = 600$$

$$\therefore 600$$

$$300x = 3x,$$

$$x = \frac{300}{3} = 100$$

Q Why Auto increment, decrement is used?

Ans When we want to access or perform same operation in multiple locations (assume 100 location) then different instructions are required.

But with the help of auto decrement or auto increment, it is possible with only one instruction we can perform same operation on multiple locations.

Note:

* Index addressing mode: This mode is used to access specific (random) element of the array.

$$A[n] = \text{Starting address} + n \times \text{Size of datatype} \rightarrow \text{Index value}$$

(base address)

* Index value stored in index register.

* Starting address present in the address field of the instruction.

$$EA = \text{Index Reg Value} + AR$$

Index Register

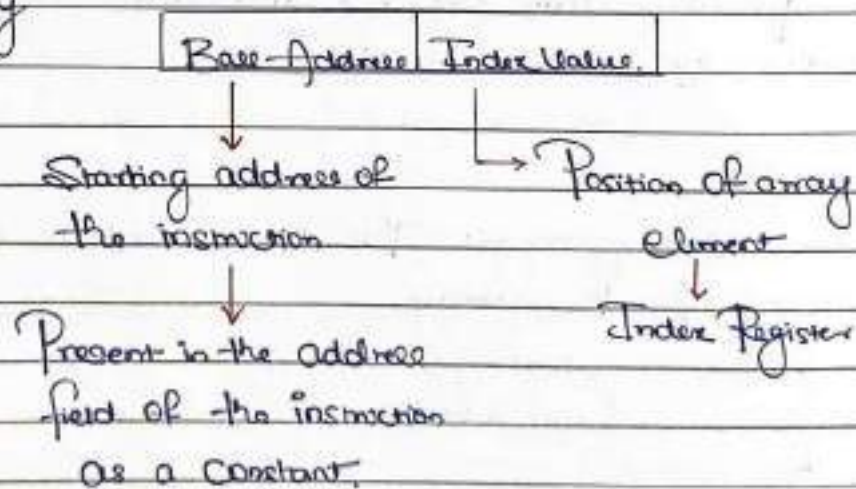
- ① At the Cpu design-time One Special register made as a index register then in this they implicit access the index value from the index register.
(Wastage of one register because we can use that register as general purpose register also.)
- So index register use very less)

② In most of the cases one general purpose register made (designated) as index register. & mentioned that register name as index register.
eg: R₆ as a index register.
We can use that register as general purpose register also.

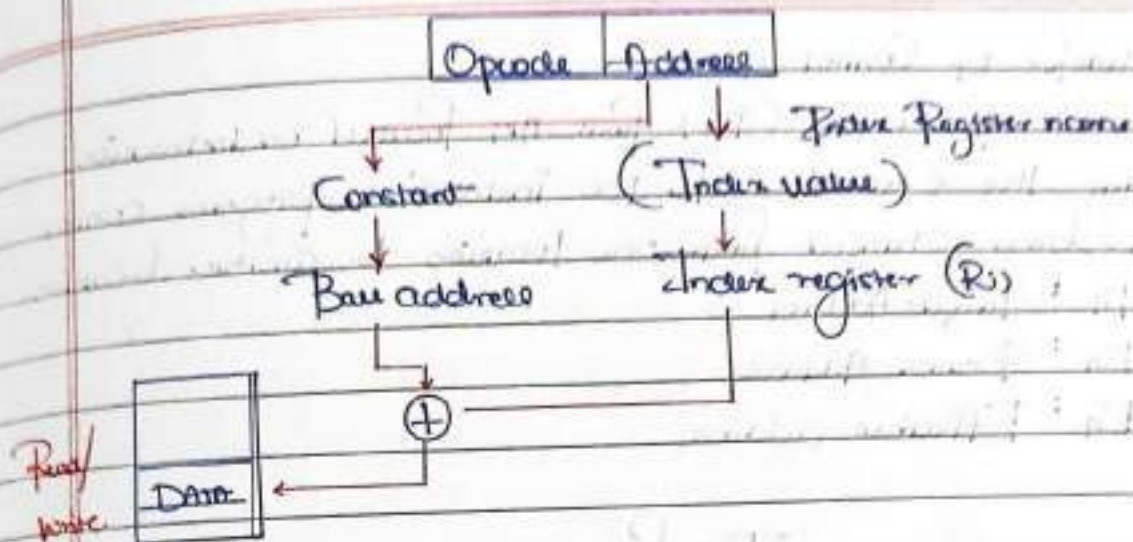
Index Addressing Mode:

1. Index Am
2. Index based Am / Base Index Am
3. Base Index with displacement Am
4. Index indirect Am
5. Auto indexed Am

- ① Indexed Am: Index Am is used to access the random element of array



- * 1 Reg reference for index value
- * 1 Am reference for EA & 1 Memory-ref for read data.



- ② Base Index - Am: Sometime Cpu contain Separate base register and index register to store the base value and index value respectively.

eg: 8086 up

Bx reg $\Rightarrow$ Base register

SI/DI : Index register (Source index / Destination index)

MOV Bx, #1000

MOV Ax [Bx][SI]

MOV SI, #20

$Ax \leftarrow [Bx + SI]$

$Ax \leftarrow M[1000 + 20]$

- ③ Base Index with displacement:

MOV Ax SD([Bx][SI])

$Ax \leftarrow M[(Bx) + (SI) + SD]$

- ④ Indirect Index - Am

1 Reg ref for index value

1 AR ref for EA

1 Memory reference for Read Data



Transfer of Control - Am.

- * Transfer of Control (Toc) flow - Am focused on instructions.
- * During the execution of Toc instructions program control has been changed from one location to another location.
- TA : Target Address
- BA : Branch Address
- EA : Effective address.

Toc flow Am.

① Unconditional Toc

eg: 1. Jmp x (address)
2. HALT

② Conditional Toc

JNZ (jump on not zero)

BNZ (branch on not zero)

BZ (branch on zero)

JZ (jump on zero)

- * Relative value is difference between current PC value and target address.

Relative value = +ve (forward jumping)

Relative value = -ve (backward jumping)

- * PC relative Am:

Target Address = Current PC value + Offset (Relative Value)

Conditional Toc: In Conditional Toc condition checking is done on the status of previous instruction output.

HALT: unconditional Toc instructions in which target address is the starting address of the instruction which it caused HALT instruction.

Branch Instruction

Conditional JOC Instruction

1. Condition: True = PC value updated as target address and now PC execute the target instruction as a next instruction.

Unconditional JOC Instruction

- PC value updated as a target address and now PC execute the target instruction as next instruction.

2. Condition: False = PC value is not updated as a target address & now next Sequential instruction is executed as a next instruction.

PC Relative - Am

$$Ea = \text{Current PC value} + \text{Address field value (Relative value)}$$

- * In this mode Effective Address (Ea) is obtained by adding the relative value to program counter (PC).
- * Relative Value means distance between current location to target location, it is a constant (signed constant), present in the address field of the instruction.
- * PC relative Am is used intra segment transfer of control (branching), when target address is present in same segment then during program execution control will be transferred within the segment called intra segment branching.

Base Register - Am

$$Ea = \text{Base Register Value} + \text{Address Field Value}$$

- Base Register - Am is used inter segment transfer of control (branching), when target address is present in different segment then during program execution control will be transferred between the segments called inter segment branching.

Note: Both PC relative -Am and Base register -Am are suitable for program relocation at runtime.

* Base Register -Am $\rightarrow$ Relocation

* PC Relative -Am $\rightarrow$ Intra Segment branching

Q1. Consider a 4 Byte long PC relative instruction is located in the memory with the starting address of 243048 (Decimal). A -8 sign displacement is present in the address field of the instruction. What is the branch address?

Ans. Instruction Size = 4 Byte 243048

I ₁
I ₂
I ₃
I ₄
Next Instr.

Starting address = 243048 243049

Current PC value = 243051 243052

Displacement = -8 (Backward Jumping) PC $\Rightarrow$ 243052

(Relative value)

$$\begin{aligned} \text{Target address} &= \text{Current PC value} + \text{relative value} \\ &= 243052 + (-8) \\ &= \underline{243044} \end{aligned}$$

Q2. Consider a 4 Byte long jump instructions stored in the word addressable memory with a word size of 16 bits. The starting address of instruction is 900 (decimal). A address field contains -32. Content of base register is 500. What is the branch address (Target address) when the instruction is designed with

- PC relative addressing model
- Base Register addressing model.

Ans. 4 Byte Jump instruction

Memory word addressable

1 word size = 16 bit = 2 Byte

Instruction size = 4 Byte = 2 words

900

I ₁
I ₂
Next Instr.

 } memory

901

902

Relative Value = -32

(i) PC relative Arm.Target address = Current PC + AF (offset)
value

$$= 902 - 32$$

$$= \underline{870}$$

(ii) Base register ArmTarget address = Base register
value + relative value

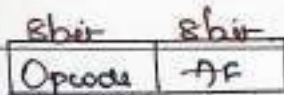
$$= 500 + (-32)$$

$$= \underline{468}$$

Q3. Consider a 16 bit which support 1 word long instruction, stored in the memory with a starting address of 900 (decimal). Instruction format contains 8 bit Opcode and address field. Instruction is designed with PC relative jmp operation. During its execution control (branch) will be transferred to an address 614 (decimal). Then what is!

- What is the relative value in address field of the instruction?
- What is the PC value before instruction fetch, after instruction fetch and after execution phase?

Ans 16 bit instruction = 2 Byte



PC = 802

Target Add = 614

(i) Target address = current pc val + relative val

$$614 = 902 + \text{relative value}$$

$$\text{relative value} = -288$$

(ii) PC value : Before instruction fetch : 900

After instruction fetch : 902

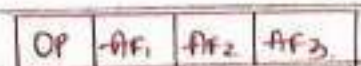
After execution : 614 (Target address)

Q4. For computers based on three-address instruction format, each address field can be used to specify which of the following:

S1 : A memory operand

S2 : A processor register

S3 : An implied accumulator register



Ans Either S1 or S2.

Instruction Set

- ① Data transfer instruction/operation
- ② Data manipulation instruction/operation.
 - (i) Arithmetic (eg. add, sub, div, mul etc)
 - (ii) Logical (OR, AND, XOR, etc)
 - (iii) Shift and rotate operation (Arithmetic & logical shift rotate with & without carry)
- ③ Program Control instruction/operation
 - (i) Unconditional Jc
 - (ii) Conditional Jc

Data Transfer Instructions : Data transferred from one location to another location

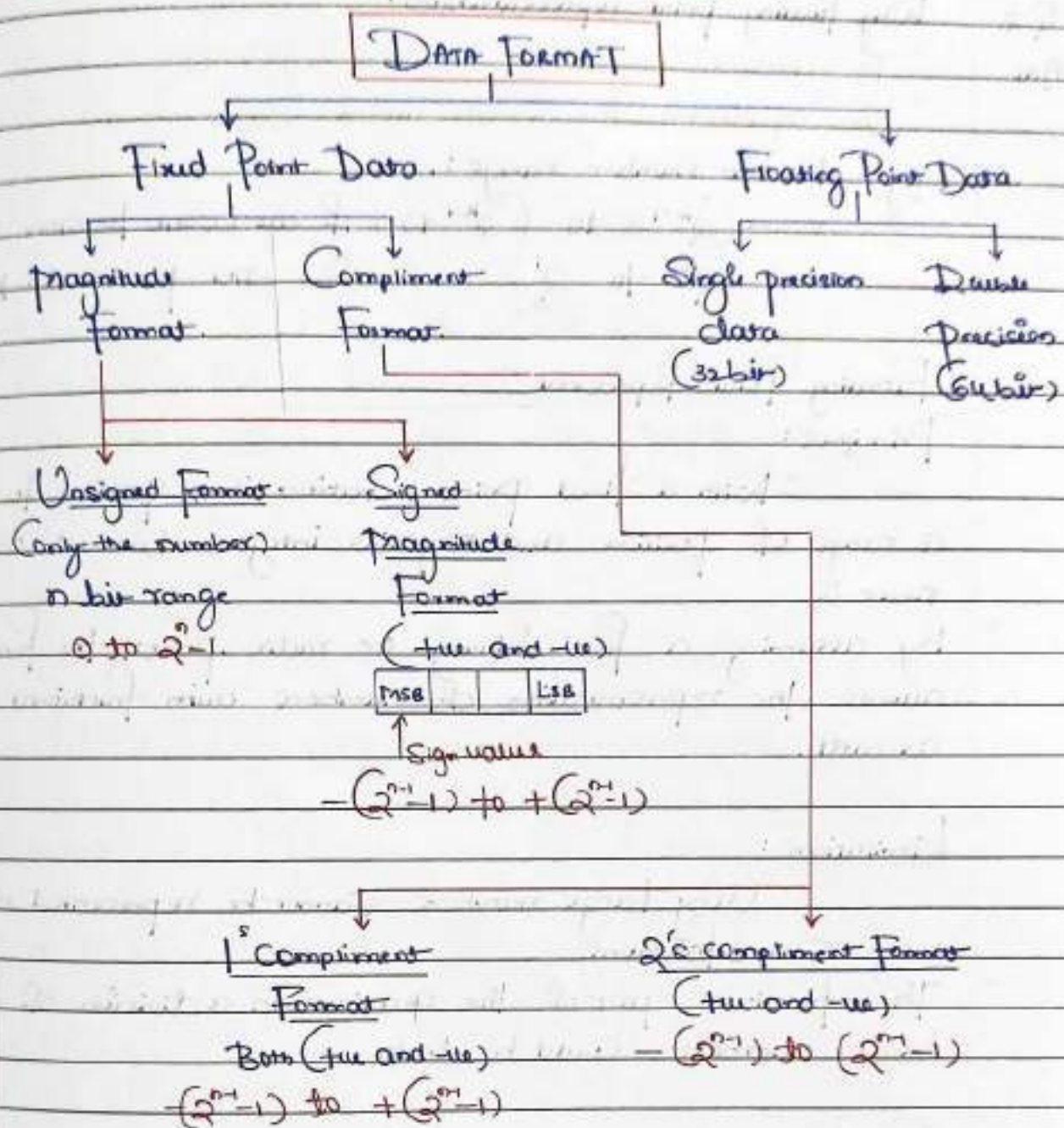
(i) MOV	Destination	Source
	Mem $\leftarrow$	Reg
	Reg $\leftarrow$	Mem
	Reg $\leftarrow$	Reg
	Mem $\leftarrow$	Immediate
	Reg $\leftarrow$	Immediate

Memory Specific { LOAD $\longrightarrow$ Memory Read
STORE $\longrightarrow$ Memory Write

Stack Specific { PUSH $\longrightarrow$ Insert
POP $\longrightarrow$ Delete

IO Specific { IN $\longrightarrow$ IO Read
OUT $\longrightarrow$ IO Write

FLOATING POINT REPRESENTATION



Note: If MSB bit (Sign bit) is '0': the number and if negative num.

Q.1. In Computer System why 2's Complement are used to represent negative number, even we have Signed and 1's Complement also?

Ans

Signed magnitude

+0	: 0000
-0	: 1000

1's Complement

+0	: 0000
-0	: 1111

Redundant representations of '0' in Signed & 1's complement so 2's Complement are used in Computer system.

Q2. Why floating point representation?

Ans

- * To represent very large number (926542...)
- * To represent very small numbers (0.000000...)

eg: 16 bit number range:

$$= -(2^{16}-1) \text{ to } (2^{16}-1), \text{ "if we want to store 512 then"} \\ = -2^{15} \text{ to } 2^{15}, \text{ it is not possible in 16 bit.}$$

Floating Point Representation:

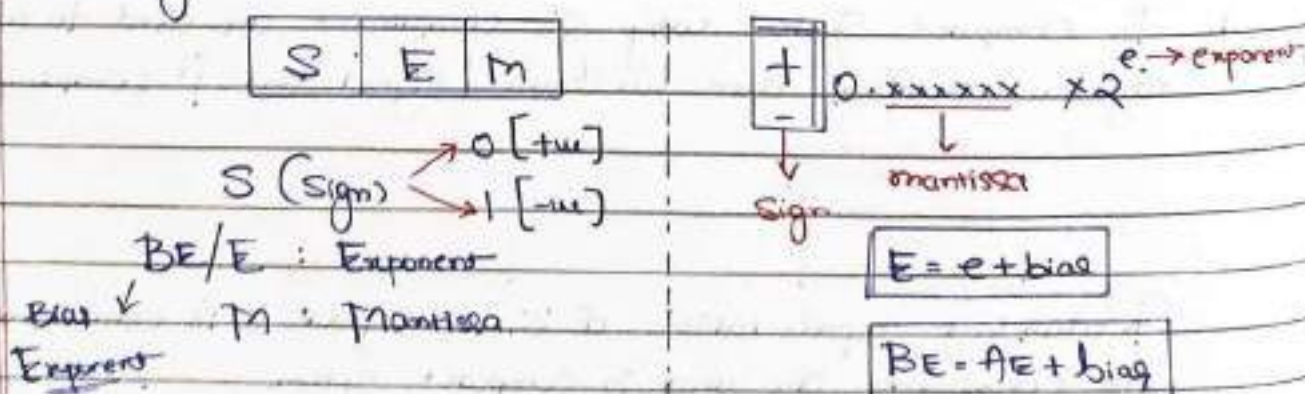
Principle:

- * With a fixed point notation it is possible to represent a range of positive and negative integers centered on or near 0.
- * By assuming a fixed binary or radix point, this format allows the representation of numbers with fractional component as well.

Limitations:

- * Very large numbers cannot be represented nor can very small fractions.
- * The fractional part of the quotient in a division of two large numbers could be lost.

So floating point representation is to represent very large data and very small fraction and consume very less memory.



Number System:

2^4	2^3	2^2	2^1	2^0	.	2^{-1}	2^{-2}	2^{-3}	2^{-4}	2^{-5}	2^{-6}
16	8	4	2	1	0	1	1	1	1	1	1
						2	4	8	16	32	64

V. Imp

① $+6.5$
 $6.5 = (110.1)_2$

$$0.1101 \times \frac{2^3}{2^3}$$

S M 2^e

$$S = 0 (+)$$

$$M = 1101$$

$$e = 3 = (11)_2$$

S	e	M
0	11	1101

$$6.5 = 110.1$$

$$= .1101 \times 2^3$$

$$= [2^1 + 2^2 + 2^4] \times 2^3$$

$$= [2^2 + 2^1 + 2^1]$$

$$= 6.5$$

② $+4.5$

$$100.1$$

$$0.1001 \times 2^3$$

$$S = 0 (+)$$

$$M = 1001$$

$$e = 3 = (11)_2$$

S	e	M
0	11	1101

③ $+4.75$

$$100.11$$

$$.10011 \times 2^3$$

$$S = 0$$

$$M = 10011$$

$$e = 3 = (11)_2$$

S	e	M
0	11	10011

Note: Mantissa alignment process is used to adjust the decimal point, in this process right alignment increments the exponent and left alignment decrements the exponent.

$2^{+ \text{shift}}$ power (+) = Right alignment $\rightarrow$ Increment the exponent
 $2^{- \text{shift}}$ power (-) = Left alignment $\rightarrow$ Decrease the exponent

Right alignment: $6.5 = 110.1$ $= [.2^1 + 2^2 + 2^4] \times 2^3$
 $= .1101 \times 2^3$ $= 2^2 + 2^1 + 2^1$
 $= 6.5$

Left alignment: $0.0000000101 \times 2^{-15}$
 $[1.01 \times 2^{-15-8}]$
 -1.01×2^{-8}

Note: If we take actual exponent $[e]$ then there is no provision to represent the negative exponent. Because sign bit is telling number +ve $[0]$ or -ve $[1]$ if $e=1$

If we take 2's complement of negative exponent then it creates ambiguity like:

$$\begin{array}{l} -2 : 1110 \\ +14 : 1110 \end{array} \left. \vphantom{\begin{array}{l} -2 \\ +14 \end{array}} \right\} \text{Create ambiguity.}$$

So the solution is instead of writing in 2's complement we use "Bias Exponent" $[E/BE]$

$$E = e + \text{bias} \quad \text{or} \quad BE = AE + \text{bias}$$

Biasing: to convert a number into '0' or +ve number.

How bias value is selected?

→ n bit 2's complement range = $-(2^{n-1})$ to $+(2^{n-1}-1)$
 4 bit 2's complement = $-2^{(4-1)}$ to $(2^{4-1}-1)$
 $= -8$ to 7

How bias is selected?

→ if exponent = k bit $\rightarrow$ $\boxed{\text{bias} = 2^{k-1}}$

If exponent k bit then 2's complement = (-2^{k-1}) to $(2^{k-1}-1)$

4 bit 2's complement range:

$$\begin{array}{l} -2^{4-1} \text{ to } +2^{4-1}-1 \\ = \boxed{-8 \text{ to } +7} \end{array}$$

→ In order to convert all number (+ve and -ve) into positive number most highest negative number and add as bias.

4) $+0.00101$
 0.101×2^2
 $m=01$

$E=2, S=0$

$E = (110)_2$ 2 's Complement.

S	E	M
0	1110	10100
	E	M

S	E	M
the	abs	value

Normalised Mantissa

Explicit Normalised

Syntax: $0.\underline{\text{mantissa}} \times 2^E$
 m

Formula to get number
 (value formula)

$(-1)^S \times 0.m \times 2^E$

$(-1)^S \times 0.m \times 2^{E-bias}$

Implicit Normalised

Syntax: $1.\underline{\text{mantissa}} \times 2^E$
 m

Formula to get number
 (value formula)

$(-1)^S \times 1.m \times 2^E$

$(-1)^S \times 1.m \times 2^{E-bias}$

0.1 After the point, immediate
 first bit should be 1.

eg: (101.11)

$= 0.10111 \times 2^3$

$M = 10111, E = 3$

$E = E + bias$

Before the point 1, means 1.....

eg: (101.11)

$= 1.0111 \times 2^2$

$M = 011, E = 2$

$E = E + bias$

eg: 6.75

Explicit

1 bit	4	5
S	E	M
	$bias = 8$	

$= 110.11 \times 2$

$= 0.11011 \times 2^3$

$S=0, M=11011$

Implicit

1 bit	4	5
S	E	M
	$bias = 8$	

$= 110.11$

$= 110.11 \times 2^0$

$= 1.1011 \times 2^2$

$S=0, M=10110$

$$e = +3, \text{ bias} = 8$$

$$E = e + \text{bias} = 3 + 8 = 11$$

$$E = 1011$$

S (1 bit) E (4 bits) M (5 bits)

0	1011	11011
---	------	-------

$$\downarrow \quad \downarrow \quad \downarrow$$

$$1 \quad 7 \quad B = (178)_{16}$$

$$e = +2, \text{ bias} = 8$$

$$E = e + \text{bias} = 2 + 8 = 10$$

$$E = 1010$$

S (1 bit) E (4 bits) M (5 bits)

0	1010	10110
---	------	-------

$$\downarrow \quad \downarrow \quad \downarrow$$

$$1 \quad 5 \quad 6 = (156)_{16}$$

Conversion:

S (1 bit) E (4 bits) M (5 bits)

0	1011	11011
---	------	-------

$$S = 0, E = 1011 \Rightarrow 11$$

$$M = 11011, \text{ bias} = 8$$

$$(-1)^S \times 0.M \times 2^E$$

$$(-1)^0 \times 0.11011 \times 2^{11-8}$$

$$0.11011 \times 2^3$$

$$110.11$$

$$= \underline{\underline{6.75}}$$

S (1 bit) E (4 bits) M (5 bits)

0	1010	10110
---	------	-------

$$S = 0, E = 1010 \Rightarrow 10$$

$$M = 10110, \text{ bias} = 8$$

$$(-1)^S \times 1.M \times 2^E$$

$$(-1)^0 \times 1.10110 \times 2^{10-8}$$

$$+ 1.10110 \times 2^2$$

$$110.110$$

$$= \underline{\underline{6.75}}$$

Q1. $4(5.5)$

S	E	M
---	---	---

1 bit 4 bits 5 bits

Explicit & Implicit represent?

$$\text{bias} = 2^{\text{Exp bit} - 1}$$

$$= 2^{4-1} = \underline{\underline{8}}$$

Explicit

$$(5.5) = 101.1$$

$$= 101.1 \times 2^0$$

$$= 0.1011 \times 2^{+3}$$

$$S = 0, M = 1011, E = 1011$$

$$e = 3, \text{ bias} = 8, E = 3 + 8 = 11$$

S (1 bit) E (4 bits) M (5 bits)

0	1011	10110
---	------	-------

$$\downarrow \quad \downarrow \quad \downarrow$$

$$1 \quad 7 \quad 6 = (176)_{16}$$

Implicit

$$+(5.5) = 101.1$$

$$= 101.1 \times 2^0$$

$$= 1011 \times 2^2$$

$$S = 0, M = 01100, E = 1010$$

$$e = 2, \text{ bias} = 8, E = 2 + 8 = 10$$

S (1 bit) E (4 bits) M (5 bits)

0	1010	01100
---	------	-------

$$\downarrow \quad \downarrow \quad \downarrow$$

$$1 \quad 4 \quad C \Rightarrow (14C)_{16}$$

Getting values:Explicit:

+ (5.5)

0	1011	10110
---	------	-------

$$(-1)^S 0.M \times 2^{E-1000}$$

$$(-1)^0 0.10110 \times 2^{11-8}$$

$$0.10110 \times 2^3$$

$$101.10 = \underline{5.5}$$

Implicit:

S(1 bit) E(4 bits) M(5 bits)

0	1010	01100
---	------	-------

$$E = 1010 = 10$$

$$(-1)^S 1.M \times 2^{E-1000}$$

$$(-1)^0 1.001100 \times 2^{10-8}$$

$$+ 1.01100 \times 2^2$$

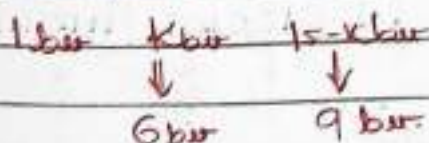
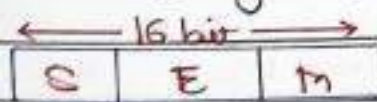
$$+ 101.100 = \underline{+ (5.5)}$$

Note: * Sometimes in explicit we don't get accurate result so either increase the bit in mantissa or use implicit normalisation.

- * Mantissa: giving precision (more and more bit in mantissa giving very accurate value for small fraction also)
- * Exponent: give the range (more bits in exponent means large number)

Q2. Consider a 16 bit register used to store floating point number. Mantissa is normalised signed fraction number. Exponent is in Excess-32 form then what is 16 bit for + (13.5) in the register? (Using explicit & implicit)?

after



$$\text{bias} = 2^{K-1} \quad \text{Excess-32}$$

$$2^{K-1} = 32 \quad \text{bias} = 32$$

$$K-1=5$$

$$K=6 \text{ bit}$$

Explicit: (13.5)

$$= 1101.1$$

$$= 1.1011 \times 2^4$$

$$S=0, M=110110000$$

$$e=4, \text{bias}=32, E=4+32=36$$

Implicit: (13.5)

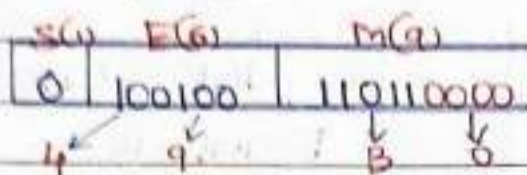
$$= 1101.1$$

$$= 1.1011 \times 2^3$$

$$S=0, M=101100000$$

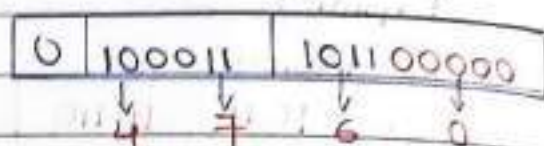
$$e=3, E=3+32=35$$

$$E = 100100$$



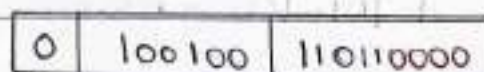
$$(4980)_{16}$$

$$E = 100011$$



$$(4760)_{16}$$

Conversion: Explicit



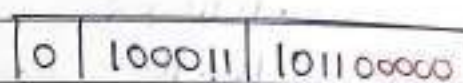
$$(-1)^0 0 \cdot m \times 2^{E-\text{bias}}$$

$$(-1)^0 0 \cdot 110110000 \times 2^{26-32}$$

$$+ 0 \cdot 110110000 \times 2^4$$

$$1101.100000 = (13.5)$$

Implicit:



$$(-1)^0 1 \cdot m \times 2^{E-\text{bias}}$$

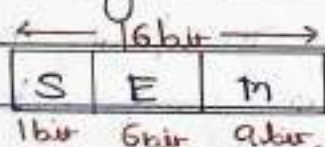
$$(-1)^0 1 \cdot 101100000 \times 2^{35-32}$$

$$+ 1 \cdot 101100000 \times 2^3$$

$$+ 1101.100000 = (13.5)$$

Q3. Consider a 16 bit register used to store floating point numbers. Mantissa is Explicit normalised signed fraction number. Exponent is in Excess-32 form then what is 16-bit for $-(29.75)_{10}$ in the register?

Ans



Excess-32

$$\text{bias} = 32 = 2^{k-1}$$

$$2^{k-1} = 32$$

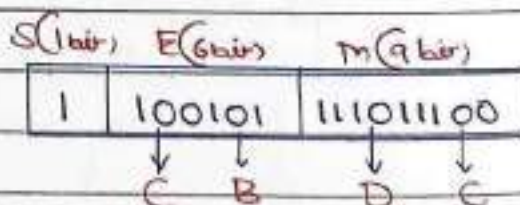
$$\underline{\underline{K=6}}$$

$$= -29.75$$

$$= -11101.1$$

$$= 0.111011 \times 2^5$$

$$S=1, M=111011, e=5$$



$$= \underline{\underline{(CBDC)_{16}}}$$

Q4. Consider a 16 bit register used to store floating point number. Mantissa is implicit normalised signed fraction number. Exponent is in Excess-64 form.

(i) What is the first smallest positive number?

- (i) What is the Second Smallest positive number?
 (ii) What is the difference between first Smallest and Second Smallest positive number?

1 bit 7 bits 8 bits

S	E	M
---	---	---

$$\text{Excess} = 64 \rightarrow \text{bias} = 64$$

$$2^{K-1} = 64 \rightarrow 2^{K-1} = 2^6$$

$$K-1 = 6$$

$$K = 7 \text{ bits}$$

First Smallest positive number:

S(1)	E(7 bits)	M(8 bits)
0	0000000	00000000

$$\text{bias} = 64$$

$$E = 0$$

$$(-1)^S 1.M \times 2^{E-\text{bias}}$$

$$(-1)^0 1.0000000 \times 2^{0-64}$$

$$\text{I. Smallest value} = 1.0000000 \times 2^{-64}$$

Second Smallest positive number:

S(1 bit)	E(7 bits)	M(8 bits)
0	0000000	00000001

$$(-1)^S 1.M \times 2^{E-\text{bias}}$$

$$(-1)^0 1.00000001 \times 2^{0-64}$$

$$\text{II. Smallest} = 1.00000001 \times 2^{-64}$$

(ii) Difference between I & II Smallest:

$$\text{Difference} = 1.00000001 \times 2^{-64} - 1.0000000 \times 2^{-64}$$

$$= [1.00000001 - 1.0000000] \times 2^{-64}$$

$$= [0.00000001] \times 2^{-64}$$

$$2^{-8} \times 2^{-64}$$

$$\Rightarrow 2^{-72}$$

Note:

$$(i) 0.111 \rightarrow \boxed{1 - \frac{1}{2^3}} \quad \text{or} \quad \boxed{1 - 2^{-3}}$$

$$(ii) 0.1111 \rightarrow 1 - \frac{1}{2^4}$$

$$(iii) 0.11111 \rightarrow 1 - \frac{1}{2^5}$$

Q2. [0.1111]

$$\rightarrow 1111.0 \times 2^{-5}$$

$$\rightarrow (2^5 - 1) \times 2^{-5}$$

$$\rightarrow 2^5 \times 2^{-5} - 2^{-5}$$

$$\rightarrow \boxed{1 - 2^{-5}}$$

0.1111

$$\text{Gp. series: } \frac{1}{2} + \frac{1}{2^2} + \frac{1}{2^3} + \frac{1}{2^4} + \frac{1}{2^5}$$

$$a = \frac{1}{2}, r = \frac{1}{2}$$

$$\frac{a(1-r^n)}{1-r} = \frac{\frac{1}{2}(1-\frac{1}{2^5})}{(1-\frac{1}{2})}$$

$$\rightarrow \boxed{1 - \frac{1}{2^5}}$$

Note: We cannot represent '0'.

→ Implicit = 1. ≠ 0 (not zero)

→ Explicit = 0.1 ≠ 0 (not zero)

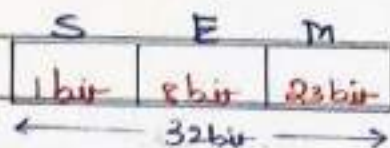
In both implicit and explicit representations we cannot represent '0'. In IEEE 754 floating point we can represent 0.

Disadvantage of Conventional Representation: It cannot store 0. Explicit = 0.1, Implicit = 1.0. It cannot represent '∞' (infinity). It cannot represent number which is not normalised.

IEEE 754 Floating Point Representation

Single Precision
(32 bit)

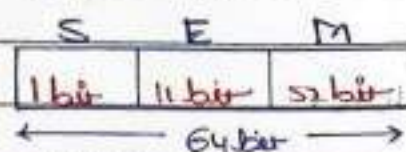
Excess 127



$$\begin{aligned} \text{bias} &= 2^{k-1} - 1 \\ &= 2^8 - 1 \\ &= \underline{127} \end{aligned}$$

Double Precision
(64 bit)

Excess 1023



$$\begin{aligned} \text{bias} &= 2^{k-1} - 1 \\ &= 1023 \end{aligned}$$

Note: In IEEE 754 default is implicit normalisation

Q1. How to represent $+(119)$ into IEEE 754 single precision and double precision floating point representation.

Ans Single Precision

$$= 1.0110111 \times 2^{-16}$$

$$= 1.011011 \times 2^{+6}$$

$S=0$, $M=11011000000000\dots$

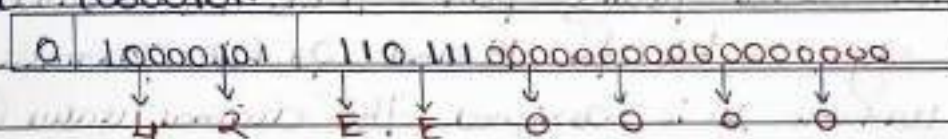
$$e=6, \text{ bias}=127 \quad E=e+\text{bias}$$

$$= 6 + 127 = 133$$

$$\text{bias} = 2^{8-1} - 1$$

bias = 127

E = 10000101



$$+119: (42EE0000)_{16}$$

Double Precision

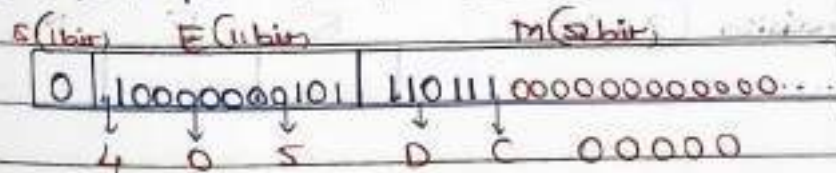
119: 11101111x3°

$$+ 1.10111 \times 2^{+6}$$

$S=0$, $T=110111000000\dots$

$e=6$, $\text{bias}=1023$

$$E = 1022 + 6 = 1029 = \boxed{10000000101}$$



$$+ 119 : (405, DC 000000000000)_{16}$$

Q2. Consider a 32 bit register which stores floating numbers in IEEE single precision format (implicit). The value of numbers, f 32 bit are given below:

Sign (1 bit)	Expo (8 bit)	Mantissa (23 bit)
0	10000100	11000000000000000000000...

Ans. $E = 132$, Bias = 127

$$(-1)^S \cdot 1.m \times 2^{E - \text{bias}}$$

$$(-1)^0 \cdot 1.11000000 \dots \times 2^{132 - 127}$$

$$+ 1.11000000 \dots \times 2^{+5}$$

$$+ 111000.000000 \dots \times 2^0$$

$\Rightarrow 56$

Q3. The value of float type variable is represented using the single precision 32 bit floating point IEEE 754 standard use 1 bit sign, 8 bit for E and 23 bit mantissa. A float type variable x is assigned the decimal value (-14.25). Then representation of x in hexa decimal notation is?

Ans. -14.25

$$1110.01 = 1.1101 \times 2^3$$

$$S=1, M=11001, e=3$$

1 bit sign, 8 bit Expo, 23 bit Mantissa

S	E	M
---	---	---

$$\text{Bias} = 2^{8-1} - 1 = 127$$

$$E = e + \text{bias} = 3 + 127$$

$$= 130$$

S	E	M
1	100000010	11001000...

$$\text{Hexadecimal} = \underline{(C1640000)}_{16}$$

Single Precision (32 bit)

S	E	M
1 bit	8 bit	23 bit

Sign (1 bit)	E (8 bit)	M (23 bit)	Value
0 or 1	00000000 $E=0$	0000000... $M=0$	± 0
0 or 1	11111111 $E=255$	0000... $M=0$	$\pm \infty$
0 or 1	$1 \leq E \leq 254$	$M = \dots$	Implicit Normalized form $(-1)^S \times 1.m \times 2^{E - 127 \text{ bias}}$

0 or 1	$E=0$	$M \neq 0$	Denormalised numbers $(-1)^S \times 0.M \times 2^{E-127}$
0 or 1	$E=255$	$M \neq 0$	Not a Number (NaN)

Q. Why bias = 2^{k-1} ?

1 bit 8 bit 23 bit

S E M

Ans. Single Precision:

If in single precision, if bias = $2^{8-1} = 2^7$ (bias = 128) then there is a possibility of getting $E=255$.

$$E = e + \text{bias} \quad e = +127 \quad \text{then } E = 127 + 128$$

$$= 255$$

8 bit 2's complement range

$$= -2^{8-1} \text{ to } 2^{8-1}-1$$

$$= -128 \text{ to } 127$$

$$\text{So bias} = \underline{2^{k-1}}$$

if $E=255$? $\begin{cases} +\infty \\ \text{NaN} \end{cases}$

Double Precision:

Excess = 1023.

1 bit 11 bit 52 bit

S E M

Sign (1 bit)	E (11 bit)	M (52 bit)	Value
0 or 1	0000000000 $E=0$	0000000000... $M=0$	± 0
0 or 1	1111111111 $E=2047$	0000000000... $M=0$	$\pm \infty$
0 or 1	$1 \leq E \leq 2046$	$M \neq 0$	Implicit Normalisation $(-1)^S \times 1.M \times 2^{E-1023}$
0 or 1	$E=0$	$M \neq 0$	Denormalised Number $(-1)^S \times 0.M \times 2^{E-1023}$
0 or 1	$E=2047$	$M \neq 0$	Not a Number

Note: When $E=0$ then value = 0 when $M=0$ and when $M \neq 0$ then value is in fractional form.

When $E=254$ then value = ∞ when $M=0$ and when $M \neq 0$ it is not a number (NaN).

Features IEEE 754 are Special Symbols to represent Unusual events...

- ① For example instead of interrupting on a divide by 0, software can set the result to a bit pattern representing $+\infty$, $-\infty$. The largest exponent is reserved for these special symbols.
- ② IEEE 754 has a symbol for the result of invalid operations such as $0/0$, or subtract infinity from infinity. This symbol is NaN for not a number.

The purpose of NaN is to allow programmer to postpone some test and decisions to a later time in this program (when it is convenient).

Denormalised Number:

Minimum possible value of $E = 00000001$, $E=1$

$$E = e + \text{bias}$$

$$e = E - \text{bias}, 1 - 127 = -126, \text{ so } e = -126$$

That means in worst case $e = -126$, if value of e is smaller than -126 , then number is not able to normalise and we store as denormalised number.

eg: 1.1001×2^{-127} , $e = -127$, $E = -127 + 127 = 0$ so $E=0$
not able to normalise.

eg: 1.1001×2^{-128} , $e = -128$ so $E = -1$, not able to normalise
or 1.1001×2^{-129} Here $M=1001$

$$e = -129, E = e + \text{bias}, -129 + 127 = -2$$

So $E = -2$ not able to normalise because E must be 1 after biasing.

For eg. The Smallest Positive (true) Single Precision normalised number is: $1.0000000 \times 2^{-126}$

But the Smallest Single Precision denormalised number is: $0.00000001 \times 2^{-126}$ or $|1.0 \times 2^{-149}|$

and double Precision range is: $|1.0 \times 2^{-1022}|$ to $|1.0 \times 2^{1024}|$

Q1. How to represent $\pm(1.0)$ into IEEE-754 Single Precision floating point representation?

Ans.

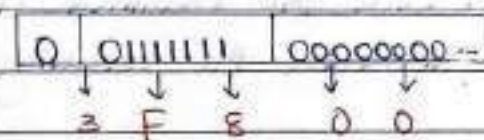
1 bit S	8 bit E	23 bit M
------------	------------	-------------

 bias = 2^{E-1}

$$1.0 \times 2^0$$

$$M=0, e=0, E=e+bias$$

$$= 0 + 127 = 127$$



$$(3FF00000)_{16}$$

Q2. Consider a 64 bit register which store floating point number in IEEE-754 Double Precision format. What is the value of the number if 64 bits are given as

0	11111111	00000000...
---	----------	-------------

S E (11 bit) M (52 bit)

Here all bits in E are 1

$$E = 2047, M = 0 \therefore \underline{\underline{\pm \infty}}$$

Add/Subtract Rule:

1. Choose the number with the smallest exponent and shift its mantissa right a number of steps equal to the difference in exponents.
2. Set the exponent of the result equal to the larger exponent.
3. Perform addition/subtraction on the mantissa and determine the sign of the result.
4. Normalise the resulting value if necessary.

Note: Multiplication and division are somewhat easier than addition and subtraction, in that, no alignment of exponents is required.

Multiply Rule

1. Add the exponents and subtract 127.
2. Multiply the mantissas and determine the sign of the result.
3. Normalise the resulting value, if necessary.

Divide Rule

1. Subtract the exponents and add 127.
2. Divide the mantissas and determine the sign of the result.
3. Normalise the resulting value if necessary.

The addition or subtraction of 127 in the multiply and divide rule results from using the excess -127 notation for exponents.

eg: -0.75 in Single Precision format.

$$\rightarrow -0.75 \times 2^0$$

$$\rightarrow -1.5 \times 2^{-1}$$

1 bit	8 bit	23 bit
S	E	M

$$E = -1 \quad \text{bias} = 127$$

$$E = -1 + 127 = 126 \quad E = 0111110$$

Sign	E	M
1	01111110	10000....

$$\Rightarrow (BF40000)_{16}$$

eg. Add $9.999 \times 10^{+1}$ and 1.610×10^{-1}

$$\text{now, } 9.999$$

$$+ 0.016$$

$$10.015 \times 10^{+1}$$

$$\text{Normalise} \rightarrow 1.0015 \times 10^{+2}$$

$$1.610 \times 10^{-1}$$

$$0.1610 \times 10^0$$

$$0.01610 \times 10^{+1}$$

$$1.002 \times 10^{+2}$$

$$\text{Roundoff}$$

Now equivalent exponents
so perform addition

Try adding the numbers 0.5 and -0.4375 in binary then using algorithm

Ans: $0.5 = \frac{1}{2} = 0.1 \times 2^0$
 $= (1.000 \times 2^{-1})_2$

$-0.4375 = -\frac{7}{16} = -\frac{7}{2^4} = -0.0111$
 $= -0.0111 \times 2^0$
 $= (-1.110 \times 2^{-2})_2$

Step 1: The significand of the number with the lesser exponent $(-1.11_2 \times 2^{-2})$ is shifted right until its exponent matches the larger number: $-1.11_2 \times 2^{-2} = -0.111 \times 2^{-1}$.

Step 2: Add the significands:
 $(1.000)_2 \times 2^{-1} + (-0.111_2 \times 2^{-1}) = 0.001_2 \times 2^{-1}$

Step 3: Normalizing the Sum, checking for overflow or underflow
 $(0.001)_2 \times 2^{-1} = 0.010_2 \times 2^{-2}$
 $= 0.100 \times 2^{-3}$
 $= 1.000_2 \times 2^{-4}$

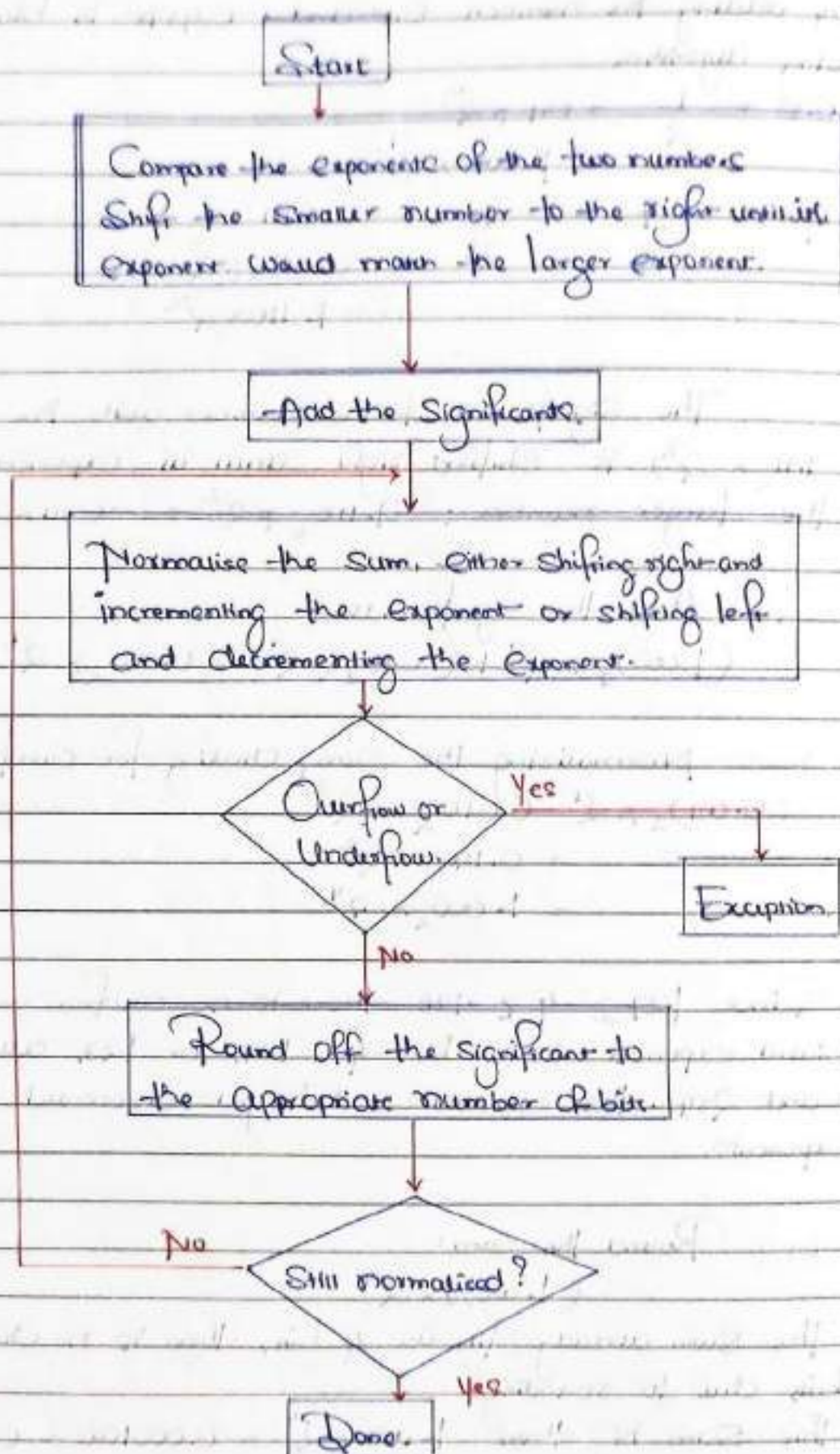
Since $127 \geq -4 \geq -126$ there is no overflow or underflow. (The biased exponent would be $-4 + 127$, or 123, which is between 1 and 254, the smallest and largest unreserved biased exponents.)

Step 4: Round the Sum:

$$(1.000)_2 \times 2^{-4}$$

The Sum already fits in 4 bits, there is no change to the bits due to rounding.

This Sum is then $1.000 \times 2^{-4} = 0.0001000 = 0.0001$
 $= \frac{1}{2^4} = \frac{1}{16} = 0.0625$



Note: The normal path is to execute Steps 3 and 4 once, if rounding causes the sum to be abnormalised, one must repeat Step 3.

→ Multiply 1.110×10^{10} and 9.200×10^5 .

Step 1: Unlike addition, one calculates the exponent of the product by simply adding the exponents of both operands.

$$\text{New exponent} = 10 + (-5) = 5$$

Let's do the same with biased exponent to make sure we obtain the same result:

$$10 + 27 = 37, \text{ and } -5 + 127 = 122, \text{ so}$$

$$\text{new exponent} = 37 + 122 = 259$$

This is too large for 8 bit exponent field, so something is amiss. The problem is with the bias because one is adding the biased as well as new exponents:

$$\text{New exponent: } (10 + 127) + (-5 + 127) = (8 \times 25 + 2 \times 127) = 259$$

Accordingly, to get the correct biased sum when one adds biased numbers, one must subtract bias from the sum.

$$\text{New exponent} = 37 + 122 - 127$$

$$= 259 - 127 = 132 \quad (5 + 127) \text{ and } 5 \text{ is indeed the}$$

exponent one calculated initially.

Step 2: Multiply the significands:

$$\begin{array}{r} 1.110 \\ \times 9.200 \\ \hline 0000 \\ 0000 + \\ 2220 + + \\ 0990 + + + \\ \hline 1021200 \end{array} = 10.212 \times 10^5$$

Step 3: The product is unnormalised, so we need to normalise it:

$$10.212 \times 10^5 = 1.0212 \times 10^6$$

Step 4: We assumed that the significand is only four digits long (excluding the sign), so we must round the number.

The number 1.0212×10^6 is rounded to four digits in the significand to 1.021×10^6 .

Step 5: The sign of the product depends on the signs of the original operands. Here they have both same sign hence the product is: $+1.021 \times 10^6$.

Binary floating point multiplication

Multiply $(0.5)_{10}$ and $(-0.4375)_{10}$

Step 1: In binary 1.000×2^{-1} by -1.110×2^{-2}

Adding the exponents (without bias): $-1 + (-2) = -3$

Or, using the biased representation:

$$\Rightarrow (-1 + 127) + (-2 + 127) - 127$$

$$\Rightarrow (-1 - 2) + (127 + 127 - 127)$$

$$\Rightarrow -3 + 127 = 124$$

Step 2: Multiply the significands

$$\begin{array}{r} 1.000 \\ \times 1.110 \\ \hline 0000 \\ 1000+ \\ 1000++ \\ 1000+++ \\ \hline 1110000 \end{array}$$

The product is 1.110000×2^{-3} but we need to keep it to 4 bits so it is 1.110×2^{-3} .

Step 3: Now we check the product to make sure it is normalised, and then check the exponent for overflow or underflow. The product is already normalised and, since $127 + (-3) = 126$, there is no overflow or underflow.

Step 4: Rounding the product makes no change:
 -1.110×2^6

Step 5: Since the signs of the original operands differ, make the sign of the product negative, hence the product is
 -1.110×2^6

Converting to decimal to check our results:

$$-1.110 \times 2^6 = -0.001110 = -0.00111$$

$$= -\frac{7}{2^5} = -\frac{7}{32} = -0.21875$$

The product of $(0.5)_{10}$ and $(-0.4375)_{10}$ is indeed $(-0.21875)_{10}$.

Note: The normal path is to execute Step 3 and 4 once, but if rounding caused the sum to be unnormalised, we must repeat Step 3.

Little Endian and Big Endian

- * The exact number of bytes that constitute a word depends on the system, for eg: in system Pentium, a word refers to four bytes or 32 bits. On the other hand, eight bytes are grouped into a word in the Itanium Processor.
- * Bits in a word are usually ordered from right to left.
 $1 \text{ Byte} = 8 \text{ bits}$
 $\text{Word Size} = \text{Word length of the processor.}$
- * Default configuration in the memory chip is Byte addressable.

So in the memory data is stored in byte wise.

- * In the processor operations are performed on a word format, so when the word length of the processor is greater than 8 bit then multiple cells accessing is required to access the data from memory simultaneously.

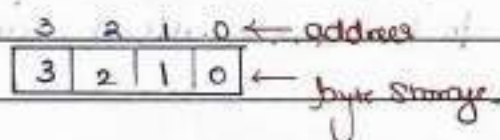
→ To solve the problem of ambiguity there is need of memory address interpretation mechanism (m/c instructions) called "Endian m/c instructions".

① Little Endian

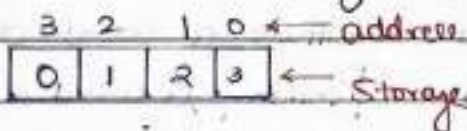
② Big Endian

Byte ordering or Endian m/c instructions: Show the order of data storage in memory.

- ① Little Endian: Lower address contains lower byte and higher address contains higher byte.



- ② Big Endian: Lower address contains higher byte and higher address contains lower byte.



* Default m/c instructions i.e. Little Endian used in processor design. But some processor use Big Endian also.

* Little Endian: Right to left
Big Endian: Left to right.

Byte Ordering:

- Storing data often requires more than a byte.
- There are two byte ordering schemes referred to as little endian and big endian.

eg: $(11, 12, 13, 14)_{16}$ Stored at starting address 100

32 bit processor
Little Endian
(Right to left)

$x0 =$

103	102	101	100
11	12	13	14

$x0 = 11121314$

Big Endian
(Left to right)

$x0 =$

103	102	101	100
14	13	12	11

$x0 = 14131211$

eg: $(51, 21, 15, 25)_{16}$
"32 bit processor"

Little Endian
(Right to left)

$x2 =$

403	402	401	400
51	21	15	25

$x2 = 51211525$

Big Endian
(Left to right)

$x2 =$

403	402	401	400
25	15	21	51

$x2 = 25152151$

ALU & CONTROL UNIT

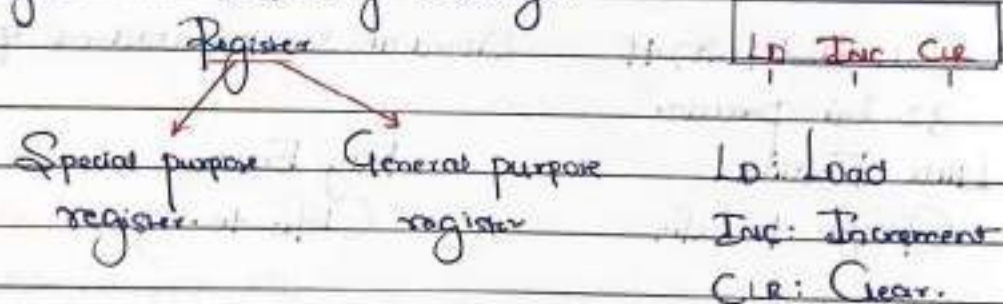
Components of a Computer

① CPU, ② Memory and ③ IO.

- Memory
- Register
- ALU
- Timing signals, Control signals
- Flag (Psw)

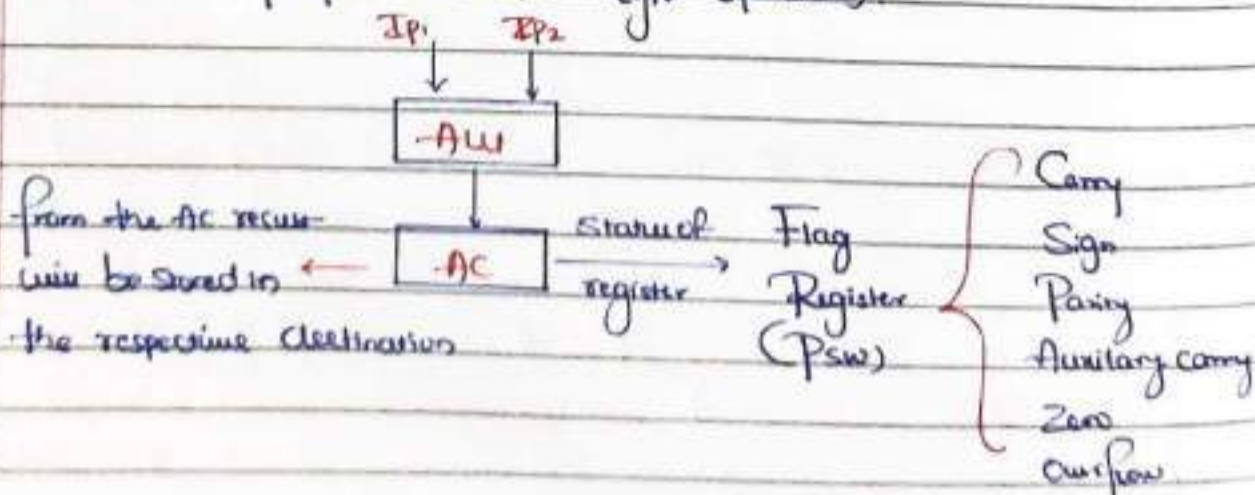
Register [Flip-flop]: Collection of bits/sequence of bits stored in flip-flop.

* Register is temporary storage.



ALU: Arithmetic and Logic Unit.

- Hardware
- Perform arithmetic and logical operation, Comparison and Condition Checking.
- Perform multiple type operations.



Timing Signal: To execute the Operation in Proper Sequence.

eg: 1. Fetch Fetch
 2. Decode T₁: PC $\rightarrow$ MAR
 3. Execute T₂: M[MAR] $\rightarrow$ Memory
 Memory $\rightarrow$ MBR
 MBR $\rightarrow$ IR

eg IF we have 16 Registers, 1 ALU, 1 Memory, 1 PSW and 1 Component.
 i.e. total 20 Component

$$20C_2 = 20 + \text{Components/connections}$$



So the solution instead of using 20+ connections is to connect all component to a common bus
 (Internal Bus)

But only two parts (2 component) can communicate at a time and which component will communicate for that control signal is required.

The number of multiplexer = No. of bit in the register (size of the register)
 Size of the multiplexer = No. of registers.

Q. IF we have 32 registers and each register size is 8 bit then
 Ans $\rightarrow$ Number of multiplexer (mux) = 8 bit
 $\rightarrow$ Size of mux = 32 (32x8)

Note: IF we have m registers and each register size is n bit then

$\rightarrow$ Number of mux = n
 $\rightarrow$ Size of mux = $m(m \times n)$

Q1. How data is transferred?

Register A to Register B.

$R_A \rightarrow R_B$

Process: Register 'A' content given to Mux (multiplexer),
Mux to Common Bus then,
Common bus to load into Register B (R_B).

Q2. Why R_{out} and R_{in} is needed?

Ans. Register output is connected to the Mux then mux to
Common bus then Common bus to respective register.

Process: When R_{out} is set to '1' then that respective
register content (data) load into the Mux. Then Mux to
Common bus and here Common bus is connected to all
registers, then the register having R_{in} is set to 1 in
that respective register bus content is loaded in that
register.

$R_A \rightarrow R_B$

R_A to R_B ; R_{out} , R_{in}

$R_{out} = 1$

Content of register A loaded into Mux then
Mux to Common Bus

Now $R_{in} = 1$ then content of Bus load into register R_B

MAE/AR [Memory Address Register]: It contains address of
memory location. It is connected to the Address line (A) of
the system bus.

MDE/MOR/DR [Memory Data Register]: It contains memory
content (Instruction and data). It is connected to the data line
of the bus.

Instruction Register (IR): Containing the instructions currently being executed by the CPU.

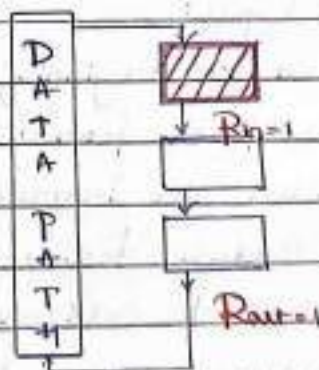
In the Processor some general purpose and some special purpose register are available. All the registers are connected to a common path called Data Path (Bus). Data Path is a collection of functional units (ALU and MUX).

Every register has two switches R_{in} and R_{out} :

R_{in} : R_{in} is set to 1, if the content of the bus loaded into register R_i .

R_{out} : R_{out} is set to 1, the content from the register R_i will be placed on bus.

Data Path: ALU, MUX, Register and other components, how they process, & pass for processing.



Micro Operations:

Micro Operations is a elementary operation in the hardware.

eg: register to register transfer is a one kind of micro operation.

Micro Operations consume 1 cycle to complete the execution.

Control signal are required to execute the micro operation.

eg: Machine Instructions: $mov\ r0, r1$

RTL (Register Transfer Language): $r0 \leftarrow r1$

Micro operation: $T_1: r1 \leftarrow r0$

Micro Operations:

The functional, or atomic, operations of a processor.

- * Series of steps, each of which involves the processor registers.
- * Micro refers to the fact that each step is very simple and accomplished very little.
- * The execution of a program consists of the sequential execution of instructions.
- * Each instruction is executed during an instruction cycle made up of shorter sub cycles (Fetch, Instruction, execute, interrupt).
- * The execution of each sub cycle involves one or more shorter operations (micro-operations).

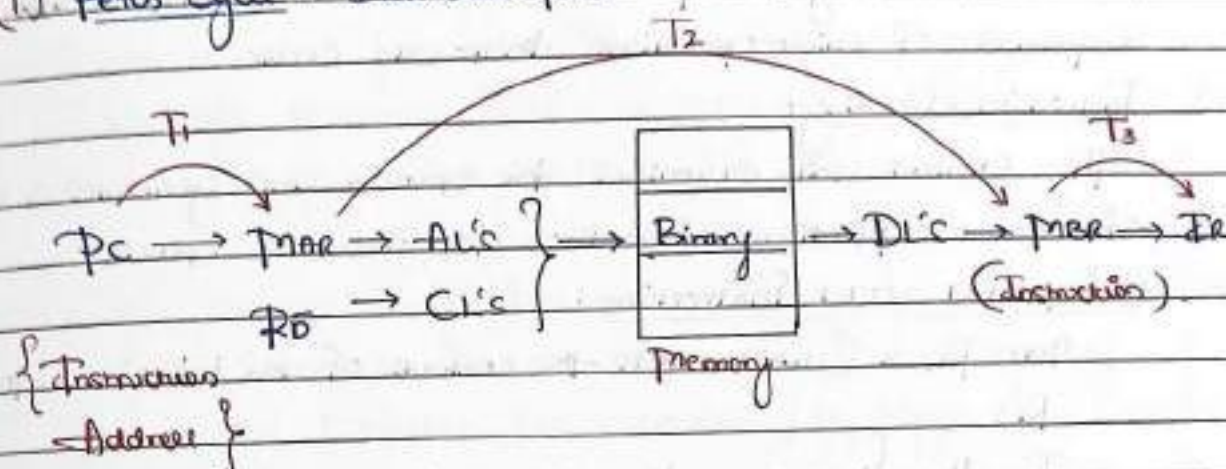
Micro Program (μ-program): Sequence of micro operations (μ-operations) to perform some operation in hardware level is called as micro program.

The Fetch Cycle:

Occurs at the beginning of each instruction cycle and causes an instruction to be fetched from memory.

Four registers are involved:

- ① Memory Address Register (MAR):
 - * Connected to address bus
 - * Specifies address for read or write operations.
- ② Memory Buffer Register (MBR):
 - * Connected to data bus
 - * Holds data to write or last data read.
- ③ Program Counter (PC):
 - * Holds ^{address} instructions of next instruction to be fetched.
- ④ Instruction Register (IR):
 - * Holds the last instruction fetched.

Instruction Cycle:(1.) Fetch Cycle: Instruction Fetch

Micro Program	$T_1: PC \rightarrow MAR$	$PC \text{ out } MAR \text{ in}$
	$T_2: M[MAR] \rightarrow MBR$	$MAR \text{ out } MBR \text{ in}$
	$PC + 1 \rightarrow PC$	$PC \text{ out } PC \text{ in}$
	Increment	
	$T_3: MBR \rightarrow IR$	$MBR \text{ out } IR \text{ in}$

* At the end of fetch cycle instruction is loaded into IR register

(2.) Execute Cycle:

1. Decode the instruction
2. Operand address calculation
3. Operand fetch using the help of addressing mode
4. Data processing
5. Result storage

Rules for Micro Operations - Grouping:

- ① Proper sequence must be followed.

$MAR \leftarrow (PC)$ must precede $MBR \leftarrow (Memory)$

- ② Conflicts must be avoided

* Must not read and write the same register at same time

* $MBR \leftarrow (Memory)$ and $IR \leftarrow (MBR)$ must not be in same cycle

* If different components then perform in same cycle

Execute Cycle:

Because of variety of Opcode, there are a number of different sequences of micro operations that can occur.

(1) Instruction Decoding:

The control unit examines the opcode and generates a sequence of micro-operations based on the value of opcode.

(2) A simplified add instruction:

(i) $\text{Add } R_1, x$ (which adds the contents of the location x to register R_1)

(ii) In the first step the address portion of the IR is loaded into the MAR.

(iii) Then the referenced memory location is read.

(iv) Finally the contents of R_1 and MAR are added by the ALU.

(v) Additional micro operations may be required to extract the register reference from the IR and perhaps to store the ALU inputs or outputs in some immediate registers.

(a) IR Stage: Enable the hardware to perform the operation (Instruction Decode).

(b) OF Stage: ARs (addressing mode) are required to access (Operand Fetch).

Interrupt Cycle:

* At the completion of the execute cycle, a test is made to determine whether any enabled interrupts have occurred and if so, the interrupt cycle occurs.

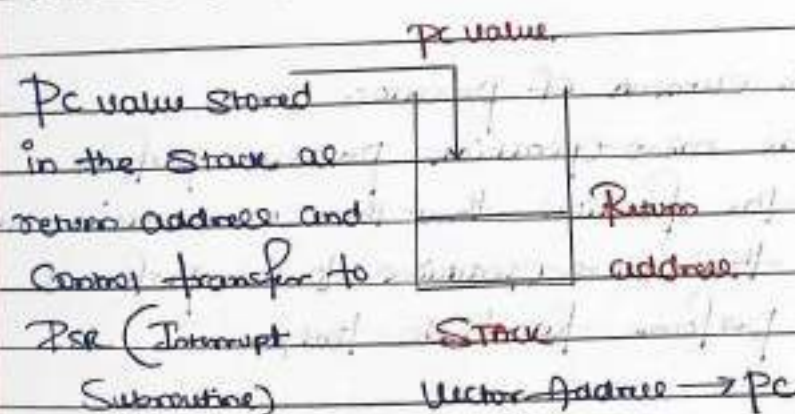
* The nature of this cycle varies greatly from one machine to another in a simple sequence of events:

(1) In the first step contents are transferred to the MBR so that they can be saved from return from the interrupt.

(2) Then MAR is loaded with the address at which the contents of the PC are to be saved, and the PC is loaded with

- the address of the start of the interrupt processing routine.
- (3) These two actions may be a single micro operation.
 - (4) Because most processors provide multiple types and/or levels of interrupt, it may take one or more additional micro-operations to obtain the same address and the routine address before they can be transferred to the MAR and PC respectively.
 - (5) Once this is done, the final step is to store the PNR, which contains the old value of PC into memory.
 - (6) Now the processor is now ready to begin the next instruction cycle.

Whenever interrupt occurs after the completion of current instruction execution, interrupt will be serviced.



Q1. The following are some events that occur after a device controller issues an interrupt while processor is under execution.

- (P) The processor pushes the processor status of L onto the Control Stack.
- (Q) The processor finishes the execution of the current instruction.
- (R) The processor executes the interrupt service routine.
- (S) The processor pops the processor status of L from the control stack.
- (T) The processor loads the new PC value based on the interrupt.

Which is the correct order: QPTRS

Q2. Consider the following sequence of micro-operations:

MBR $\leftarrow$ R

MBR $\leftarrow$ X

PC $\leftarrow$ Y

Memory $\leftarrow$ MBR

Which one of the following is a possible operation performed by this sequence?

Ans. Initiation of interrupt service

Control Unit Functional Requirements

* By reducing the operations of the processor to its most fundamental level we are able to define exactly what is that the control unit must cause to happen.

* Three step process to lead to a characterization of the control unit:

1. Define basic elements of processor.
2. Describe the micro-operations processor performs.
3. Determine the functions that the control unit must perform to cause the micro-operations to be performed.

* The control unit performs two basic tasks:

- * Sequencing
- * Execution

Control Unit:

* Control unit is the supervisor in the system that controls each and every activity.

* Control unit takes various inputs but produces only one output (control signals).

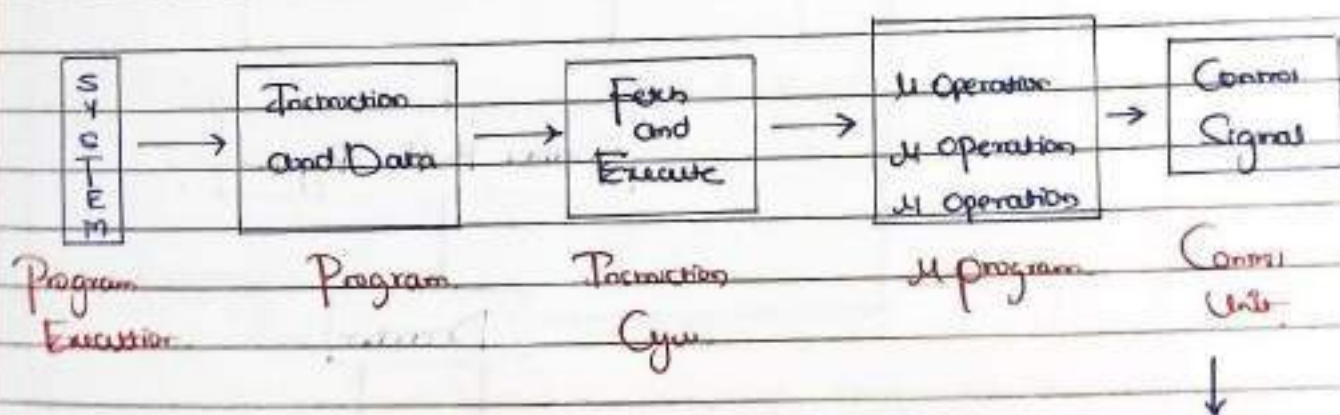
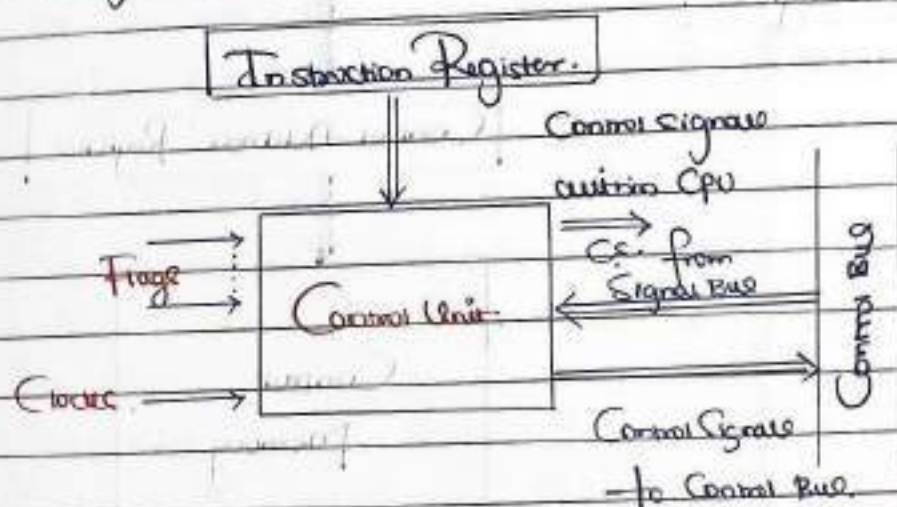
* These control signals are required to execute the micro-operations.

* Control signals are directly executed on a base hardware so hardware generates the desired output.

* Computer system functionality is program execution.

- Q. Why Control Unit is set after decoder?
 Ans. In fetch and decode same type of operation is performed. After the decode, input given to the Control Unit, so according to the operations Control signal is generated to perform micro operations.

Block Diagram of the Control Unit



Control Unit: Control word

It is PC $\rightarrow$ MAR; PC $\rightarrow$ MAR

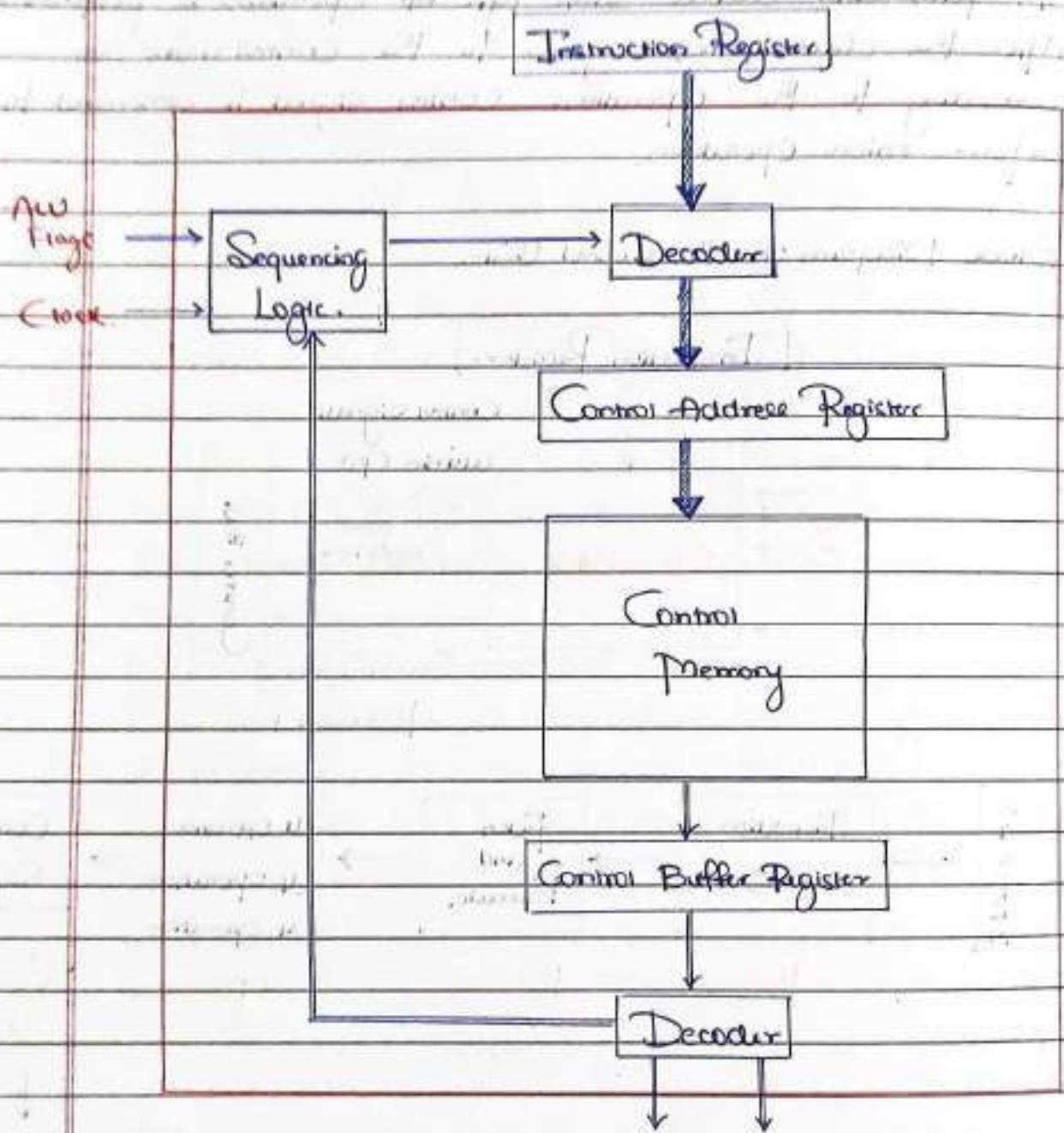
Program Output

Hardware
are (H/W)

PC, PC, MAR, MAR, PC, PC, MAR, MAR, PC, PC

1 control word for 1 microoperation

Functioning of ^{micro}programmed Control Unit



Control Signals within CPU Control Signals to System Bus

→ Control Signals will be implemented into the Control Unit by using the following approach:

1. Hardwired CU design
2. Microprogrammed CU design.

Working of Control Unit

- Control unit generates the Control Signals, these Control Signals are same for fetch and decode.
- After the decode, different type Control Signal (Control word) generated according to different type of operations.
- At the Control unit design time, designer decide which Control are generated in which cycle ($T_1, T_2, \dots, T_n$) for different type of instruction and that is stored in the table.
- Later we make a boolean function for implementation.

① Hardwired CU Design:

- In the Hardwired Control Unit design Control Signals are expressed in S.o.p. Expressions (Sum of products).
- They are directly realised on hardware.
- In Hardwired Control unit, they used fixed logic circuit to interpret the instructions and generate the Control Signal.

Note: * Hardwired CU is fastest CU design.
* RISC is a Hardwired CU.

Disadvantages:

- It is not flexible.
- Even a minor modification require rendering redesigning and reselling.
- It does not support new operations (Core designed).

Process:

Step 1: Searches where the Control Signals A_n and B_n are present.

Step 2: Options are in I.I format or T.I format.

Step 3: For any particular time interval. Is the Control Signal present for all the instructions?

Q1. Consider the following hypothetical CPU which uses 3 data register A, B and C and suppose 3 instructions I_1, I_2 and I_3 . Obtain the logic functions that will generate the hardwired control for the signal A_{out} and B_{out} using the following data.

	I_1	I_2	I_3
T_1	A_{in}, B_{out}	A_{in}, C_{in}, B_{out}	B_{in}, B_{out}
T_2	B_{in}, C_{in}, A_{out}	A_{in}, A_{out}	A_{in}, B_{out}, C_{out}
T_3	B_{in}, B_{out}	B_{in}, B_{out}	B_{in}, B_{out}
T_4	C_{in}, A_{out}	B_{in}, A_{out}	A_{in}, A_{out}
T_5	End	End	End

$$B_{out} = T_1 I_1 + T_1 I_2 + T_1 I_3 + T_2 I_1 + T_3 I_2 + T_3 I_3$$

$$= T_1 (I_1 + I_2 + I_3) + T_3 (I_2 + I_3)$$

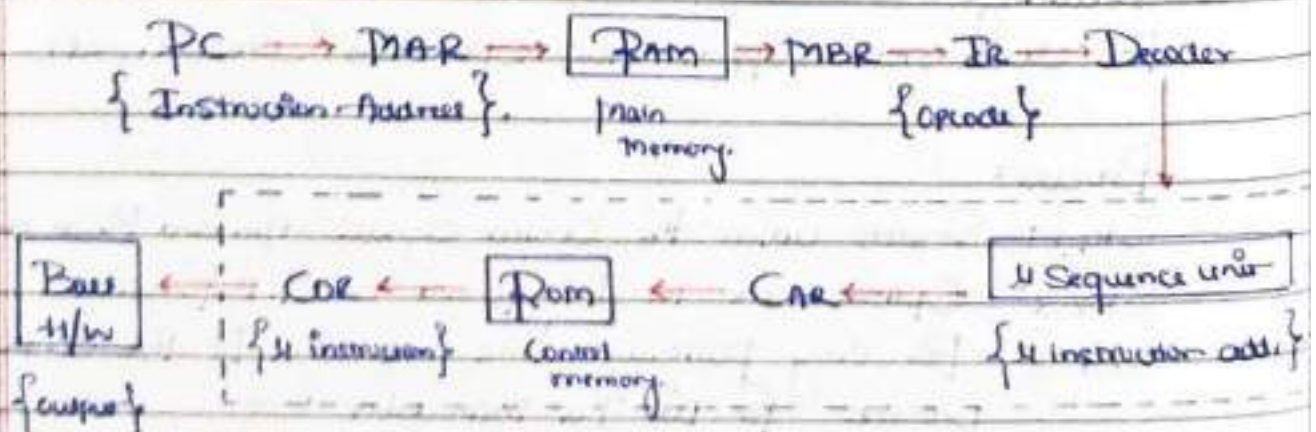
$$B_{out} = T_1 + T_3$$

$$A_{in} = T_1 I_1 + T_1 I_2 + T_2 I_2 + T_3 I_3 + T_4 I_3$$

$$A_{in} = T_1 I_1 + (T_1 + T_2) I_2 + (T_3 + T_4) I_3$$

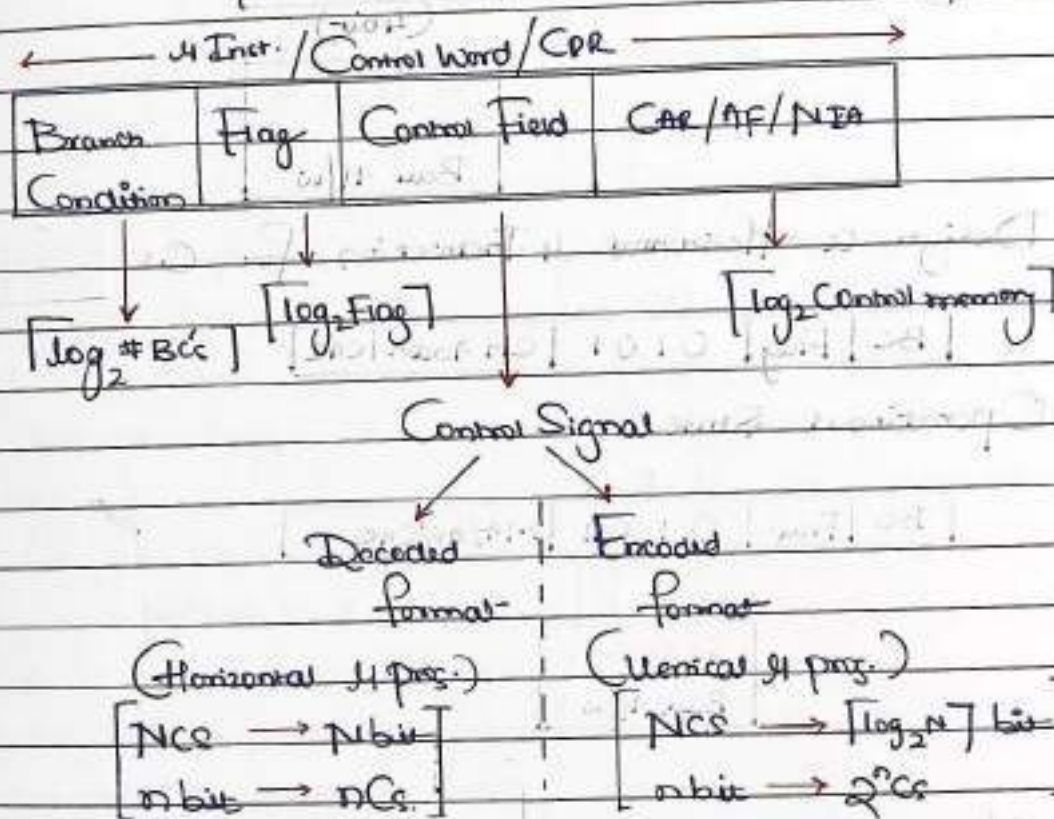
② Micro Programmed Control Unit:

- In micro-programmed CU design control words are stored in the control memory.
- According to the type of operations control signals are generated.
- Control memory is associated with CAR and CDR Register to contain control memory address and data respectively.
CAR: Control Address Register
CDR: Control Data Register.



Process:

- * The Set of micro instructions stored in the Control memory.
- * The [CAR] Control-Address contains the address of next instruction or micro instruction to be read.
- * Then a micro instruction is read from the Control memory and it's transferred to control buffer register.
- * Control buffer register is connected to the control line so in this micro instruction execution occurs.



Control Signal

① Hardwired CU Design

- S.O.P expression
- Logic function
- Fastest (Risc)
- Not flexible

② Microprogrammed CU

- Micro program stored in Control memory

→ Micro Instructions are implemented by 2 Approaches:

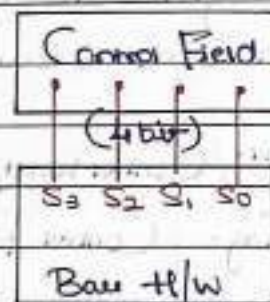
- ① Horizontal μ programming
- ② Vertical μ programming

Horizontal Programming:

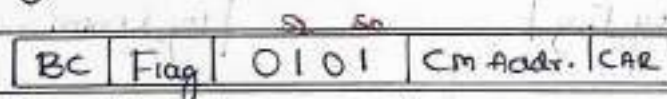
1. Number of control in the hardware = $[S_3 S_2 S_1 S_0]$
2. Decoded for CS $\Rightarrow$

Enable = 1

Disable = 0



3. Design a Horizontal μ Instruction for CS = $(S_3 S_2)$



4. Operational State.



Vertical Microprogrammed μ :

1. Number of control signal in H/W = $(S_3 S_2 S_1 S_0)$
2. Encoded form of CS $\Rightarrow$ eg: $4CS = \lceil \log_2 4 \rceil = 2 \text{ bit}$



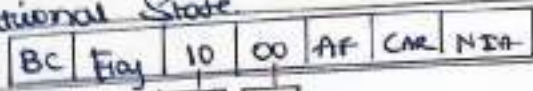
Note: External decoder is required

3. Design a vertical microinstruction for CS = $[S_3 S_2]$



Function Code [FC]
generated by the Control Unit to give the signal to CPU to perform operation.

4. Operational State



Horizontal Programming

1. In this control signal are expressed in a decoded format.
2. 1 bit $\rightarrow$ 1 CS
NCS $\rightarrow$ Nbit
3. It supports longer control words.
eg: for 200 control signal we require 200 bit in control field.
4. No external decoder is required to generate the control signal.
5. It is flexible compared to hardwired CU.
6. It support high degree of parallelism (none/more-than-1)

Vertical Programming

- In this control signal are expressed in encoded format.
n bit $\rightarrow 2^n$ CS
NCS $\rightarrow \lceil \log_2 n \rceil$ bits
- It supports shorter control word.
eg: for 200 control signal we require 8 bit $\lceil \log_2 200 \rceil$ or 2^8 in control field.
- External decoder is required to generate the control signal.
It is more flexible.
- It support low degree of parallelism (one/one)

Speed: Hardwired > Horizontal > Vertical

Time Consumption: Vertical > Horizontal > Hardwired.

Q2. Arrange the following configuration for CPU in decreasing order of operating speeds: - Hardwired, Vertical micro programming, Horizontal micro programming.

Ans. Hardwired control, Horizontal micro programming, Vertical micro programming.

Q3. Horizontal micro programming:

a. does not require use of signal decoders

b. results in larger sized microinstructions than vertical micro programming

c. Use one bit each control signal

d. all of the above

Q4. An instruction set of a processor has 125 signals which can be divided into 5 groups of mutually exclusive signals as follows:

Group 1: 20 signals, Group 2: 70 signals, Group 3: 2 signals

Group 4: 10 signals, Group 5: 23 signals

How many bits of the control words can be saved by using vertical micro programming over horizontal?

Ans.

	<u>Horizontal</u>	<u>Vertical</u>	
G ₁ : 20cc	20	5 bit	# bit = 125-122
G ₂ : 70cc	70	7 bit	
G ₃ : 2cc	2	1 bit	Saved = <u>103</u>
G ₄ : 10cc	10	4 bit	
G ₅ : 23cc	23	5 bit	
	125 bit	20 bit	

Q5. Consider a hypothetical CU it has 1024 control words memory. To support 48 control signals and 16 flags, what is the size of the control words in bits and control memory in byte using:

(i) Horizontal Programming

(ii) Vertical Programming

Control memory = 1024 control word = 2^{10} CW $\rightarrow$ Cn. Address / AF = 10 bitsFlags = 16 Flag = 2^4 flag
= 4 bits

(i) Horizontal Programming

48 control signal = CF = 48 bits

Flag	CF	AF
4 bit	48 bit	10 bit

1 CW = 4 + 48 + 10 = 62 bits

Control memory = 1024 CW
= 1024×62 bits
 $\approx$ 8k Byte

(ii) Vertical Programming

48 Control Signal = CF = 48 bits

Flag	CF	AF/NIA
4 bit	6 bit	10 bit

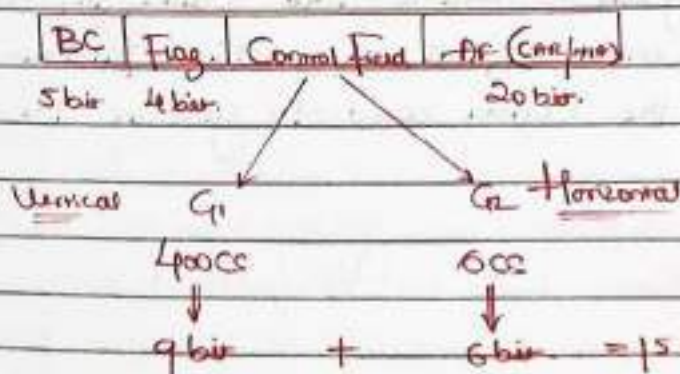
1 Control Word = 4 + 6 + 10 = 20 bits

Control memory = 1024 CW
= 1024×20 bits
 $\approx$ 3k Byte

Qc. Control field of a micro instruction support 2 groups of control signal in which Group 1 indicate None/one of 48 control signal and Group 2 (Horizontal) indicate 6 signals. Hardware contains 16 Flag and 32 branch condition.

If Cn register size is 20 bits then what is CnR in bits and Control memory in bits?

Ans. 16 Flag $\Rightarrow$ Flag = 4 bits. 32 Branch = BC = 5 bits. CnR/AF = 20 bits



1 CW Size = 5 + 4 + 15 + 20 = 44 bits

Control memory = 2^{20} CW = $2^{20} \times 44$ bits
= 44 to bits

<u>RISC</u>	<u>CISC</u>
Reduced Instr. Set Computer.	Complex Instr. Set Computer.
1. It supports less number of addressing modes.	It supports more number of Am.
2. It supports smaller instruction set.	It supports large instruction set.
3. It supports more number of registers.	It supports less number of register.
4. It supports fixed length instr.	It supports variable length instr.
5. It supports 1 instruction per cycle (CPI=1)	It does not support 1 instruction per cycle (CPI $\neq$ 1)
6. It supports pipeline successfully.	It supports unsuccessful pipeline.
7. It is the expensive processor used in real-time applications.	It is less expensive processor.
8. It is a Super Computer.	General purpose Computer.
9. It uses hardwired control unit (Motorola processor, Power processor, ARM processor)	It uses microprogrammed (vertical) control unit (Pentium processor)

Q7. A micro program control unit is required to generate a total of 25 control signals. Assume that during any micro instruction at most two control signals are active. Minimum number of bits required in the control word to generate the required control signal.

Ans

$$25 \times 1 \Rightarrow 25 \text{ bits} \\ \Rightarrow 10 \text{ bits}$$

$$\frac{25 \text{ bits}}{2} = 12.5 \text{ bits} \\ \Rightarrow 13 \text{ bits}$$

Q8. A hypothetical processor contains word length of 12 bits. It supports 1 word opcode. Each instruction takes 8 cycles to complete the execution. Processor contains 25 control signals & 16

Flag Processor used horizontal Control unit. 64 Branch is used then what is the size of Cw, Cx in bin of Control memory?

Word length = 12 bit

Op code = 1 word = 12 bit

Total # of Instructions = 2^2

cycle/instruction = 8 cycle/instruction

Total no. of Instructions = $2^2 \times 8$

= 2^{15} Instructions/Cw.

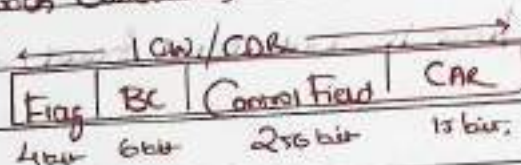
Control memory = 2^{15} Cw

AE/Cw Add = 15 bit

16 Flag $\Rightarrow$ Flag = 4 bit

64 Branch Conditions = BC = 6 bit

Horizontal: 256 Cx = 28 bit



1 Cw = 28 bit

Control memory = 2^{15} Cw

= $2^{15} \times 28$ bit

Q9. Consider the following processor design characteristics:

I. Register-to-register arithmetic operations only

II. Fixed length instruction format

III. Hardwired control unit

Which of the characteristics above are used in the design of a RISC processor?

Ans: I, II and III

CPU Time Calculations

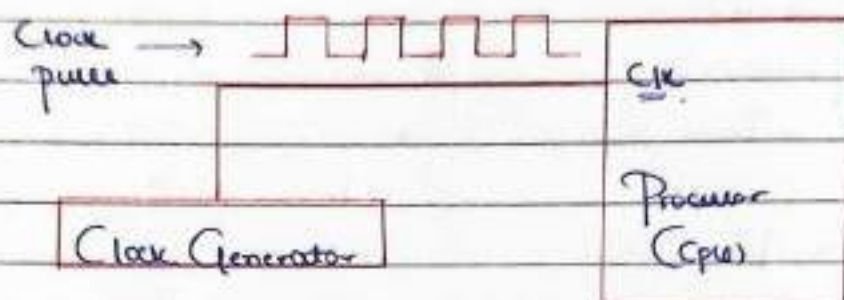
* CPU-time Calculations Program Execution time.

* Program execution time is based on the clock.

* Processor contains clock pins and these clock pins are externally connected with the clock generator.

So in the computer system all the operations are controlled by the clock, so CPU contains clock pins which are externally connected with clock generator.

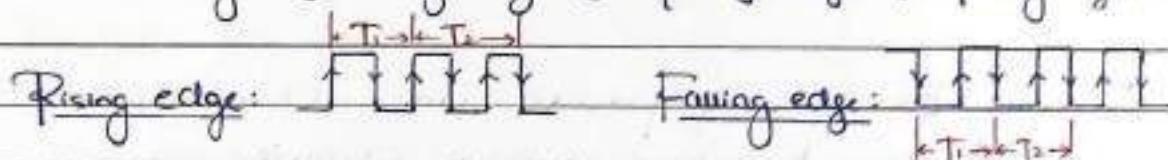
- Clock generator is operating with a constant frequency to generate the clock pulse (clock signal).
- These clock signals are carried into the CPU through (with the help of) Clock pins. So CPU operations are controlled by the clock signal.



- Program Execution time is calculated based on 2 factors:

1. Cycle
2. Cycle time

- ① Cycle: Cycle is defined as clock pulse transition either from rising edge to rising edge (or) falling edge to falling edge.



- ② Cycle Time: The time required to transfer the pulse either from rising edge to rising edge or falling edge to falling edge is called as cycle time.

Cycle time depends on the clock frequency

$$\text{Cycle time} \propto \frac{1}{\text{Clock frequency}}$$

Q: In program we have 100 Instructions, and each instruction takes 6 clock cycles operating at 2GHz processor. Program E.T. = ?

Ans: 100 Instructions, each takes 6 cycles

Total = $100 \times 6 = 600$ cycles

Program E.T. = $600 \times 0.5 \text{ nsec} = 300 \text{ nsec}$

$$\text{Cycle time} = \frac{1}{2 \text{ GHz}} = 0.5 \text{ nsec}$$

CPU Time Calculation / Program ET

Program ET = $\frac{\# \text{seconds}}{\text{Program}}$

$$= \frac{\# \text{Instruction}}{\text{Program}} \times \frac{\# \text{cycle}}{\text{Instruction}} \times \frac{\# \text{Second}}{\text{cycle}}$$

↓
Instruction Count

↓
Cycle per Instruction

↓
Cycle time

$$\text{Program ET} = IC \times CPI \times \text{Cycle time}$$

- * Program is a combination of data-transfer, data manipulation, & transfer of control (Jcc) instructions. Different instructions takes (consume) different cycle to complete the execution so,

$$\text{Program ET} = \left[\sum (IC_i \times CPI_i) \right] \text{ Cycle time} \quad i = \text{Instruction}$$

Let CPI_i be the number of cycles required for the instruction type i and I_i be the number of executed instructions of type i for a given program. Then we can calculate an overall (average) CPI as follows:

$$CPI = \frac{\sum_{i=1}^n (CPI_i \times I_i)}{I_c}$$

The processor time T needed to execute a given program can be expressed as $T = IC \times CPI \times \tau$ (where $\tau = 1/f$, and $I_c = \text{instr. Count}$).

A common measure of performance for a processor is the rate at which instructions are executed, expressed as millions of instructions per second (MIPS), referred to as the MIPS rate. We can express the MIPS rate in terms of the clock rate and CPI as follows:

$$\text{MIPS rate} = \frac{I_c}{T \times 10^6} = \frac{f}{CPI \times 10^6} \text{ MIPS}$$

Q2. Common data question. 1, 2 and 3. Consider a 1.5 GHz clock frequency processor used to execute the following program segments.

Instruction type	Instruction count	CPI
Load	300	11
Store	200	9
Arithmetic	250	7
Shift	150	6
Branch	50	4

(i) What is the average instruction execution?

Ans
$$\text{Avg CPI} = \frac{300 \times 11 + 200 \times 9 + 250 \times 7 + 150 \times 6 + 50 \times 4}{950}$$

$$= 8.36 \text{ cycle}$$

$$\text{Cycle time} = \frac{1}{1.5 \text{ GHz}} \text{ sec}$$

$$\text{Avg Instruction cycle} = 8.36 \text{ cycle}$$

$$= 8.36 \times 0.66 \text{ nsec}$$

$$= 0.66 \text{ nsec}$$

$$\boxed{\text{Avg Inst. ET} = 5.51 \text{ nsec}}$$

(ii) What is the MIPS rate of a program?

Ans
$$1 \text{ Instruction Avg. ET} = 5.51 \times 10^{-9} \text{ sec}$$

$$1 \text{ Instr.} = 5.51 \times 10^{-9} \text{ sec}$$

$$\text{In 1 sec} = \# \text{ Instr.} = \frac{1}{5.51 \times 10^{-9}} \text{ Instr/sec}$$

$$= \frac{1}{5.51} \times 10^9 \text{ Instr/sec} = 0.1814 \times 10^9 \text{ Instr/sec}$$

$$= \underline{\underline{181.4 \text{ MIPS}}}$$

(iii) What is the total program ET?

Ans
$$\text{Avg Inst. ET} = 5.51 \text{ nsec}, \text{ Program contains 950 instructions}$$

$$\text{Program ET} = 950 \times 5.51 \text{ nsec}$$

$$= 5234.5 \times 10^{-9}$$

$$= 5.2345 \times 10^{-6}$$

$$\boxed{\text{Program ET} = 5.2345 \mu\text{sec}}$$

Q2. 1 nsec clock cycle processor consumes 4 cycles for load and store operations and 6 cycle for ALU operations and 2 cycle for branch operation. The relative frequency of these operations are 40%, 40% and 20% respectively.

(i) What is the average instruction ET?

$$\text{Ans. Avg. CPE} = 40 \times 4 + 40 \times 6 + 20 \times 2$$

$$= 1.6 + 2.4 + 0.4$$

$$\text{Avg. CPE} = 4.4 \text{ cycles}$$

$$\text{Avg. Instruction ET} = 4.4 \times 1 \text{ nsec}$$

$$= 4.4 \text{ nsec}$$

(ii) What is the performance in terms of MIPS?

$$1 \text{ Instruction} = 4.4 \times 10^{-9} \text{ sec}$$

$$\text{In 1 sec} = \frac{1}{4.4} \times 10^9 \text{ Inst/sec}$$

$$= \frac{1000}{4.4} \times 10^6 \text{ Inst/sec}$$

$$= 227.2 \text{ MIPS}$$

(iii) If program contains 10^6 instructions then what is total program ET?

$$1 \text{ Instruction} = 4.4 \times 10^{-9} \text{ sec}$$

$$\text{Program contains} = 10^6 \text{ Instructions}$$

$$\text{Program ET} = 4.4 \times 10^{-9} \times 10^6$$

$$= 4.4 \times 10^{-3}$$

$$= 4.4 \text{ msec}$$

Q4. Consider a 2 nsec clock cycle processor which consumes 7 cycles for load and store instructions and 7 cycles for ALU instructions and 3 cycle for branch instructions. Relative frequency of these instructions are 40%, 40% and 20% respectively. Processor is sustained with an average CPE of 1. During the execution

INSTRUCTION PIPELINING

Pipelining

Pipeline is a mechanism which are used to improve the performance of the system, in which task are executed in an overlapping manner.

Pipelining is a decomposition technique that means the problem is divided into sub problem and assign the sub problem to the pipes then operate the pipe under the same clock.

Accepting new input before the previously accepted input appear as a output at other end. i.e it means new input are executed along with old input in a overlapping manner.

Non Pipeline: Accepting new input after the previously accepted input appear as a output at other end. In which non overlapping execution occur.

o/p end

Non Pipeline

S ₄				I ₁				I ₂				I ₃				I ₄	
S ₃			I ₁				I ₃				I ₃					I ₄	
S ₂		I ₁				I ₂				I ₃					I ₄		
S ₁	I ₁				I ₂			I ₃				I ₄					
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	

I/f end

Pipelining

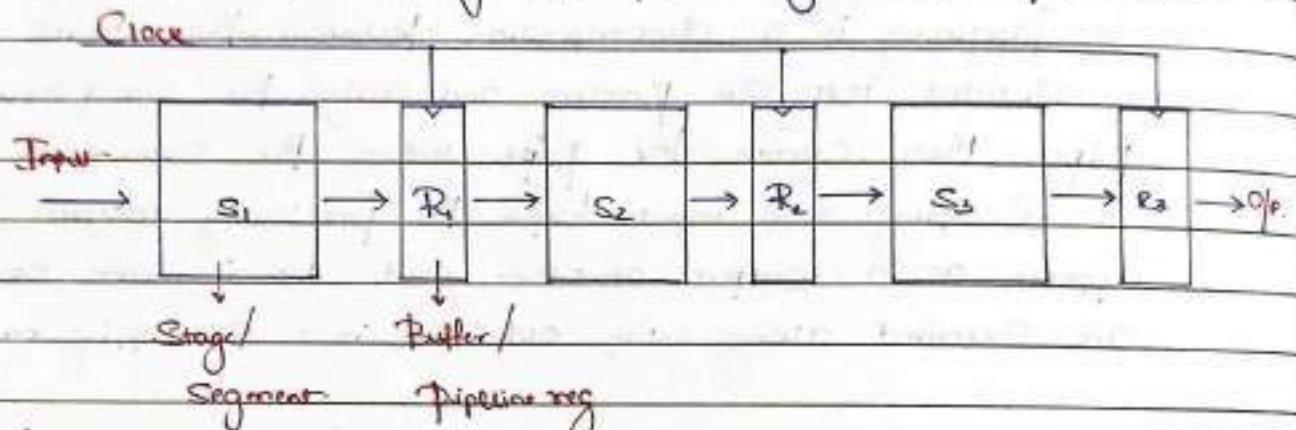
o/p end

S ₄				I ₁	I ₂	I ₃	I ₄										
S ₃			I ₁	I ₂	I ₃	I ₄											
S ₂		I ₁	I ₂	I ₃	I ₄												
S ₁	I ₁	I ₂	I ₃	I ₄													
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	

I/f end

Pipeline Design:

- * Pipeline has 2 ends $\swarrow$ Input-end
 $\searrow$ Output-end
- * Between the input and output end multiple pipes are interconnected to satisfy the functionality of the pipeline.



- * These pipes are called stage or segment.
- * Between the stages buffer are used to store the intermediate result.
- * These buffers are also called as latch, pipeline registers, interface registers.
- * All the stages along with the buffer are controlled by common clock.

Pipeline Performance Evaluation:

Now consider the case where a K -segment pipeline with a clock cycle time t_p , is used to execute n tasks. The first task T_1 requires a time equal to Kt_p to complete its operations since there are K -segments in the pipe. The remaining $(n-1)$ tasks emerge from the pipe at the rate of one task per clock cycle and they will be completed after a time equal to $(n-1)t_p$.

Therefore, to complete n tasks using a K -segment pipeline requires $K + (n-1)$ clock cycles.

$$ET_{pipe} = K * tp + (n-1) tp$$

$$ET_{pipe} = [K + (n-1)] tp$$

K : # Stages (segments)

n : # instructions

tp : Each stage delay in pipeline.

Execution Time in Non-Pipeline:

$$ET_{Non\ pipeline} = n * t_n$$

t_n : Each instruction execution time in non pipeline

n : # instructions (tasks)

Performance Gain of Pipeline Processor

Performance Gain

or
Speed up factor $[S]$

$$= \frac{\text{Performance of pipeline}}{\text{Performance of non pipeline}}$$

$$= \frac{1/ET_{pipe}}{1/ET_{non\ pipe}}$$

$$\rightarrow \text{Speed up factor } [S] = \frac{ET_{non\ pipe}}{ET_{pipe}} \Rightarrow S = \frac{n t_n}{[K + (n-1)] tp}$$

$$S = \frac{t_n}{tp}$$

As the number of tasks increases, n becomes much larger than $K-1$, and $K t_n$ approaches the value of n .



When very large number of task/instructions executed

on
When no. of task is not given.

eg: $K=4, n=1$ $t_p = (\text{Stage delay}) = 2 \text{ nsec}$

$$ET_{\text{pipe}} = [4 + (1-1)] \times 2 \text{ nsec}$$

$$= 8 \text{ nsec}$$

Pipeline

$$t_n = 2+2+2+2 = 8 \text{ nsec}$$

$$ET_{\text{non pipe}} = n t_n$$

$$= 1 \times 8 \text{ nsec}$$

$$= 8 \text{ nsec}$$

Non-pipeline

Note: In ideal case when pipeline stage are perfectly balanced then one task ET in non pipeline

$$ET_{\text{NP}} = K \times t_p$$

eg: $K=4$

1 Task ET in non pipeline

 $t_p = 2 \text{ cycle}$ (2 cycle in each stage)

$$ET_{\text{NP}} = 2+2+2+2$$

$$ET_{\text{NP}} = 8 \text{ cycle}$$

$$ET_{\text{NP}} = K \times t_p$$

$$= 4 \times 2$$

$$ET_{\text{NP}} = 8 \text{ cycle}$$

eg: 1 Task ET in non pipeline

(1 cycle in each stage)

$$ET_{\text{NP}} = 1+1+1+1 = 4 \text{ cycle}$$

$$ET_{\text{NP}} = K \times t_p$$

$$= 4 \times 1 \Rightarrow$$

$$ET_{\text{NP}} = 4 \text{ cycle}$$

Note: This only happens in the condition when pipeline are perfectly balanced

Each stage takes same amount of time/cycle

$$S = \frac{K \times t_p}{t_p}$$

$$S = K$$

Throughput

$[k + (n-1)] tp$ time unit n task completed

In 1 time unit = n

$$(\text{Throughput}) \quad [k + (n-1)] tp$$

Note: In pipeline when number of tasks are not given
Then number of tasks completed per unit time

$$\text{Throughput} = \frac{1}{tp}$$

Efficiency:

$$\eta = \frac{S}{k}$$

S : Speed up factor

k : number of stages (segment)

Types of Pipeline:

1. Uniform delay Pipeline

2. Non uniform delay Pipeline

- ① Uniform Delay Pipeline: In uniform pipeline each stage taking same amount of time/delay to execute the assigned operation/task.

Pipeline		
1 Task ET in <u>pipe</u>	1 Task ET in <u>non pipe</u>	If <u>buffer delay</u> is included:
$k = 4 \text{ (stages)}$		
$tp = (\text{stage delay}) \times 2$	$ET_{np} = n \times t_n$	$tp = (\text{stage delay} + \text{buffer delay})$
$n = 1$	$t_n = 2 + 2 + 2 + 2$	
	$= 8 \text{ nsec}$	$tp = 2 + 1 = 3 \text{ nsec}$
$ET_{pipe} = [k + (n-1)] tp$	$ET_{np} = 1 \times 8 \text{ nsec}$	$ET_{pipe} = [4 + (1-1)] \times 3$
$= [4 + (1-1)] \times 2 \text{ nsec}$	$ET_{np} = 8 \text{ nsec}$	$ET_{pipe} = 12 \text{ nsec}$
$= 8 \text{ nsec}$		$ET_{np} = 8 \text{ nsec}$

1 Task ET in Pipeline = 1 Task ET in non pipeline

- (2) Non uniform delay pipeline: In non uniform delay pipeline each stage take different amount of time (delay) to complete the assigned task/operation.

1 Task ET in pipeline

$$k=4, n=1$$

$$t_p = \max(\text{stage delay})$$

$$t_p = \max(2, 8, 4, 2)$$

$$t_p = 8 \text{ nsec}$$

$$ET_{\text{pipe}} = [k + (n-1)] t_p$$

$$ET_{\text{pipe}} = 32 \text{ nsec}$$

1 Task ET in non pipeline

$$ET_{\text{non pipe}} = n t_n$$

$$t_n = 2 + 8 + 4 + 2$$

$$= 16 \text{ nsec}$$

$$ET_{\text{non pipe}} = 1 \times 16 \text{ nsec}$$

$$ET_{\text{non pipe}} = 16 \text{ nsec}$$

If buffer delay is

included:

$$t_p = \max(\text{storage delay}, \text{buffer delay})$$

$$= \max((2+1), (8+1), (4+1), (2+1))$$

$$= 9 \text{ nsec}$$

$$ET_{\text{pipe}} = [k + (n-1)] \times 9$$

$$ET_{\text{pipe}} = 36 \text{ nsec}$$

When number of task/instruction increase:

if $n=1000$ in previous question. (above one)

$$ET_{\text{pipe}} = [k + (n-1)] t_p$$

$$= [4 + (1000-1)] \times 8$$

$$ET_{\text{pipe}} = 8024 \text{ nsec}$$

$$ET_{\text{non pipe}} = n t_n$$

$$t_n = 2 + 8 + 4 + 2 = 16 \text{ nsec}$$

$$ET_{\text{non pipe}} = 1000 \times 16$$

$$ET_{\text{non pipe}} = 16000 \text{ nsec}$$

Important Points:

- (1) When pipelines are perfectly balanced (uniform delay) then 1 task (instruction) execution time in pipelined is same as 1 task ET in non pipelined.

- (2) When pipeline are not perfectly balanced (non uniform delay) then 1 task execution in pipeline is generated/equal to 1 Task ET in non pipeline.

T_1 : one task ET in pipeline

$$T_1 \geq T_2$$

T_2 : one task ET in non pipeline

- (3) When the number of task are increased then pipeline perform - once is best compared to non pipeline.

- Q1. Buffer delay is not included in non pipeline, because in non pipeline we are not storing the intermediate result. So buffer delay is included only in pipeline when it is given in question.

Uniform delay.

$$t_p = \text{Storage delay} + \text{buffer delay.}$$

Non uniform delay

$$t_p = \max(\text{Storage delay}, \text{buffer delay})$$

- Q1. Consider an instruction Pipeline which is speed up factor 20 while operating with 80% efficiency. What could be the number of stages in the pipeline?

Ans $\eta = \frac{S}{K} \Rightarrow 80\% = \frac{20}{K}$

$$K = \frac{20 \times 100}{80} = 25$$

- Q2. A pipeline is having speed factor as 10 and operating with efficiency of 80%. What will be the number of stages in the pipeline?

Ans $\eta = \frac{S}{K} \Rightarrow 80\% = \frac{10}{K}$

$$K = \frac{10 \times 100}{80} = 12.5 = 13$$

- Q3. 4 segment pipeline have the respective stage delay of 10ns, 15ns, 20ns, and 30ns. What is the efficiency of the pipeline when very large number of tasks are executed?

Ans $K = 4$, $t_p = 10 + 15 + 20 + 30 = 75 \text{ nsec}$

$$S = \frac{t_0}{t_p} = \frac{75}{30} = 2.5$$

$$t_p = \max(\text{stage delay}) = \max(10, 15, 20, 30)$$

$$t_p = 30 \text{ nsec.}$$

$$\eta = \frac{S}{K} = \frac{2.5}{4} = 0.625 = 62.5\%$$

Q4. A 4 segment instruction pipeline has the respective stage delay of 8ns, 11ns, 20ns, 2ns respectively. The interface register are used between the stages have a delay of 2ns. What is the approx speed up value when very large number of tasks are pipelined?

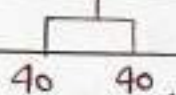
Ans. $K=4$. Stage delays = (8, 11, 20, 2) Buffer delay = 2nsec
 $t_0 = 8 + 11 + 20 + 2 = 41 \text{ nsec}$
 $t_p = \max(\text{stage delay} + \text{buffer delay})$
 $= 20 + 2 = 22$
 $S = \frac{t_0}{t_p} = \frac{41}{22} = 1.86$

Q5. Consider 4 stage Pipeline with the respective stage delay of 20ns, 80ns, 50ns, and 10ns. In the enhancement process of the pipeline largest stage is split into 2 equal stage delay:
 (i) What is the speedup between new and old design?

Ans. 4 stages, stage delay = (20ns, 80ns, 50ns, 10nsec)
 $t_p = 80 \text{ nsec}$

New design: $K=5$ stage.

larger stage
80



Stage delay = [20nsec, 40nsec, 40nsec, 50nsec, 10nsec]

$t_{p \text{ new}} = 50 \text{ nsec}$

$$S = \frac{\text{performance of old}}{\text{performance of new}} = \frac{80}{50}$$

$$S = 1.6$$

(ii) What is the clock frequency of the new pipeline?

Ans. Frequency $\propto \frac{1}{\text{cycle time}}$

$$\text{Frequency} = \frac{1}{50 \times 10^{-9}} = \frac{1}{50} \times 10^9 = \frac{1000 \times 10^6}{50}$$

$$= 20 \text{ MHz}$$

Q6. The stage delay in 5 stage pipeline are 900, 600, 550, 450 and 400 nanoseconds. The larger stage delay is replaced with a functionally equivalent design involving two stages with respective stage delay 440 and 460 nanoseconds. What is the throughput increase of the pipeline?

Ans. Old design:
 $K=5$, Stage delay = (900, 600, 550, 450, 400)

$$t_p = \max(\text{stage delay}) \\ = 900 \text{ nsec}$$

$$1 \text{ Inst. in old pipe} = 900 \times 10^{-9} \text{ sec}$$

$$\text{In 1 sec \# instr. ex.} = \frac{1}{900 \times 10^{-9}}$$

$$\text{Throughput} = \frac{1}{t_p} = \frac{1}{900 \text{ nsec}}$$

New design:

$$K=6$$

$$\text{Stage delay} = (440, 460, 600, 550, 450, 400)$$

$$t_p = \max(\text{delay})$$

$$t_{p, \text{new}} = 600 \text{ nsec}$$

$$\text{Throughput}_{\text{new}} = \frac{1}{600}$$

$$\% \text{ of throughput increment} = \frac{\text{new} - \text{old}}{\text{old}} \Rightarrow \frac{1/600 - 1/900}{1/900} = \frac{1/6 - 1/9}{1/9}$$

$$= \frac{9-6}{54} \times \frac{9}{9} = \frac{3}{54} = \frac{1}{18}$$

$$= 50\%$$

Q7. A 4-stage pipeline has the stage delays as 150, 120, 160 and 140 nsec respectively. Registers that are used between the stages have a delay of 5 nsec each. Assuming constant clocking rate, the total time taken to process 1000 data items on the pipeline will be

Ans. $N=1000$ $K=4$ $t_p = \max(\text{stage delay} + \text{buffer delay})$
 $t_p = 165 \text{ nsec} (160+5)$

$$E_{\text{pipe}} = [K + (N-1)] t_p = [4 + (1000-1)] \times 165 \times 10^{-9}$$

$$= 1003 \times 165 \times 10^{-9}$$

$$= 165.5 \times 10^{-6} \Rightarrow 165.5 \text{ micro seconds}$$

Q8. We have two designs, D₁ and D₂ for a synchronous pipeline processor. D₁ has 5 pipeline stages with execution times of 2, 2, 4, 2, and 3 nsec. while the design D₂ has 8 pipeline stages each with 2 nsec execution time. How much time can be saved using design D₂ over design D₁ for executing 100 instructions?

Ans. Design D₁
 $K=5$, Stage delays = (2, 2, 4, 2, 3)
 $t_p = \max(\text{stage delays}) = 4 \text{ nsec}$
 $ET_{D_1} = [K + (n-1)] t_p$
 $= [5 + (100-1)] \times 4$
 $ET_{D_1} = 416 \text{ nsec}$

Design D₂
 $K=8$, $t_p = 2 \text{ nsec}$
 $ET_{D_2} = [K + (n-1)] t_p$
 $= [8 + (100-1)] \times 2$
 $ET_{D_2} = 214 \text{ nsec}$

$$\text{Time Saved} = 416 - 214 \Rightarrow \underline{202 \text{ nsec}}$$

Q9. A five stage pipeline has stage delays of 150, 120, 150, 160 and 140 nanoseconds. The registers that are used between the pipeline stages have a delay of 5 nanoseconds each.

The total time to execute 100 independent instructions on the pipeline, assuming there are no pipeline stalls, is 17160 nsec.

Ans. $K=5$ Stage delay = (150, 120, 150, 160, 140)
 buffer delay = 5 nsec
 $t_p = \max(\text{stage delay} + \text{buffer delay}) = 160 + 5$
 $= 165 \text{ nsec}$

$$ET_{\text{pipe}} = [K + (n-1)] t_p$$

$$= [5 + (100-1)] \times 165$$

$$= 104 \times 165 = \underline{17160 \text{ nsec}}$$

Q10. Consider a 3-stage pipelined processor having a delay of 10ns, 20ns and 14ns for the first, second and third stages respectively. Assume that there is no other delay and the

processor does not suffer from any pipeline hazards. Also assume that one instruction is fetched every cycle.

The total execution time for executing 100 instructions on the processor is _____ ns.

Ans $K=3$, Stage delay = (10, 20, 14 nsec) $N=100$

$$t_p = \max(10, 20, 14) = 20$$

$$\begin{aligned} ET_{\text{pipe}} &= [K + (N-1)] t_p \\ &= [3 + (100-1)] \times 20 \\ &= 100 \times 20 = 2000 \text{ nsec} \end{aligned}$$

Q11. Consider the following processors. Assume that the pipeline registers have zero latency.

P₁: Four stage pipeline with stage latencies 1ns, 2ns, 2ns, 1ns.

P₂: " " " " " " 1ns, 1.5ns, 1.5ns, 1.5ns

P₃: Five stage " " " " " 0.5, 1, 1, 0.5, 1ns

P₄: " " " " " 0.5, 0.5, 1, 1, 1.1ns

Which processor has the highest clock (peak) frequency?

Ans P₁ $\Rightarrow t_{p1} = \max(1, 2, 2, 1) = 2 \text{ nsec}$

P₂ $\Rightarrow t_{p2} = \max(1, 1.5, 1.5, 1.5) = 1.5 \text{ nsec}$

P₃ $\Rightarrow t_{p3} = \max(0.5, 1, 1, 0.5, 1) = 1 \text{ nsec}$

P₄ $\Rightarrow t_{p4} = \max(0.5, 0.5, 1, 1, 1.1) = 1.1 \text{ nsec}$

Frequency $\propto \frac{1}{\text{Time}}$ $\therefore$ P₃ has highest clock frequency.

Q12. The stage delays in a 4-stage pipeline are 800, 500, 400 and 300 picoseconds. The first stage (with delay 800 picoseconds) is replaced with a functionally equivalent design involving two stages with respective delays 600 and 350 picoseconds. The throughput increase of the pipeline is 33.33 percent.

Ans Old design: $t_{p\text{old}} = \max(800, 500, 400, 300) = 800 \text{ nsec}$

$$t_{p\text{new}} = \frac{1}{\text{clk}}$$

New design: $f_p \text{ new} = \max(600, 350, 500, 450, 300) = 600 \text{ ps}$
 $\therefore \text{Throughput} = \frac{1}{600}$

$\therefore \text{Increment in Throughput} = \frac{\text{New} - \text{Old}}{\text{Old}} = \frac{\frac{1}{600} - \frac{1}{800}}{\frac{1}{800}}$
 $= \frac{8-6}{48} \times \frac{8}{1} = \frac{2}{48} \times \frac{8}{1} = \frac{16}{48} = \frac{1}{3}$
 $= \underline{\underline{33.3\%}}$

Note: In the Successful Pipeline

Cycle per instruction = 1.

$\therefore$ for n instructions $ET_{\text{pipe}} = [k + (n-1)] \text{ cycle}$

1 instruction = $\frac{k + n - 1}{n} = \frac{n}{n} = 1$ ($k-1$ is ignored)

$\therefore \text{CPI} = 1$

Q1. How to Construct (Design) the pipeline?

Ans. If we want to construct 'N' stage pipeline, the entire Cpu (Control unit) is divided into 'N' functional unit (Independent functional unit) which is independent from each other.

Independent function unit means when one functional unit performing one task, at the same time other functional unit performing the other task/operation.

eg: Instruction Pipeline

At clock cycle no.2

WB				
Ex				
Do		I ₁		
If	I ₁	I ₂		
	1	2	3	4

I₁ is in Decode stage and
 I₂ is in Fetch stage at
 the same time (clock cycle 2)

Q2. What is the meaning of $CPI=1$ in Pipeline?

Ans. Characteristic of Pipeline is in every new cycle new input must be inserted into the pipeline.

$$CPI = 1.$$



clock cycle

Q3. Why Clock is required?

Ans. Because whenever we enable the clock then operations are performed and to provide the synchronization between the stages.

Q4. How to Set this CPI in uniform delay Pipeline?

Ans. Uniform delay: $t_p = \text{Stage delay}$

If buffer delay is given: $t_p = \text{Stage delay} + \text{buffer delay}$

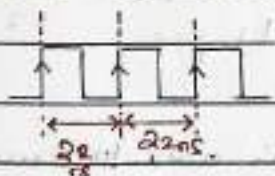
Q5. How to Set this CPI in non uniform delay Pipeline?

Ans. $t_p = \max(\text{Stage delay})$

If buffer delay is included: $t_p = \max(\text{stage delay} + \text{buffer delay})$

Q6. Why don't we take minimum in calculating t_p ?

Ans.



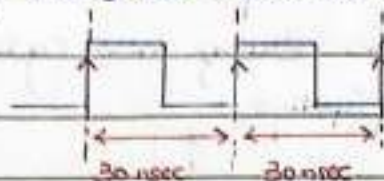
$$t_p = 20 + 2 = 22 \text{ nsec}$$

$$F = \frac{1}{22} \text{ nsec}$$

If we take minimum (22 nsec) then Stage 1 work finish and data move from Stage 1 to stage 2. But in 22 nsec Stage 2, Stage 3 and Stage 4 work not finish.

So for proper functioning and working we take maximum

If we take maximum



$$t_p = 28 + 2$$

$$= 30 \text{ nsec}$$

$$F = \frac{1}{30 \text{ nsec}}$$

If we take maximum 30 ns but stage 1 task finish at 28 nsec
But clock is set to be 30 nsec. Every phase output will be available after 30 nsec. } Synchronization

When one cycle complete the data given from stage 1 to stage 2
so Synchronization

Q1. Consider a non-pipelined processor with a clock rate of 2.5 GHz and average cycles per instructions of four. The same processor is upgraded to a pipelined processor with five stages but due to the internal pipeline delay the clock speed is reduced to 2 GHz. Assume that there are no stage in the pipeline. The speedup achieved in the pipelined processor is _____

Ans

Non pipeline

Each instruction takes = 4 cycle

$$\text{Cycle time} = \frac{1}{2.5} \text{ GHz}$$

$$\text{Cycle time} = 0.4 \text{ nsec}$$

$$\text{ET in non pipeline (t}_{np}) = 4 \times 0.4 \text{ nsec} \\ = 1.6 \text{ nsec}$$

Pipeline

Each instruction takes 1 cycle

$$\text{CPI} = 1 \text{ cycle}$$

$$\text{Cycle time} = \frac{1}{2 \text{ GHz}} = 0.5 \text{ nsec}$$

$$t_p = 0.5 \text{ nsec}$$

$$S = \frac{\text{ET}_{\text{non pipe}}}{\text{ET}_{\text{pipe}}} = \frac{1.6}{0.5} = 3.2$$

Q2. Consider a 4 stage pipeline processor. We want to execute a loop:
For ($i=1; i \leq 100; i++$) { T_1, T_2, T_3, T_4 } where the time taken (nsec)
by instructions T_1 to T_4 for stages S_1, S_2, S_3, S_4 is shown below

	S_1	S_2	S_3	S_4
T_1	1	2	1	2
T_2	2	1	2	1
T_3	1	1	2	1
T_4	2	1	2	1

The output of T_i for $i=2$ will be available after ?

Ans.

	S ₁	S ₂	S ₃	S ₄
I ₁	↓ 1	3	4	6
I ₂	↓ 3	4	6	7
I ₃	↓ 4	5	8	9
I ₄	↓ 6	7	10	11
I ₅	7	9	11	(13)

Ans.

Output of I₅ for i=2 will be 13

Q3. Consider a 4-stage Pipeline Processor. The number of cycles needed by the four instructions I₁, I₂, I₃, I₄ in stages S₁, S₂, S₃, S₄ is shown below:

	S ₁	S ₂	S ₃	S ₄
I ₁	2	1	1	1
I ₂	1	3	2	2
I ₃	2	1	1	3
I ₄	1	2	2	2

What is the number of cycle needed to execute the following loop: for (i=1 to 2) { I₁, I₂, I₃, I₄ }.

Ans.

	S ₁	S ₂	S ₃	S ₄
I ₁	↓ 2	3	4	5
I ₂	↓ 3	6	8	10
I ₃	↓ 5	7	9	13
I ₄	↓ 6	9	11	15
I ₁	↓ 8	10	12	16
I ₂	↓ 9	3	15	18
I ₃	↓ 11	14	16	21
I ₄	↓ 12	16	18	(23)

Ans

23 cycle will be needed to execute the loop

Q4. Consider a 3GHz processor with a three-stage pipeline and stage latencies τ_1 , τ_2 , τ_3 such that $\tau_1 = 3\tau_2/4 = 2\tau_3$. If the longer pipeline stage is split into two pipeline stages of equal latency, the new frequency is 4 GHz, ignoring delays in the pipeline registers.

Ans. $T_1 = \frac{3T_2}{4} = 2T_3$

$$T_1 = \frac{3T_2}{4} \quad \frac{3T_2}{4} = 2T_3$$

$$\frac{T_1}{T_2} = \frac{3}{4} \left[\frac{6}{8} \right] \quad \frac{T_2}{T_3} = \frac{8}{3}$$

$$T_1 : T_2 : T_3 = 6 : 8 : 3$$

Assume time = x

$$T_1 = 6x, T_2 = 8x, T_3 = 3x$$

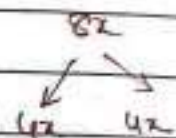
$$t_p = \max(6x, 8x, 3x) = 8x$$

$$\text{Frequency} = \frac{1}{\text{Time}}$$

$$3 \text{ GHz} = 1/8x$$

$$\frac{1}{x} = 24 \text{ GHz} \text{ --- eq(1)}$$

New design:



$$t_{p\text{new}} = \max(6x, 4x, 4x, 3x)$$

$$t_{p\text{new}} = 6x$$

$$\text{Frequency} = \frac{1}{\text{Time}}$$

$$= \frac{1}{6x} \Rightarrow \frac{1}{6} \times \frac{1}{x}$$

$$= \frac{1}{6} \times 24 \text{ GHz}$$

$$\therefore 4 \text{ GHz}$$

Q5. Consider two processors P₁ and P₂ executing the same instruction set. Assume that under identical conditions, for the same input, a program running on P₂ takes 25% less time but incurs 20% more CPI (clock cycle per instruction) as compared to the program running on P₁. If the clock frequency of P₁ is 1 GHz, then the clock frequency of P₂ (in GHz) is

Ans. $E_T = I_C \times \text{CPI} \times \text{Cycle time}$

In the question same no. of instructions

$$E.T = \text{cycle/instruction} \times \text{cycle time}$$

$$E.T_{P1} = \text{CPI} \times \text{cycle time}_1$$

$$E.T_{P2} = 1.2 \text{ CPI} \times \text{cycle time}_2$$

$$E.T_{P2} = 0.75 \times E.T_{P1}$$

$$\text{Cycle time } P_1 = \frac{1}{1 \text{ GHz}} = 1 \text{ nsec}$$

$$0.75 \times E.T_{P1} = 1.2 \text{ CPI} \times \text{cycle time}_2$$

$$0.75 \times \text{CPI} \times 1 \text{ nsec} = 1.2 \text{ CPI} \times \text{cycle time}_2$$

$$\text{Cycle time } q = \frac{0.75}{1.2} = 0.625 \text{ nsec}$$

$$\text{Clock frequency} = \frac{1}{0.625 \text{ nsec}} = 1.6 \text{ GHz}$$

Q6. Consider a machine with 40 MHz processor which has run a benchmark program. The executed program consist of 100,000 instruction executions, with the following instruction mix and clock cycle count. What will be the effective CPI, MIPS rate and execution time.

Instruction Type	Instruction Count	Cycle/Instruction
Integer arithmetic	45000	1
Data transfer	32000	2
Floating point	15000	2
Control transfer	8000	2

Ans. Avg CPI = $0.45 \times 1 + 0.32 \times 2 + 0.15 \times 2 + 0.08 \times 2$

Avg CPI = 1.55 cycle

Cycle time = $\frac{1}{40 \text{ MHz}}$

Avg. Per instruction ET = $1.55 \times \frac{1}{40} \times 10^{-6} \text{ sec} \rightarrow 0.03875 \times 10^{-6}$

1 Instruction ET = $0.03875 \times 10^{-6} \text{ sec}$

In 1 sec no of instructions = $\frac{1}{0.03875} \times 10^6 \text{ Inst/sec}$

= 25.8 MIPS

Total program ET:

= $0.03875 \times 10^{-6} \times 10^5$

= 3.875×10^{-3}

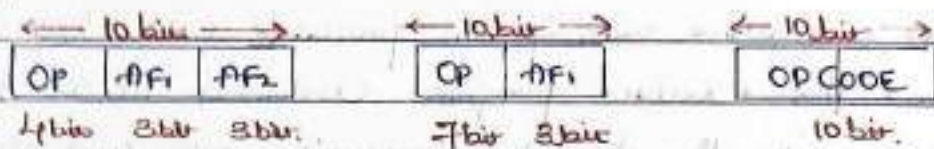
= 3.875 msec

Q7. Which of the following is not a form of main memory?

Ans. Instruction Opcode

Q8. In a 10-bit Computer instruction format, the size of the address field is 3 bit. The Computer uses Expanding Op code technique and has 4 two-address instructions and 16 one-address instructions. The number of zero-address instructions it can support is

Ans



Primitive

Derived

Further derived

Primitive: Total number of operations = $2^4 = 16$ operations.

Given $2AF = 4$

Free opcode after allocating $2AF = 16 - 4 = 12$

Total # operations in IAF = free opcode $\times 2$

$$= 12 \times 2^4 \Rightarrow 12 \times 2^3$$

$$\Rightarrow 96$$

Given IAF = 16

Free opcode after allocating IAF = $96 - 16 = 80$

Total # operations in OAF = $80 \times 2^{10-7} = 80 \times 2^3$

$$\Rightarrow 640$$

Q9.

Design D ₁	50ns	45ns	60ns	80ns
Design D ₂	60ns	52ns	75ns	65ns

What is the cycle time latency of one instruction and throughput in the pipeline design D₁ and D₂ if in both design D₁ and D₂ having buffer register between each stage have a delay of 20 ns?

Buffer delay = 20ns

Ans

Design D₁

$$tp_{D1} = \max(\text{stage delay}, \text{buffer delay})$$

$$= \max(80, 20)$$

$$tp_{D1} = 80$$

Design D₂

$$tp_{D2} = \max(\text{stage delay}, \text{buffer delay})$$

$$= (80 + 20)$$

$$tp_{D2} = 100$$

Latency of one instruction

$$ET_{D1} = 4 \times 820 = 3280 \text{ nsec}$$

Latency of one instruction

$$ET_{D2} = 4 \times 840 = 3360 \text{ nsec}$$

$$\text{Throughput} = \frac{1}{tp} = \frac{1}{820 \text{ ns}}$$

$$\text{Throughput} = \frac{1}{840 \text{ ns}}$$

$$\rightarrow D1 = (820 \text{ ns}, 3280 \text{ ns}, 1/820)$$

$$\rightarrow D2 = (840 \text{ ns}, 3360 \text{ ns}, 1/840) \therefore \text{Pipeline } D1 \text{ performing better than } D2.$$

Pipeline Stages

① 2-Stage Pipeline

FI

Ex

② 4-Stage Pipeline

I ₁ :	IF	ID	EX	WB			
I ₂ :		IF	ID	EX	WB		
I ₃ :			IF	ID	EX	WB	
I ₄ :				IF	ID	EX	WB

IF : Instruction Fetch

ID : Instruction Decode

EX : Execute

WB : Write back

③ 6-Stage Pipeline $\Rightarrow$

FI	DI	CO	FO	EE	WO
----	----	----	----	----	----

Additional Stages:

① Fetch Instruction:

Read the next expected instructions from buffer

②

Decode Instruction:

* Determine the opcode and the operand specifier

③ Calculate Operands (Co)

* Calculate the effective address of each source operand

* This may involve displacement register indirect or other forms of address calculations

④ Fetch Operands (Fo)

* Fetch each operand from memory

* Operands in register need not be fetched

⑤ Executed Instruction (EI)

* Perform the indicated operation and store the result, if any, in the specified destination operand locations.

⑥ Write Operand (WO)

* Store the result in memory.

Note: Generally we use Classical Pipelines:

Classical Pipeline	MIPS Pipeline	Standard Pipeline	RISC Pipeline
-----------------------	------------------	----------------------	------------------

RISC Pipeline: 5 Stages

In RISC pipeline 5 Stages:

1. Instruction Fetch (IF)
2. Instruction Decode (ID)
3. Execute (EX)
4. Memory Access (MA)
5. Write Back (WB)

1. Instruction Fetch: In this stage memory instruction is fetched from memory.

2. Instruction Decode: In this stage 2 operations are performed:
 (i) Decode the instruction
 (ii) Operand loading (fetching) from the register file.
 This stage also contains Comparator Circuit to evaluate the branch condition.

3. Execute: In this stage data processing (Arithmetic) are performed.

4. Memory Access : In this stage Operand (Data) will be accessed from memory (load or store).

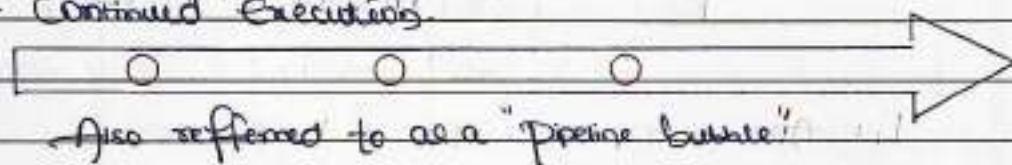
5. Write Back : In this stage register write (Operand) storing into reg. file, Operation performed.

Pipeline Hazards

Occur when the Pipeline, or some portion of the pipeline must stall because conditions do not permit continued execution.

There are 3 types of hazard:

1. Resource
2. Data
3. Control



→ Hazard is a situation that makes the Pipeline to idle or stall.

1. Structural hazard (Resource conflict)
2. Data hazard (Data/operand)
3. Control hazard (Branch instructions)

→ In general, there are three major difficulties that cause the instruction Pipeline to deviate from its normal operations:

* Resource conflicts caused by access to memory by two segments at the same time. Most of these conflicts can be resolved by using separate instruction and data memories.

* Data dependency conflicts arise when an instruction depends on the result of a previous instruction, but the result is not yet available.

* Branch difficulties arise from branch and other instructions that change the value of pc.

Hazards/Dependencies in the pipeline:

- * Dependency is a major problem in the pipeline, causes extra cycle.
- * Cycle in the pipeline without new input is called as extra cycle. Also named as "Stall".
- * When stall is present in the pipeline then $CPI \neq 1$

eg: 5 Stage Pipeline: Normal Execution.

	CC1	CC2	CC3	CC4	CC5	CC6	CC7
I ₁	IF	ID	Ex	MA	WB		
I ₂		IF	ID	Ex	MA	WB	

I₁: Add r1, r2, r3: $r1 \leftarrow r2 + r3$

I₂: mul r1, r1, r5: $r1 \leftarrow r1 * r5$

Instructions I₂ data dependent on I₁

	CC1	CC2	CC3	CC4	CC5	CC6	CC7
I ₁	IF	ID	Ex	MA	WB		
I ₂		IF	ID	"	"		

Stall
Bubble/Extra Cycle.

Structural Dependency/Hazards

- * Structural dependency is created in the pipeline when two or more phase require the same resource at the same time and they are not able to run simultaneously.
- * Structural dependency is created in the pipeline due to resource conflict.
- * Resource maybe registers, functional unit, ALU, memory etc.

eg: 1.

	CC1	CC2	CC3	CC4	CC5	CC6	CC7
I ₁ :	Mem	To	Ex	Mem	WB		
I ₂ :		Mem	To	Ex	Mem	WB	
I ₃ :			Mem	To	Ex	Mem	WB
I ₄ :				Mem	Mem	Mem	Mem

" " " "
 (1) " (2) " (3) "

3 Extra cycles / bubbles / stalls

Here I₁ is accessing the memory to store the data and
I₄ is accessing the memory to fetch the instruction.

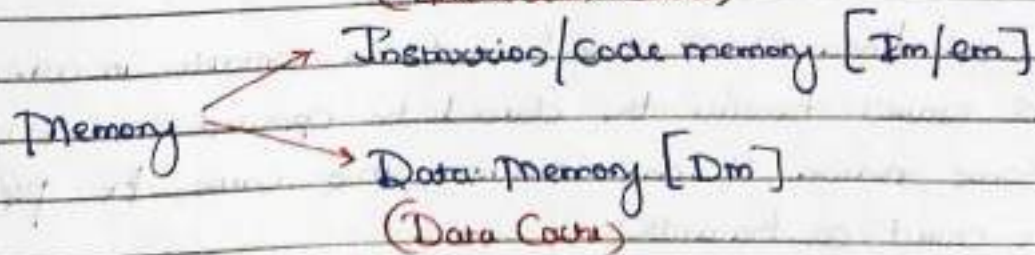
eg: 2.

	CC1	CC2	CC3	CC4	CC5	CC6	CC7
	Mem	To	Ex	Mem	WB		
		Mem	To	Ex	Mem	WB	
			Mem	To	Ex	Mem	WB
				Mem			

Resource Conflict

- * In clock cycle (CC4) both I₁ and I₄ are trying to access the same resource 'Memory'.
- * This situation is called resource conflict so (I₁ & I₄) not go for execution.
- * So keep I₄ into the waiting until the resource become available.
- * This waiting creates stalls/bubbles/extra cycle in pipeline.

(Instruction Cache)



- * To minimize the stalls due to structural dependency one hardware type of instruction mechanism is used called as "reordering".

- * Renaming state then divide the memory into independent module to store the instructions and data separately called Instruction/Code Memory [Cm/Dm] and data memory (Dm) respectively.

	CC1	CC2	CC3	CC4	CC5	CC6	CC7	CC8
I ₁ :	Cm	To	Ex	Dm	WB			
I ₂ :		Cm	To	Ex	Dm	WB		
I ₃ :			Cm	To	Ex	Dm	WB	
I ₄ :				Cm	To	Ex	Dm	WB

No Stalls.

Data Dependency:

I₁: Add r₁, r₂, r₃; r₁ ← r₂ + r₃

I₂: mul r₄, r₁, r₅; r₄ ← r₁ * r₅

Instruction I₂ is data dependent on instruction I₁ because I₂ is using the result of instruction I₁, as operand.

Data dependency is created in the pipeline when next (I_j or I_{j+1}) instructions using the result of previous instructions.

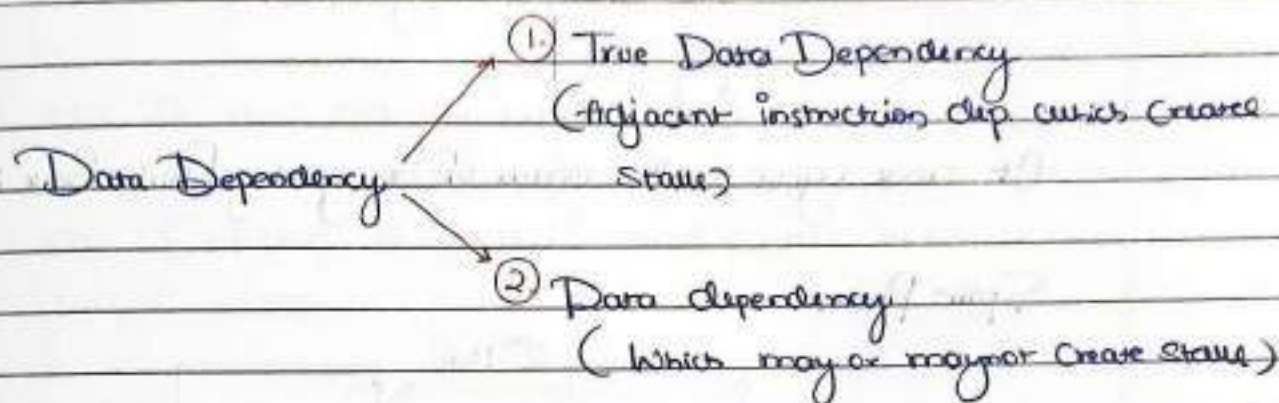
1. Data Dependency/Flow dependency → RAW hazards
2. Anti dependency → WAR hazards
3. Output/Write dependency → WAW hazards

Data Hazard: A data hazard is a situation in which the pipeline is stalled because the data to be operated on are delayed for some reason or any condition that cause the pipeline to stall is called as hazard.

- Data Hazard:
1. RAW (Read after write)
 2. WAR (Write after read)
 3. WAW (Write after write)

Important Points:

- * Every dependency not creates stall.
- * Anti and Output dependency never creates stall.
- * Data dependency may or may not creates stall/bubble/extra cycle.



Q1. Consider the following program segment, Executed on a Risc pipeline. Find how many data dependency?

I₁ : Add r0, r1, r2

I₂ : Sub r3, r0, r2

I₃ : Mul r4, r0, r5

I₄ : Div r0, r0, r7

I₅ : Mul r8, r0, r9

Data dependency

I₂ - I₁

I₃ - I₁

I₅ - I₄

Q2. How to check that dependency creates hazards or not?

Ans. Execution of instructions in a normal pipeline manner. If there is a stall/bubble/extra cycle occur then data dependency creates stall.

Q4. I₁ : Add r0, r1, r2; r0 ← r1 + r2

I₂ : Mul r2, r0, r4; r2 ← r0 * r4

Now check this dependency (I₂ - I₁) creates hazards or not?

Ans. Risc : 5 Stage IF, Io, Ex, MA, WB

	cc1	cc2	cc3	cc4	cc5	cc6	cc7	cc8	cc9
I ₁	IF	Io	Ex	MA	WB				
I ₂		IF	Io	Io	Io	Io	Ex	MA	WB

3 stalls

Correct value.

-At Cc3 I2 read wrong value of r0

-At Cc4 I2 read wrong value of r0

-At Cc5 I2 read wrong value of r0

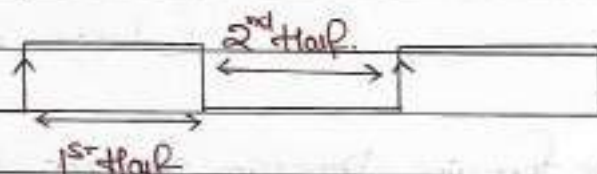
Because r0 available at end of Cc5

So 3 extra stalls/extra cycle/bubbles



At clock cycle 5 the value of r0 getting [write back into register]

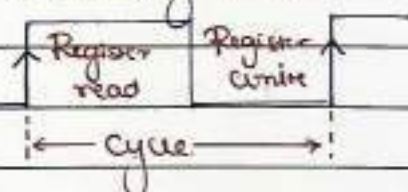
Split Phase



I Case

1st half : Reg Read

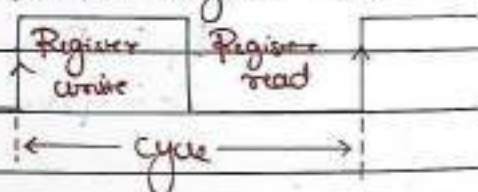
2nd half : reg write



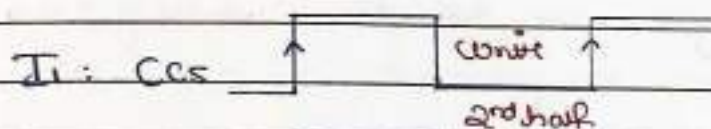
II Case

1st half : register write

2nd half : register read

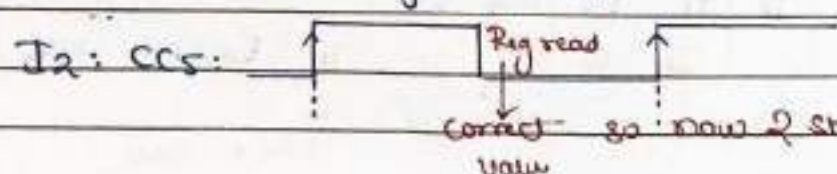
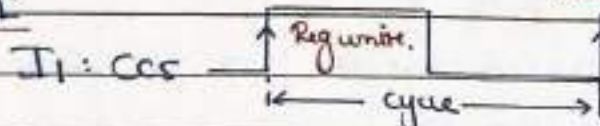


Case I in Previous Question:



old (wrong) value so 3 stalls in previous question

Case II



correct value so now 2 stalls

II Case of previous Questions (Q4)RISC : 5 Stage

	CC1	CC2	CC3	CC4	CC5	CC6	CC7	CC8
I ₁	IF	IO	EX	MA	WB			
I ₂		IF	IO	EX	MA	WB		

2 Stalls

At CC3 I₂ read old value of r0At CC4 I₂ read old (wrong) value of r0At CC5 (2nd pair) of cycle I₂ read correct value of r0.

So 2 stalls.

Data Dependency (Formal Definition)

* Data dependency will be occurred when the instruction 'i' try to read the data before 'i' writes it.

eg: I₁: Add r0, r1, r2 : $r0 \leftarrow r1 + r2$ I₂: mul r3, r0, r2 : $r3 \leftarrow r0 * r2$ * When the above instructions are executed in a non-pipelined system then data dependency condition does not occur because "I₂" executing after "I₁" so, I₂ reads the register (r0) data I₁ writes it (Read-After-Write)* When the above instructions are executed in a pipeline system then data dependency condition will be occurred because (I₂) will be executing along with (I₁) so (I₂) reads the (r0) data before I₁ writes it. Therefore I₂ incorrectly get the old value from (r0) (Data loss)

	CC1	CC2	CC3	CC4	CC5
I ₁	IF	IO	EX	MA	WB
I ₂		IF	IO		

So: r0, r2

D: r3 : r0 = value - - - - Read before write.
r2 = value

To handle above problem, some logic will be maintained in the "Is" stage of pipeline to stop the accessing of a dependent data. Structure of the logic is as follows: "Tomasi's Rule".

S/No	Function Unit	Destination	Independent Source 1	Independent Source 2	Dependent Source 1	Dependent Source 2	Control	Reg. file access
T ₁	Add	R ₀	r ₁	r ₂	-	-	0	r ₁ = value r ₂ = value Ex-stage allowed
T ₂	Mul	r ₃	-	r ₂	r ₀ (T ₁)	-	0	r ₂ = value Ex-stage allowed

"0" Execution yet not complete

"1" execution complete

By using the above logic, we need to stop the access of dependent data until the data becomes available.

This waiting creates extra cycle/bubble/stall in the pipeline.

Solution of Data Dependency (Raw Hazard)

→ Hardware Solution

→ Software Solution (Compiler based)

→ No operation instruction inserted

Hardware Solution of Data Dependency

(1) Hardware interlocks: The most straightforward method is to insert hardware interlocks. An interlock is a circuit that detects instructions whose source operands are destinations of instructions further up in the pipeline.

(2) Operand forwarding: Another technique called operand forwarding uses special hardware to detect a conflict and then avoid

it by routing the data through special paths between pipeline segments. For eg: instead of transferring an ALU result into a destination register, the hardware register checks the destination operand, and if it is needed as a source in the next instruction, it passes the result directly into the ALU input, bypassing the register file.

* To minimize the data dependency stall in the pipeline hardware technique is used i.e. Open forwarding. Also named as Bypassing or Short Circuit.

* This mechanism state that use the buffer between the stage to hold the immediate result so that, dependent instruction can be accessing the new data from the buffer before update register file.

eg: $I_1: r_0 \leftarrow r_1 + r_2$
 $I_2: r_3 \leftarrow r_0 + r_4$

Without Operand forwarding:

		CC ₁	CC ₂	CC ₃	CC ₄	CC ₅	CC ₆	CC ₇
I_1	IF	ID	EX	MA	WA			
I_2			IF	ID	ID	ID	ID	EX WB

3 Stalls

Buffer

With Operand forwarding:

		CC ₁	CC ₂	CC ₃	CC ₄	CC ₅	CC ₆	CC ₇
I_1	IF	ID	EX	MA	WB			
I_2			IF	ID	EX	MA	WB	

If here wrong (old) value of r_0 then in CC₄

Correct value of r_0 using

① Data/Read Dependency (Read After Write)

- * An instruction modifies a register or memory location.
- * Succeeding instruction reads data in memory or register location.

eg. $I_1: \text{Add } r0, r1, r2$ $I_i: r0 \leftarrow r1 + r2$
 $I_2: \text{Sub } r3, r0, r2$ $I_j: r3 \leftarrow r0 + r2$

② Anti Dependency/Write Dependency (Write After Read)

- * An instruction reads a register or memory location.
- * Succeeding instruction writes to the location.

eg: $I_1: \text{Mul } r3, r1, r2$ $I_i: r3 \leftarrow r1 * r2$
 $I_2: \text{Add } r1, r4, r5$ $I_j: r1 \leftarrow r4 + r5$

③ Output Dependency/Write Dependency (Write After Write)

- * Two instructions both write to the same location.

eg: $I_1: \text{Mul } r3, r1, r2$ $I_i: r3 \leftarrow r1 * r2$
 $I_2: \text{Add } r3, r4, r5$ $I_j: r3 \leftarrow r4 + r5$

Types of Data Hazard:① Read after Write, or true dependency

- * An instruction modifies a register or memory location.
- * Succeeding instruction reads data in memory or register location.
- * Hazard occurs if the read takes place before write operation is complete.

② Write After Read, or antidependency

- * An instruction reads a register or memory location.
- * Succeeding instruction writes to the location.
- * Hazard occurs if the write operation completes before the read operation takes place.

③ Write After Write or Output dependency

- * Two instructions both write to the same location.

* Hazards occur if the write operations take place in the reverse order of the intended sequence.

Q1 Consider the following Program Segment, Executed on a Risc Pipeline:
(i) How many RAW dependency? (ii) How many RAW Hazards?

Ans Data (RAW) Dependency:

$I_2 - I_1$

$I_3 - I_1$

$I_4 - I_1$

$I_5 - I_1$

4 Raw dependency

3 Raw Hazards

$I_1: \text{Add } r0, r1, r2$

$I_2: \text{Sub } r3, r0, r4$

$I_3: \text{Mul } r5, r0, r6$

$I_4: \text{Xor } r7, r0, r8$

$I_5: \text{Mul } r9, r0, r10$

	CC1	CC2	CC3	CC4	CC5	CC6	CC7	CC8	CC9
I_1	IF	ID	EX	MA	WB				
I_2		ID	(ID)	EX	MA	WB			
I_3			ID	(ID)	EX	MA	WB		
I_4				ID	(ID)	EX	MA	WB	
I_5					ID	(ID)	EX	MA	WB

After CC5 $\rightarrow r_0$ output available.

4 Raw Dependency

3 Raw Hazards

Q2 Consider the following Program Segment, Executed on a Risc Pipeline:
Check RAW dependency Create RAW Hazards?

Ans $I_1: \text{Add } r0, r1, r2$

Raw Dependency

$I_2: \text{Sub } r3, r4, r5$

$I_1 - I_3$

Raw Hazard : $I_3 - I_1$

$I_3: \text{Mul } r6, r0, r7$

$I_1 - I_5$

$I_4: \text{Add } r7, r5, r8$

$I_5: \text{Add } r9, r0, r10$

	CC1	CC2	CC3	CC4	CC5	CC6	CC7	CC8	CC9
I_1	IF	ID	EX	MA	WB				
I_2		ID	ID	EX	MA	WB			
I_3			ID	(ID)	EX	MA	WB		
I_4				ID	ID	EX	MA	WB	
I_5					ID	(ID)	EX	MA	WB

r_0 available after CC5

Raw hazard : $I_3 - I_1$

Q3. How to check RAW Dependencies create RAW hazards or not?

Ans Pipeline Execution then check if that result is required earlier or not by next instructions.

Q4 Consider a pipelined processor with the following four stages: IF, ID, EX, WB.

The IF, ID and WB stages take one clock cycle each to complete the operation. The number of clock cycles for the EX stage depends on the instructions. The ADD and SUB instructions need 1 clock cycle and the MUL instruction needs 3 clock cycles in the EX stage. Operand forwarding is used in the pipelined processor. What is the number of clock cycles taken to complete the following sequence of instructions?

ADD R2, R1, R0 $R2 \leftarrow R1 + R0$

MUL R4, R3, R2 $R4 \leftarrow R3 * R2$

SUB R6, R5, R4 $R6 \leftarrow R5 - R4$

Ans Without Operand Forwarding:

	CC1	CC2	CC3	CC4	CC5	CC6	CC7	CC8	CC9	CC10	CC11	CC12
I1	IF	ID	EX	WB								
I2		ID	EX	EX	ID	EX	EX	EX	WB			
I3			ID	EX	EX	EX	EX	EX	WB	ID	EX	WB

→ 12 clock cycles without Operand Forwarding

	1	2	3	4	5	6	7	8	9	10	11	12
WB				I1					I2			I3
EX			I1			I2	I2	R2			R3	
ID		I1	I2	I2	I2				I3			
IF	I1	I2	I3									

Without Operand Forwarding → 12 clock cycles

With Operand Forwarding

	cc1	cc2	cc3	cc4	cc5	cc6	cc7	cc8	cc9
I ₁	IF	IO	Ex	WB					
I ₂		IF	IO	Ex	Ex	Ex	WB		
I ₃			IF			IO	Ex	WB	

8 Clock cycle with Operand Forwarding

WB				I ₁			I ₂	I ₂	
Ex			I ₁	I ₂	I ₂	I ₂	I ₃		
IO		I ₁	I ₂	I ₂	I ₂	I ₃			
IF	I ₁	I ₂							

Ans: 8 Clock Cycles

Q5. Consider the sequence of machine instructions given below:

Mul R5, R0, R1

Div R6, R2, R3

Add R7, R5, R6

Sub R8, R7, R4

In the above sequence, R0 to R8 are general purpose registers. In the instructions shown, the first register shows the result of the operation performed on the second and third registers. This sequence of instructions is to be executed in a pipelined instruction processor with the following 4 stages:

(1) Instruction fetch and decode

(2) Operand fetch

(3) Perform Operation

(4) Write back the result

(5) The PO stage takes 1 clock cycle for Add or Sub instructions.

3 clock cycle for Mul instructions and 5 clock cycle for Div instructions. The pipelined processor uses operand forwarding from the PO stage to the OF stage. The number of clock cycle taken for the execution of the above sequence of instructions is 13 clock cycle.

Ans 1

WB						I ₁					I ₂	I ₃	P ₄
PO			I ₁	I ₁	I ₁	I ₂	I ₂	I ₂	I ₂	I ₂	I ₂	I ₄	
OF		I ₁	I ₂								I ₃	P ₄	
IF	I ₁	I ₂	I ₃	I ₄									
	1	2	3	4	5	6	7	8	9	10	11	12	13

∴ 13 clock cycle

Another Approach:

$N=4$ (I₁, I₂, I₃, I₄)

CPI=1

$K=4$ (4 stages)

Extra

IF 1

Mul 3 2

OF 1

DIV 5 4

PO 1

6 stalls

WB 1

$$\begin{aligned}
 ET_{\text{pipe}} &= [K + (n-1)] t_p \\
 &= [4 + (4-1)] \text{ cycle} \\
 &= 7 \text{ cycle}
 \end{aligned}$$

$$\begin{aligned}
 ET &= ET_{\text{pipe}} + \text{Stalls} \\
 &= 7 + 6 \\
 &= \underline{13 \text{ cycle}} \text{ (Ans)}
 \end{aligned}$$

Q6. The instruction pipeline of a RISC processor has the following stages: IF, IO, OF, PO and WB. The IF, PO, OF and WB stage take 1 clock cycle for every instruction. Consider a sequence of 100 instructions. In the PO stage, 40 instructions take 3 clock cycles each, 35 instructions take 2 clock cycles each, and the remaining 25 instructions take 1 clock cycle each. Assume that there are no data hazards and no control hazards.

The number of clock cycles required for completion of execution of the sequence of instructions is 219.

Ans. $K=5$, $N=100$

Extra (Stalls)

IF : 1

WB : 1

PO : 40 Instr : 3

2

Total stalls = $40 \times 2 +$

IO : 1

35 Instr : 2

1

35×1

OF : 1

25 Instr : 1

0

= 115 stalls

PO : 1

$$E_{\text{pipe}} = [k + (n-1)] \text{ cycle}$$

$$= [2 + (100-1)] \text{ cycle}$$

$$= 104 \text{ cycle}$$

$$\text{Total Et} = E_{\text{pipe}} + E_{\text{extra}}$$

$$= 104 + 15$$

$$\Rightarrow 219$$

- Q7. The performance of the pipelined processor suffer if:
- The pipeline stages have different delays.
 - Consecutive instructions are dependent on each other.
 - The pipeline stages share hardware resources.
 - All of the above.

Note: Anti and Output Dependency does not create any stalls but in Anti and output we are getting the problem of for reading/Updating the correct value.

The solution of Anti and Output dependency is "Register Renaming".
Register Renaming: Use some temporary storage registers to store the result of the out of order instructions output.

Q8. When and how Anti dependency and Output dependency create problem?

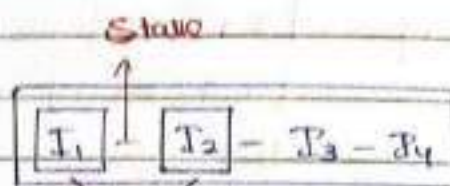
- Ans:
- If the previous instructions taking more time (very high time) compare to next instructions. (Unused delay)
 - When there is true data dependency between two instructions then to minimise the stalls, we try to reschedule the instructions (out of order instructions execution).
 - In Super Scalar system, multiple instructions inserted in the stages of so next instructions if read/write the value that was used in previous instructions, then it creates a problem.

* CPU always execute a sequence the program from top to bottom in a sequence called as in-order sequence execution.

In this execution sequence if any instruction is dependent then the remaining instructions are also sharing the same cycles even

they are independent:

eg: I_1 : Add r_0, r_1, r_2
 I_2 : Sub r_3, r_0, r_4
 I_3 : Mul r_4, r_5, r_6
 I_4 : Div r_2, r_7, r_8



In the above execution, sequence ' I_2 ' is data dependent on I_1 , so I_2 will be waiting until the ' I_1 ' execution is completed when the hardware does not support operand forwarding.

This waiting creates stall in the pipeline this stall also shared by I_3 and I_4 instruction even they are independent.

To utilize this point instruction scheduling concept states that insert the independent instruction first. It causes out of order execution (Re-order execution).

Out of order execution sequence is $I_1 - I_3 - I_4 - I_2$.

Out of order execution sequence create two more dependencies in the pipeline...

- (i) Anti data dependency
- (ii) Output data dependency.

To handle the above dependency problem hardware technique is used i.e. "register renaming".

This technique states that, use the re-order buffer to store the out-of-order ~~buffer~~ instruction output later update the register file with a re-order buffer content after the completion of a dependent instruction therefore data is safe.

Control Dependency:

Control dependency is created in the pipeline due to:

- * Branch instruction
- * Program Control

- * Function Call
- * Transfer of control instruction
 - Conditional Jcc
 - Unconditional Jmp

Assume 5 Stage Pipeline and I_2 is the branch instruction. Assume the target address of the branch instruction is available at the end of the last stage.

	1000	I_1	Unconditional Jmp								
I_2 is a	1001	I_2	Jmp 2000								
branch instr	1002	I_3		I_1	S1	S2	S3	S4	S5		
	1003	I_4		I_2		S1	S2	S3	S4	S5	
	1004	I_5		I_3			S1	S2	S3	S4	
	1005	I_6		I_4				S1	S2	S3	
	1006	I_7		I_5					S1	S2	
2000	BI_1			I_6						S1	

2000 BI_1 2001 BI_2 I_3, I_4, I_5 and I_6 is unwanted instruction BI_1 is the wanted instruction

					I_1	I_2					
S5					I_1	I_2	I_3				
S4				I_1	I_2	I_3	I_4				
S3			I_3	I_2	I_3	I_4	I_5		BI_1		
S2		I_1	I_2	I_3	I_4	I_5	I_6		BI_1		
S1	I_1	I_2	I_3	I_4	I_5	I_6	BI_1				
	1	2	3	4	5	6	7	8	9	10	

4 Stalls

Assume the target address of the branch address of branch instruction is available at end of 4th stage.

					I_1	I_2					BI_1
S5					I_1	I_2				BI_1	
S4				I_1	I_2	I_3			BI_1		
S3			I_1	I_2	I_3	I_4		BI_1			
S2		I_1	I_2	I_3	I_4	I_5		BI_1			
S1	I_1	I_2	I_3	I_4	I_5	I_6	BI_1				

3 Stalls

Assume the target address of the branch instruction is available at the end of 3rd stage.

S ₆				I ₁				BT ₁	
S ₅				I ₁				BT ₁	
S ₄			I ₁	I ₂				BT ₁	
S ₃		I ₁	I ₂	I ₃		BT ₁			
S ₂		I ₁	I ₂	I ₃		BT ₁			
S ₁	I ₁	I ₂	I ₃	I ₄	BT ₁				
	1	2	3	4	5	6	7	8	9

2 Stalls

If the target address of branch instruction is available at k^{th} stage

$$\# \text{ stalls from branch instruction} = k - 1$$

Ideal CPI = 1

But due to control hazard / Control dependency

$$\text{Average Instruction ET} = 1 + \# \text{ instructions which create stalls} \times \# \text{ stalls from branch}$$

$$\text{Average Instruction ET} = 1 + \left(\text{Branch Instr. Frequency} \times \# \text{ stalls due to branch instr.} \right)$$

Control Dependency

- * Control dependency is caused in the pipeline due to execution of branch (Jcc instruction). So program execution will be changed.
- * In this the next sequential instruction become wanted or unwanted, it depends on the output previous instruction decoding.
- * If the previous instruction decodes the instruction as simple instruction (Arithmetic, data transfer etc.) then the next instruction become wanted instruction.

- If the previous instruction decode the instruction as branch instruction, unconditional T_{cc} , Conditional T_{cc} gets True then next sequential instruction become unwanted

It is $\rightarrow$

1000	I_1
1001	I_2
1002	I_3
1003	I_4
1004	I_5
1005	I_6
1006	I_7

a branch instruction.

$Jmp 2000$

When I_1 execute then I_2 is wanted instruction.

But when I_2 execute (decode) then $I_3, I_4 \dots$ is a wanted instruction.

Because I_2 decodes as unconditional T_{cc} ($Jmp 2000$) so I_1 is the new wanted instruction.

- The number of stalls created due to branch operation is called Branch Penalty.
- This branch penalty value depends on the outcome of the target address.

Note: In the Risc pipeline branch penalty value is '1' because target address is available at the end of 2nd stage.

$$\text{Branch penalty} = K - 1 = 2 - 1 = 1$$

- Generally target address is available in decoding stage/phase but condition checking done in execution phase/stage.

- If target address available at 'K' stage

$$\text{Branch penalty} = K - 1$$

$$\text{Branch penalty} = \left(\begin{array}{l} \text{At which stage} \\ \text{Target address available} \end{array} \right) - 1$$

$\rightarrow$ Avg $CPI = 1$ But due to Control Hazard

$$CPI = 1 + \left(\frac{\text{Branch instr}}{\text{freq}} \times \frac{\# \text{ stalls due to branch}}{\text{freq}} \right)$$

Q1. Suppose we have 100 instructions. Out of 100 instructions 30 are branch type, which create 3 stall cycles then avg CPI?

Ans. 100 Instructions. Total # = no. of branch instructions $\times$ # stall cycles/branch

$$ET = 100 \times 1 = 100$$

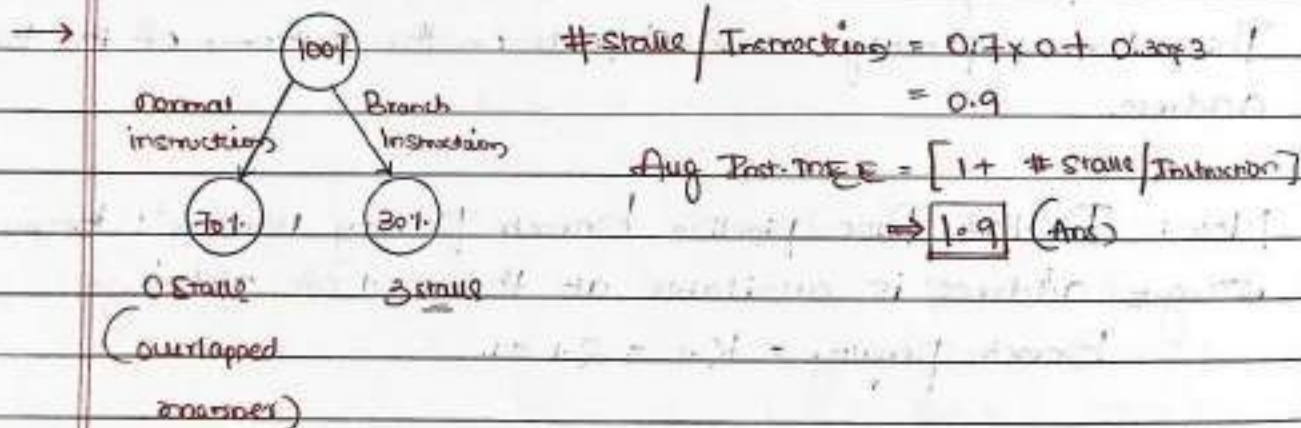
$$\Rightarrow 30 \times 3$$

$$\text{For total 100 instr.} = 100 + 90 = 190$$

$$\Rightarrow 90$$

$$\text{Avg CPI} = \frac{190}{100} = \boxed{1.9} \text{ (Ans)}$$

$$\begin{aligned} \rightarrow \text{Average instructions ET} &= 1 + \left(\frac{\text{branch instructions}}{\text{frequency}} \times \frac{\# \text{ stall cycles}}{\text{branch instruction}} \right) \\ &= 1 + (0.30 \times 3) \\ &= 1 + 0.9 \\ &\Rightarrow \boxed{1.9} \text{ (Ans)} \end{aligned}$$



Stalling: Design a pipeline in such a manner, if it detect the branch instructions, then we have to stall (stop feeding the next instructions into the pipeline) until branch instructions evaluating (execution) not completed.

Solution of Control Dependency:

- Hardware Solution $\rightarrow$ Branch prediction
- Software Solution $\rightarrow$ Delayed Branch

Branch Prediction: In this we predict that the branch is taken or branch is not taken and if prediction is correct then accordingly instructions will execute.

(i) Branch is always taken: In this we assume everytime branch is taken, so in this next instruction is target address instructions for execution.

(ii) Branch is never taken: In this we assume everytime branch is not taken, so in this next instruction is the next sequential instruction for execution.

Note: If prediction goes wrong then we have to suffer from stalls.

Dynamic Prediction

1. Bit implementation

2. Branch target / Branch loop / Branch predictions
buffer / buffer / buffer

Branch loop buffer: It is a high speed buffer maintained in the instruction fetch stage of pipeline.

It's just like a database!

Branch instruction address	Target address
1001	2000

PC → Memory

→ Loop buffer

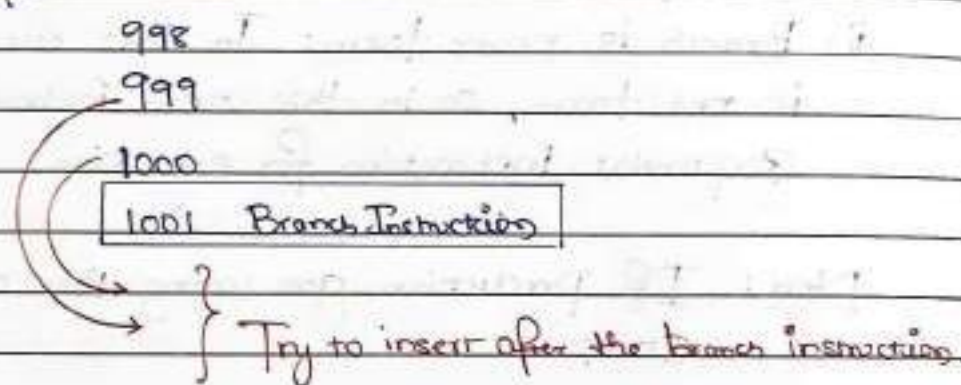
If address matches then go to target address.

When the next time this type of instruction come then if matches with the address directly getting the target address.

Delayed Branch: It is a compile technique (Software Solution) in which trying to rearrange the instructions if possible.

If it is not possible to rearrange, then insert 'Nop' instruction after the branch instruction.

Nop Instruction: This is the instructions that fetch and execute but not update any register or not affect our desired output.



→ 'K' Stage Target Address

Delay slot: $k-1$ delay slot.

Q. Delayed branching can help in the handling of control hazards. The following code is to run on a pipelined processor with one branch delay slot:

I_1 : Add $R_2 \leftarrow R_7 + R_8$

I_2 : Sub $R_4 \leftarrow R_5 - R_6$

I_3 : Add $R_1 \leftarrow R_2 + R_3$

I_4 : Store memory $[R_1] \leftarrow R_1$

Branch to label if $/R_1 == 0$

Which of the above instructions I_1, I_2, I_3 or I_4 can legitimately occupy the delay slot without any other program modification?

Ans I_4 - Can insert in delay slot because no changes in program.

INSTRUCTION PIPELINING (Continued)

Control Dependency

* Generally $CPI = 1$.

$$\text{Avg Instruction Et} = CPI \times \text{Cycle Time}$$

But due to Control Hazard $CPI \neq 1$.

$$CPI = (1 + \# \text{ Stalls/Instruction})$$

$$\# \text{ Stalls per instruction} = \text{Branch instruction frequency} \times \text{Branch penalty}$$

$$\text{Average Instruction Et} = (1 + \# \text{ Stalls/Instruction}) \times \text{cycle time}$$

Speed up factor with Stalls:

$$S = \frac{\text{Avg Instruction Et non pipe}}{\text{Avg Instruction Et pipe}}$$

Ideal CPI of pipeline is always '1' due to a dependency problem. Some Stalls are created in the pipeline.

$$S = \frac{CPI_{\text{non pipe}} \times \text{Cycle time non pipe}}{(1 + \text{no. of stalls/Instruction}) (\text{Cycle time pipe})}$$

When the pipeline stages are perfectly balanced then 1 task execution time in the non-pipeline is also equal to number of stages in the pipeline i.e. $t_n = k \cdot t_p$ Under this condition:

$$\Rightarrow S = \frac{k \cdot t_p}{(1 + \text{no. of Stalls/Instruction}) (\text{Cycle time pipe})}$$

$$\Rightarrow S = \frac{k}{(1 + \text{number of stalls/Instruction})}$$

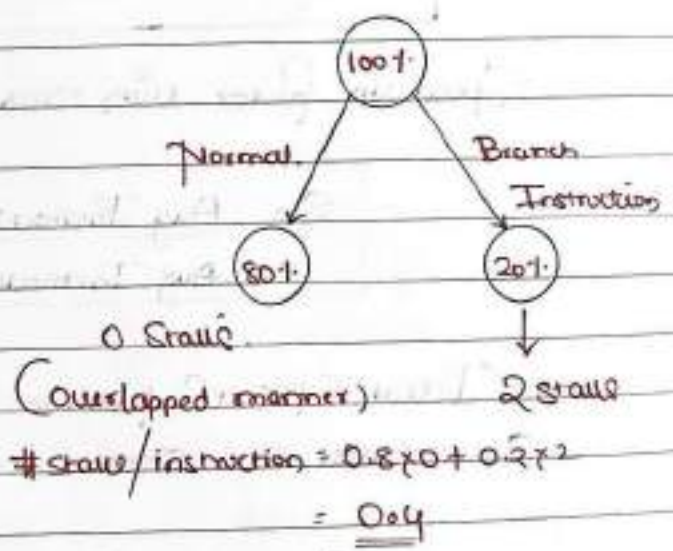
Q1. A CPU has five stage pipeline and runs at 1GHz frequency. Instruction fetch happens in the first stage of pipeline. A conditional branch instruction computes the target address and evaluates the condition in the third stage of the pipeline. The processor stops fetching new instructions following a conditional branch until the branch outcome is known. A program executes 10^9 instructions out of which 20% are conditional branches. If each instruction takes one cycle to complete on average, then total execution time of the program

Ans Frequency = 1GHz $\rightarrow$ Cycle time = $\frac{1}{1\text{GHz}}$ Sec = 1nsec
 5 Stage pipeline
 Total = 10^9 instructions.
 Target address = 3rd stage.

Branch penalty = 3-1 $\Rightarrow$ 2

Avg Instr. = $(1+0.4) \times 1$

ET = 1.4 nsec



Program ET = $1.4 \times 10^{-9} \times 10^9$
 = 1.4 sec (Ans)

Another Approach:

Avg CPI = $0.80 \times 1 + 0.2 \times 3$
 = $0.8 + 0.6$
 = 1.4 cycle

Avg Instruction ET = $1.4 \times 1\text{nsec}$
 = 1.4 nsec

Program ET = $1.4 \times 10^{-9} \times 10^9$
 = 1.4 sec (Ans)

- Q2. Consider a 6-stage pipeline, where all stages are perfectly balanced. Assume that there is no cycle-time overhead of pipelining. When an application is executing on this 6-stage pipeline, the speedup achieved with respect to non-pipelined execution if 25% of the instructions incur 2 pipeline stage cycle is _____.

Ans. $K = 6$ Stage
 $t_0 = 6t_c$

$$S = \frac{ET_{nonpipe}}{ET_{pipe}} \quad t_0 = K \times t_p$$

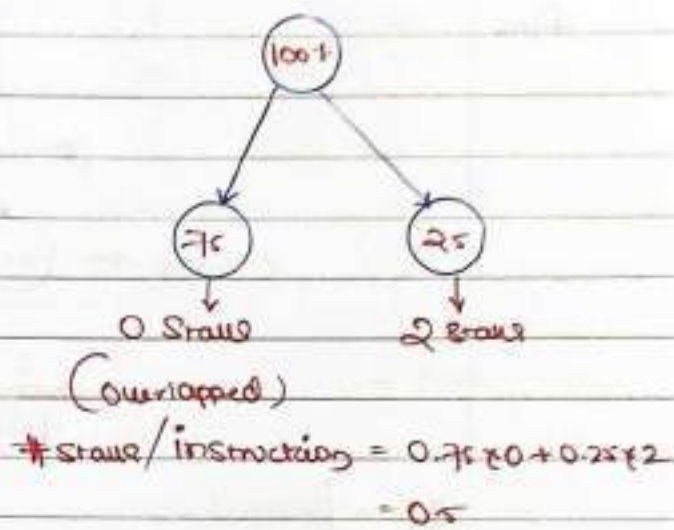
$$= 6 \times t_c$$

$$(1 + \# \text{stage} / \text{Inst}) \times \text{cycle time}$$

$$= 6 \times t_c$$

$$(1 + 0.5) \times t_c$$

$$= \frac{6}{1.5} \Rightarrow \underline{4} \text{ (Ans)}$$



- Q3. Consider a 5-stage instruction pipeline, where all stages are perfectly balanced and have a cycle time of 20ns. Target address of branch instruction is available at end of the execution (last stage). There are 30% instructions.

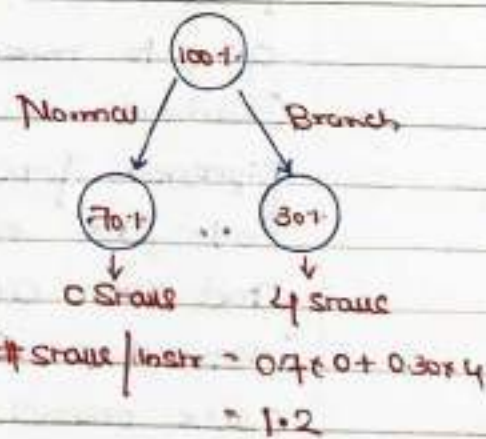
- (i) What is average instruction execution time (ignore the fact some are conditional)?

Ans. $\# \text{stage} = 5$
 Cycle time = 20nsec.

Target address available at end of 5th stage.

Branch Penalty = 5 - 1 = 4.

$$\begin{aligned} \text{Avg Instruction } ET_{\text{pipe}} &= (1 + 1.2) \times 20 \\ &= 2.2 \times 20 \\ &= \underline{44 \text{ nsec}} \text{ (Ans)} \end{aligned}$$



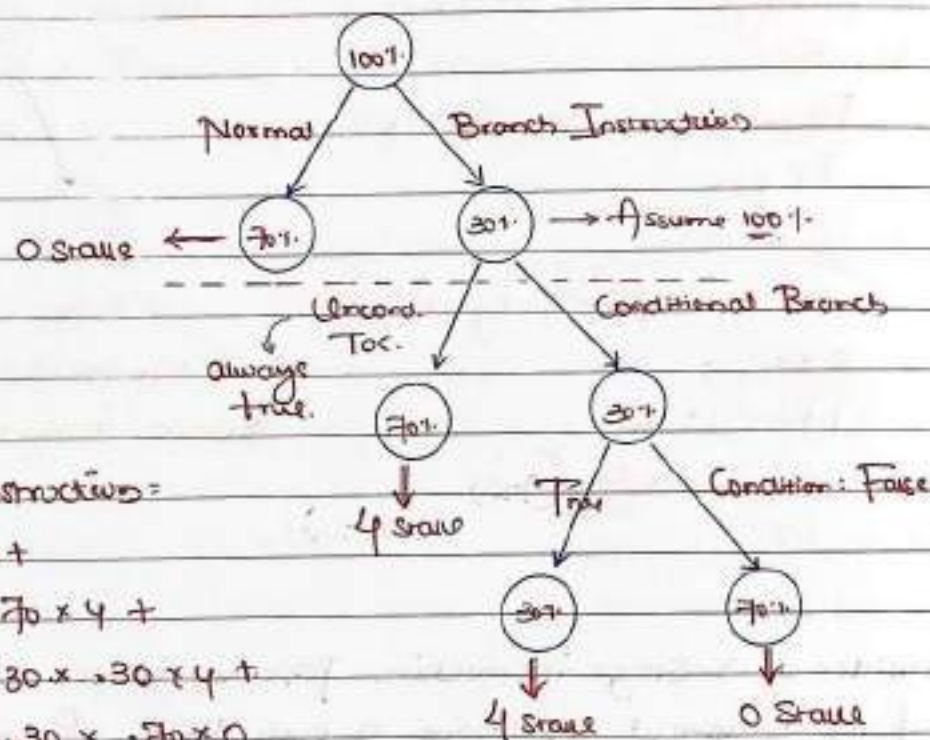
- (ii) What is Speed up factor?

Ans. $S = \frac{ET_{np}}{ET_p} \Rightarrow \frac{5 \times 20 \text{ nsec}}{(1 + \# \text{stalls} / \text{instr}) \times \text{cycle}}$

$$= \frac{5 \times 20}{(1+1/2) \times 20} \rightarrow \frac{5}{2.2} \therefore S = 2.27 \text{ (Ans)}$$

- (iii) Among the branch instructions 30% are Conditional and 70% of them does not satisfy the condition then what is avg instruction ET?

Ans



Stalls/instructions =

$$0.70 \times 0 +$$

$$+ 20 \times 0.70 \times 4 +$$

$$+ 30 \times 30 \times 30 \times 4 +$$

$$+ 30 \times 30 \times 70 \times 0$$

$$\Rightarrow \underline{0.980 \text{ (Ans)}}$$

- Q4. Consider a non pipelined Processor/operating at 2.5 GHz. It takes 5 Clock Cycle to complete an instruction. You are going to make a 5-Stage Pipeline out of this Processor. Overheads associated with Pipelining force you to operate the Pipelined Processor at 2 GHz. In a given program, assume that 30% are memory instructions, 60% are ALU instructions and rest are branch instructions. 5% of the memory instructions cause Stalls of 50 clock cycle each due to Cache miss. and 50% of the branch instructions cause Stalls of 2 cycle each. Assume that there are no stalls associated with the execution of ALU instructions. For this program, the Speedup achieved by the pipelined processor

Over the non-pipelined processor (Rounded off to two decimal place) is _____

$$\text{Cycle time} = \frac{1}{2\text{GHz}} = 0.5\text{nsec}$$

Stalls/instruction =

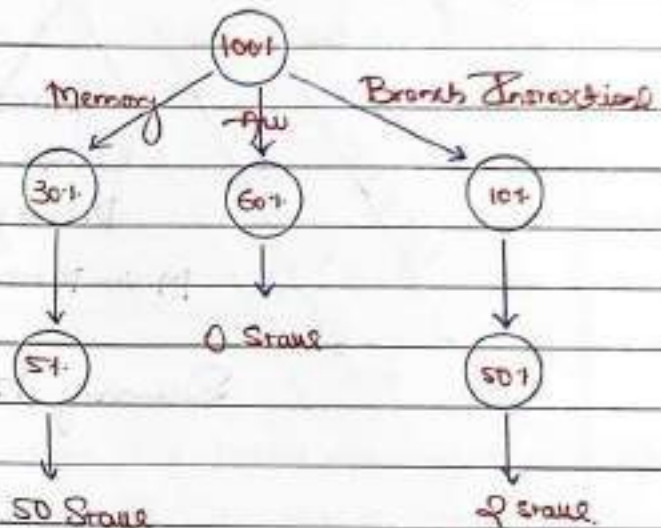
$$+ 30 \times 0.05 \times 50$$

$$+ 60 \times 0$$

$$+ 10 \times 0.50 \times 2$$

$$\Rightarrow 0.85$$

$$\begin{aligned} \text{Avg Inst. ET} &= (1 + 0.85) \times 0.5 \\ &= 1.85 \times 0.5 \\ &= \underline{0.925\text{ nsec}} \end{aligned}$$



Nonpipeline: 5 clock cycle
2.5 GHz

$$\text{Cycle time} = \frac{1}{2.5\text{ GHz}} = 0.4\text{nsec}$$

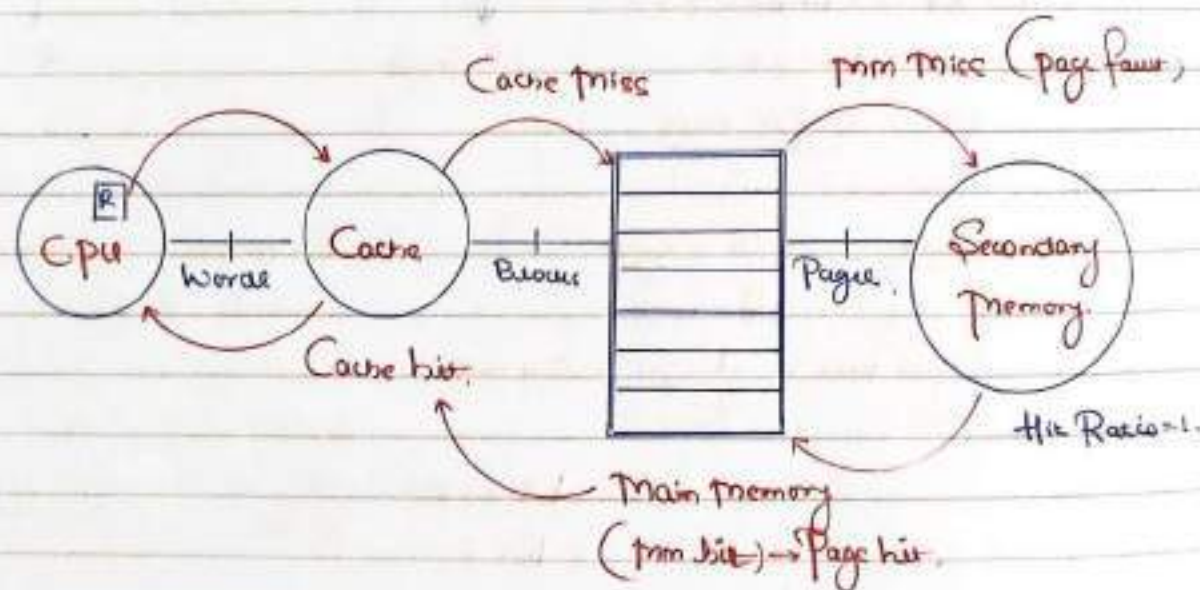
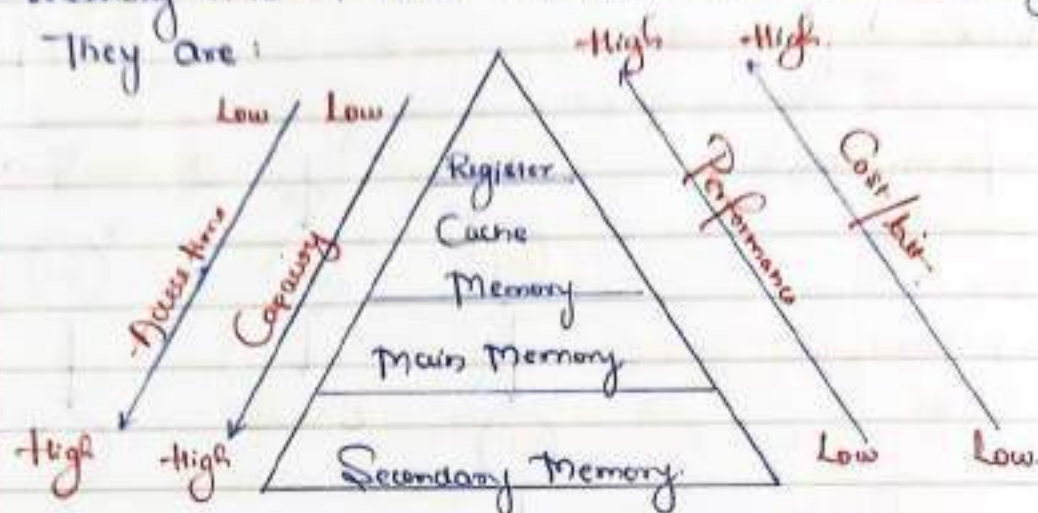
$$\begin{aligned} \text{ET}_{\text{nonpipe}} &= 5 \times 0.4\text{ nsec} \\ &= \underline{2\text{ nsec}} \end{aligned}$$

$$S = \frac{\text{ET}_{\text{np}}}{\text{ET}_{\text{p}}} = \frac{2}{0.925} \therefore \underline{S = 2.16 \text{ (Ans)}}$$

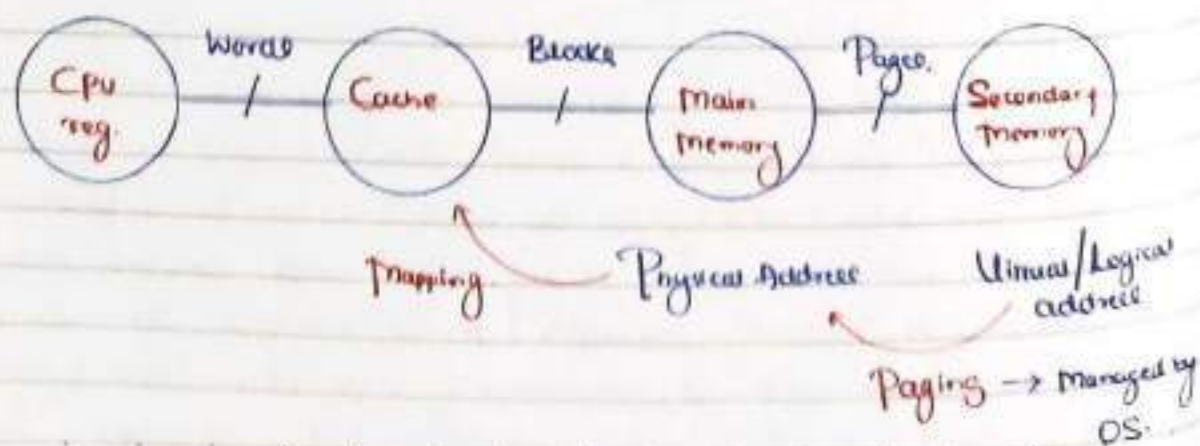
CACHE MEMORY.

Memory hierarchy: Hierarchy design organise the system support memory into 4 levels to minimise the accessing times.

They are:



$$\text{Hit Ratio} = \frac{\text{Number of hit}}{\text{Total number of access}}$$



Memory:

1. CPU generate request initially refer to the cache.
2. If the reference (respective data) found in the cache that is called Cache hit. Operation is called hit, then respective data give from cache to CPU in the form of words.
3. If the reference is not found in the cache, that is known as Cache miss, the reference is forwarded to main memory.
4. If the reference found in the main memory then it is called mm hit or page hit then respective data is given from main memory to cache in the form of blocks and cache to CPU in the form of words.
5. If the reference is not found in the main memory that is called mm miss or page fault then the response is forwarded to Secondary memory. So respective data is transferred from secondary memory to main memory in the form of pages, main memory to cache memory in the form of blocks, then cache memory to CPU in the form of words.

Note: * Secondary memory is the last level of memory in which hit ratio is always "1".

* The process of transfer the data from main memory to cache memory is called "mapping".

Average Memory Access Time [T_{avg}]

$$T_{avg} = h_{hit} \times \text{Time taken by memory when there is a hit} + (1-h_{hit}) \left\{ \begin{array}{l} \text{Time taken by memory when there is a miss} \end{array} \right\}$$

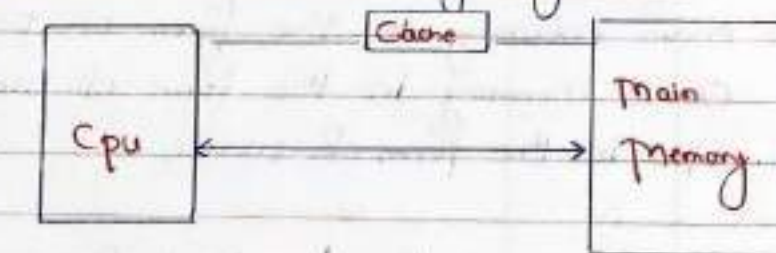
- Q1. Consider a CPU generate 100 Request. Out of 100 requests 90 times hit and 10 times miss. When there is a hit then time taken is 20 nsec and when there is a miss then time taken is 150 nsec. Calculate the T_{avg} ?

Ans. Total CPU Request = 100
 Hit = 90 times
 Miss = 10 times
 Hit will take 20 nsec
 Miss will take 150 nsec.
 Total time = $90 \times 20 + 10 \times 150$
 $= 1800 + 1500$
 $= 3300 \text{ nsec.}$
 $T_{avg} = \frac{3300}{100} = 33 \text{ nsec (Ans)}$

Total CPU Request = 100
 Hit = 90 times, Miss = 10 times
 Hit Ratio = $\frac{90}{100} = 0.9$
 $T_{avg} = H \times \text{Time taken when there is hit} + (1-H) \times \text{Time taken when there is miss}$
 $= 0.9 \times 20 + 0.1 \times 150$
 $= 18 + 15$
 $T_{avg} = 33 \text{ nsec (Ans)}$

Types of Memory Organisation:

① Simultaneous Access Memory Organisation:

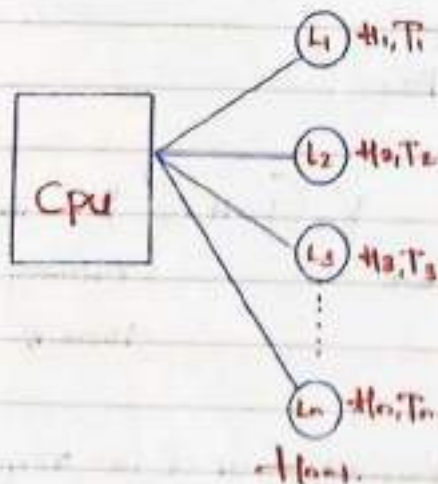


$$T_{avg} = h \cdot t_c + (1-h) \cdot t_m$$

h : hit ratio

t_c : cache access time

t_m : main memory access time



In the Simultaneous access organisation all the level of memory is directly connected to CPU. But follow in sequence (access in a sequence) when there is a miss in level 1 memory then reference forward to level 2 memory.

When there is a hit in level 2 memory then directly data is transferred from level 2 to CPU without copying into level 1 memory.

When there is a miss in level 1 and level 2 memory and hit in level 3 memory then directly data is transferred from level 3 memory to CPU without copying into level 1 and level 2 memory.

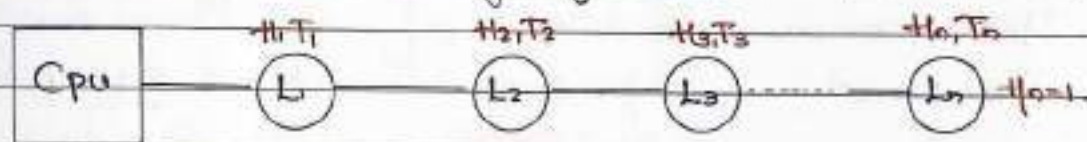
Here $h_1, h_2, h_3, \dots$ are the hit ratios and $T_1, T_2, T_3, \dots, T_n$ are the access of respective level memory.

The time required to access 1 word from memory.

$$T_{avg} = h_1 T_1 + (1-h_1) h_2 T_2 + (1-h_1)(1-h_2) T_3 + \dots + (1-h_{n-1}) h_n T_n$$

* In last level hit ratio = 1.

② Hierarchical Access Memory Organisation



When there is a miss in level 1 and level 2 memory and hit in level 3 memory then firstly data is transferred from level 3 $[L_3]$ memory to level 2 $[L_2]$ memory then level 2 $[L_2]$ memory to level 1 $[L_1]$ memory then from level 1 $[L_1]$ memory to CPU.

$$T_{avg} = h_1 T_1 + (1-h_1) h_2 (T_2 + T_1) + (1-h_1)(1-h_2) h_3 (T_3 + T_2 + T_1) + \dots + (1-h_{n-1}) h_n [T_n + T_{n-1} + \dots + T_2 + T_1]$$

Note: In an question if they have mentioned the keyword 'hierarchical access' or 'level of access' or hierarchical then it means using "Hierarchical Access."

Q2. Calculate the average access time with the cache access time 1ns, and main memory access time 100ns, hit ratio 90%. Using hierarchical access?

Ans. $t_c = 1 \text{ nsec}$ $h = 90\% = 0.9$

$t_m = 100 \text{ nsec}$

$$T_{avg} = h \cdot t_c + (1-h) (t_m + t_c)$$

$$= 0.9 \times 1 + (1-0.9) (100+1)$$

$$= 0.9 + (0.1)(0.1)$$

$$\Rightarrow \underline{11 \text{ nsec}}$$

Q3. In a 2 level memory, level 1 memory is 5 times faster than level 2. And its access time is 10ns < Average access time. But level 1 access time is 20ns, what is the hit ratio? Using Simultaneous access org?

Ans.

$L_1 = T_1$	$S = \frac{P_{L1}}{P_{L2}} = \frac{1/T_1}{1/T_2}$	$T_{avg} = h \cdot t_c + (1-h) \cdot t_m$
$L_2 = T_2$		$30 = h \cdot 20 + (1-h) \cdot 100$
$S = \frac{T_2}{T_1}$		$= 20h + 100 - 100h$
$T_2 = 5 \cdot T_1$	$T_1 = T_{avg} = 10$	$80h = 70$
$T_2 = 5 \times 20$	$T_{avg} = T_1 + 10$	$h = \frac{70}{80} = 0.875$
$T_2 = 100 \text{ nsec}$	$T_1 = 20 \text{ nsec}$	
	$T_{avg} = 20 + 10$	$\Rightarrow \underline{87.5\%} \text{ (Ans)}$
	$= 30 \text{ nsec}$	

Q4. Assume that for a certain processor, a read request takes 50 nsec on a cache miss and 5 nsec on a cache hit. Suppose while running a program, it was observed that 80% of the processor read request result in cache hit. The average read access time in nsec is _____

Ans

Hit Ratio = 80 = 0.8	When there is hit, time = 5
Miss ratio = $(1 - 0.8) = 0.2$	When there is miss, time = 50
$T_{avg} = 0.8 \times 5 + 0.2 \times 50$	
$= 4 + 10$	
$= \underline{14 \text{ nsec}} \text{ (Ans)}$	

Hierarchical Access:

$$T_{avg} = h \cdot t_c + (1-h) \cdot [t_m + t_c]$$

$$\Rightarrow h \cdot t_c + t_m - h \cdot t_m + t_c - h \cdot t_c$$

$$\Rightarrow t_c + t_m - h \cdot t_m$$

$$\boxed{T_{avg} = t_c + (1-h) \cdot t_m}$$

eg. Hierarchical access bit ratio 20:1, $t_c = 20\text{ns}$, $t_m = 100\text{nsec}$

$$\begin{aligned} T_{\text{avg}} &= h \cdot t_c + (1-h) [t_m + t_c] \\ &= 0.8 \times 20 + (1-0.8) (100+20) \\ &= 16 + 0.2 (120) \end{aligned}$$

$$T_{\text{avg}} = \underline{40\text{ nsec}}$$

$$\begin{aligned} T_{\text{avg}} &= t_c + (1-h) \cdot t_m \\ &= 20 + (1-0.8) 100 \\ &= 20 + (0.2) 100 \end{aligned}$$

$$T_{\text{avg}} = \underline{40\text{ nsec}}$$

Q5. Consider a system with 2 levels. Level 1 Access time is 20ns level 2 Access time = 150ns $T_{\text{avg}} = 30$ using simultaneous access.

(i) What is the bit ratio?

Ans $T_1 = 20\text{ns}$, $T_2 = 150\text{ns}$ $T_{\text{avg}} = 30\text{ns}$

$$T_{\text{avg}} = h \cdot t_1 + (1-h) \cdot t_2$$

$$30 = h \cdot 20 + (1-h) \cdot 150$$

$$30 = 20h + 150 - 150h$$

$$120 = 130h$$

$$h = \frac{120}{130} = 0.923 \Rightarrow \underline{92.3:1}$$

(ii) If the bit ratio is made to 100:1 then what is the access time of L1 and L2 memory?

Ans $T_1 = 20\text{ns}$, $T_2 = 150\text{ns}$

L1 and L2 access remain same, $h=1$

If bit ratio = 100:1

$$T_{\text{avg}} = h \cdot t_1 + (1-h) \cdot t_2$$

$$= 1 \times 20 + (1-1) 150$$

$$T_{\text{avg}} = \underline{20\text{ nsec}}$$

} Only T_{avg} changes

Q6. In the above question if T_{avg} is increased by 10% then what is the % change in the bit ratio?

Ans $T_1 = 20$, $T_2 = 150\text{nsec}$ $T_{\text{avg}} = 30\text{nsec} + 10\% = 33\text{nsec}$

$$T_{avg} = h \cdot t_1 + (1-h) \cdot t_2$$

$$33 = h \cdot 20 + (1-h) \cdot 150$$

$$33 = 20h + 150 - 150h$$

$$130h = 117$$

$$h = \frac{117}{130} = hf = 0.9$$

$$= 90\%$$

Previous hit ratio = 92.31.

$\Rightarrow$ Hit ratio decreased by 2.31%.

Note: * 1 Word Access Time = T_{avg} .

$$\# \text{ words/sec} = \frac{1}{T_{avg}}$$

Data Transfer Rate.

* Hierarchical Access Memory org: 3 level

h_1 : hit ratio in level 1

$(1-h_1)$: miss ratio in level 1

(m_1)

$(1-h_2)$: miss ratio in level 2

(m_2)

$$T_{avg} = h_1 t_1 + (1-h_1) h_2 (t_2 + t_1) + (1-h_1) (1-h_2) h_3 (t_3 + t_2 + t_1)$$

$$T_{avg} = h_1 t_1 + m_1 h_2 (t_2 + t_1) + m_1 m_2 h_3 (t_3 + t_2 + t_1)$$

m_1 : Miss ratio in level 1.

Q7. Assume that for a Certain processor, a read request takes 50 ns on a Cache miss and 5 ns on a Cache hit. Suppose while running a program, it was observed that 80% of the processor's read request result in a cache hit. The average read access time in nanoseconds is _____.

(i) What is the data transfer rate (performance) of this memory system (in words/sec)?

Ans.

$$\text{Data Transfer rate} = \frac{1}{T_{avg}} \text{ words/sec}$$

$$= \frac{1}{14 \times 10^{-9}} \text{ words/sec}$$

$$= \frac{1}{14} \times 10^9 \text{ words/sec}$$

$$\frac{1000 \times 10^6}{14}$$

$$= 71.42 \times 10^6$$

$$= 71.42 \text{ million words/sec}$$

$$= \underline{72 \text{ million words/sec (Ans)}}$$

(ii) What is the bandwidth required of this memory system if word size is 8 bit?

Ans Bandwidth: Word size = 8 bit

- 72 million words/sec = 72×10^6 words/sec
- $72 \times 10^6 \times 8$ bit per sec
- 72×8 M bit/sec
- 576 M bit/sec or 72 MByte/sec (Ans)

Locality of Reference

Accessing the higher level of memory data from the level 1 memory (Cache memory) is called locality of reference (Factor Memory).

1. Temporal LOR

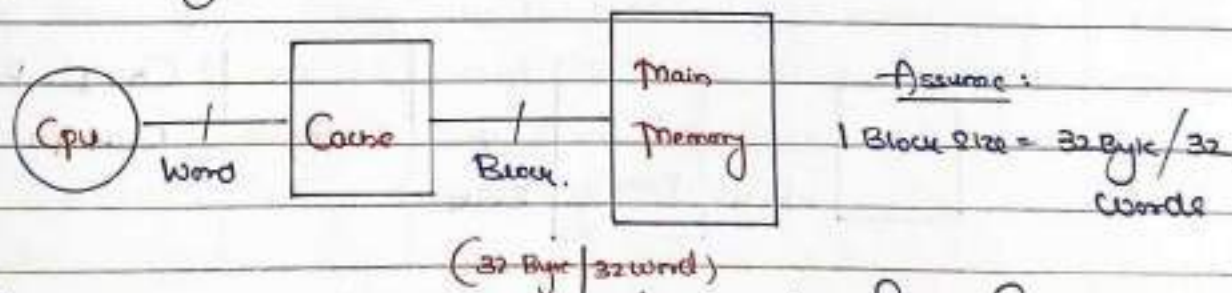
2. Spatial LOR

① Temporal LOR: This means the same word in the same block is referenced by the CPU in near future (frequency).

or

Same data which access again and again then that type of data stored in Temporal LOR.

② Spatial LOR means adjacent word is in the same block is referenced by the CPU in a sequence.



If miss in Cache then 1 complete block is transferred from main memory to Cache, the respective word (which is requested by the CPU) given from Cache memory to CPU.
(Next reference there is a Cache hit)

Q.1 Here 1 block size = 32 word / 32 Byte & CPU require only 1 word?

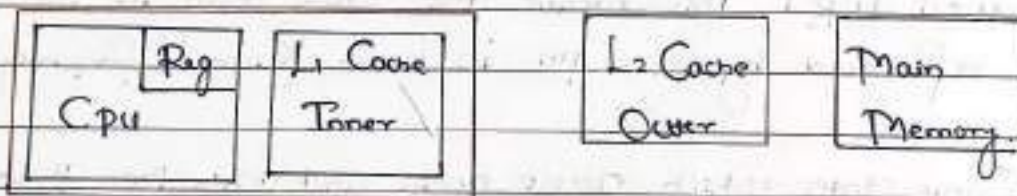
Ans In this process 1 complete block of 32 words / 32 Byte transferred

from main memory to cache memory then requested word is transferred from cache memory to CPU.

Note: So in the next instructions time when CPU request some word or adjacent word then that request available in Cache (Cache hit).

Types of Cache

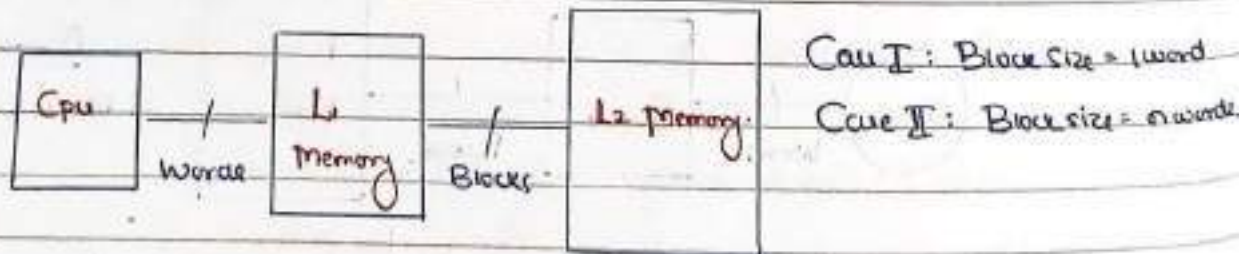
- ① Unified Cache: Instructions and data both are placed in same cache.
- ② Split Cache: This cache logically divided into two parts.
 - (i) Instruction Cache (I-cache)
 - (ii) Data Cache (D-cache)
- ③ Multilevel Cache:



Size $L_1 < L_2$

Speed $L_1 > L_2$

2 Level of Memory (If locality of reference included)



CPU always access the data from the fastest memory (level 1/cache memory). If there is a miss in level 1 (cache) memory then one complete block is transferred from level 2 (slow) memory to level 1 (cache) memory and addressed word (which is requested) that respective word given from cache memory to CPU.

T_B : Block transfer time from L_2 memory to L_1 memory.

→ Case 1: Block size = 1 word

$$T_B = T_2$$

→ Case 2: Block size = n words

$$T_B = n \cdot T_2$$

If 2 level memory:

$$h_2 = 1$$

$$T_{avg} = h_1 t_1 + (1-h_1)(t_2+t_1)$$

or

$$T_{avg} = t_1 + (1-h_1)t_2$$

If locality of reference included

$$(T_B = n \cdot T_2)$$

$$T_{avg} = h_1 t_1 + (1-h_1)[T_B + t_1]$$

or

$$T_{avg} = t_1 + (1-h_1)T_B$$

If 3 level memory:

$$h_3 = 1$$

$$T_{avg} = h_1 t_1 + (1-h_1)h_2(t_2+t_1) + (1-h_1)(1-h_2)(t_3+t_2+t_1)$$

If locality of reference included:

$$T_{avg} = h_1 t_1 + (1-h_1)h_2(T_{B1} + t_1) + (1-h_1)(1-h_2)[T_{B2} + T_{B1} + t_1]$$

Q1. In a 3 level memory, level 1 memory access time is T_1 , level 2 memory access time is T_2 (T_{B1}) and level 3 memory access time is T_3 (T_{B2}). Hit ratio of level 1 is h_1 and hit ratio of level 2 is h_2 . What is the average access time using hierarchical access?

(i) If there is a hit in level 1 ($h_1 = 100\%$)?

Ans	$h_1 = 100 \Rightarrow 1$	$h_1 = 100 \Rightarrow h_1 = 1$
	$(1-h_1) = 0$	$(1-h_1) = 0 \Rightarrow 0 \cdot x = 0$
	$T_{avg} = T_1$	$T_{avg} = T_1$

(ii) If there is a miss in level 1 and hit in level 2 ($h_2 = 100\%$)

Ans	$h_1 = 0$ & $h_2 = 100 \Rightarrow h_2 = 1$	$h_1 = 0, h_2 = 1$ & $(1-h_2) = 0$
	$1-h_2 = 0$	
	$T_{avg} = T_2 + T_1$	$T_{avg} = T_{B1} + T_1$

- (iii) If there is a miss in level 1 and level 2 and hit in level 3
 Ans. $h_1=0, h_2=0, h_3=1$ | $h_1=0, h_2=0, h_3=1$
 $T_{avg} = T_3 + T_2 + T_1$ | $T_{avg} = T_3 + T_2 + T_1$

Q2 In a 2 level memory. Level 1 memory access time is 30ns and level 2 memory access time is 250ns/word. Hit ratio of level 1 is 90%. If there is a miss in level 1 then 4 word block must be transferred (moved) from level 2 into level 1 and then addressed words is given to the Cpu. What is the average access time?

Ans. $h_1 = 90\% = 0.9$ | $t_1 = 30\text{ nsec}$
 $t_2 = 250\text{ nsec/word}$
 $T_{B1} = 4\text{ word} \times 250\text{ nsec/word} = 1000\text{ nsec}$
 $T_{avg} = h_1 t_1 + (1-h_1)(T_{B1} + t_1)$
 $= 0.9 [30] + (1-0.9) [1000 + 30]$
 $= 27 + 103 \Rightarrow 130\text{ nsec (Ans)}$

Q3. A cache memory that has a hit rate of 0.8 has an access latency 10 ns and miss penalty 100 ns. An optimization is given done on the cache to reduce the miss rate. However, the optimization results in an increase of cache access latency to 15 ns, whereas the miss penalty is not affected. The minimum hit rate (rounded off to two decimal places) needed to after the optimization such that it should not increase the average memory access time is _____.

Ans. $h = 0.8$, $t_c = 10\text{ nsec}$, miss penalty (t_m) = 100 nsec
 $T_{avg} = h t_c + (1-h)(t_m + t_c)$
 $= 0.8 \times 10 + (1-0.8)(100 + 10)$
 $= 8 + 0.2(110) \Rightarrow 30\text{ nsec}$

New Optimization: $t_{c\text{ new}} = 15\text{ nsec}$

$$T_{avg(\text{new})} = h \cdot t_c + (1-h)(t_m + t_c)$$

$$30 = h \times 15 + (1-h)(100 + 15)$$

$$30 = 15h + 115 - 115h$$

$$85 = 100H \therefore \frac{85}{100} = 0.85 \text{ (Ans)}$$

Q4. A direct mapped cache memory of 1mb has a block size of 256 Bytes. The Cache has an access time of 3ns and a hit rate of 94%. During a cache miss, it takes 20ns to bring the first word of a block from the main memory, while each subsequent word takes 5ns. The word size is 64 bits. The average memory access time in ns is _____

Ans.

$$\begin{aligned} \text{Block Size} &= 256 \text{ Byte} \\ \text{1 word size} &= 64 \text{ bits} = 8 \text{ Byte} \\ \text{No. of words} &= \frac{256 \text{ Byte}}{8 \text{ Byte}} = 32 \text{ words} \\ \text{1 word takes} &= 20 \text{ nsec} \\ \text{Remaining 31 words take} &= 5 \text{ ns} \end{aligned}$$

$$\begin{aligned} T_{\text{avg}} &= .94 \times 3 + (1 - 0.94) (20 + 31 \times 5) \\ &= 2.82 + (0.06) (31 \times 20 + 155) \\ &= 13.5 \text{ nsec (Ans)} \end{aligned}$$

Basic Elements of Cache Design:

1. Memory Organisation
2. Mapping technique
 - i. Direct Mapping
 - ii. Set associative mapping
 - iii. Fully associative mapping
3. Replacement algorithm
4. Updating technique
5. Multi level cache

* In the cache design, data is transferred from main memory to cache memory in the form of blocks.

* So both memory (main memory and cache memory) must be organised based on block size.

* Main memory and cache memory are divided into pairs (equal size parts) based on block size called Mm Block and Cm Block (number of lines) respectively.

$$\# \text{ Mm Block} = \frac{\text{Mm Size}}{\text{Block Size}}$$

$$\# \text{ Cm Block} = \frac{\text{Cm Size}}{\text{Block Size}}$$

Cache Memory:
Before Organisation:

CM = 8 Byte

000	1 Byte
001	1 B
010	1 B
011	1 B
100	1 B
101	1 B
110	1 B
111	1 B

Cache index = 3 bits

* Consider a 8 Byte Cache with 2 Byte Block Size. The CM. organisation is as follows:

After Organisation

00	0	2 Byte
01	0	2 Byte
10	0	2 Byte
11	0	2 Byte

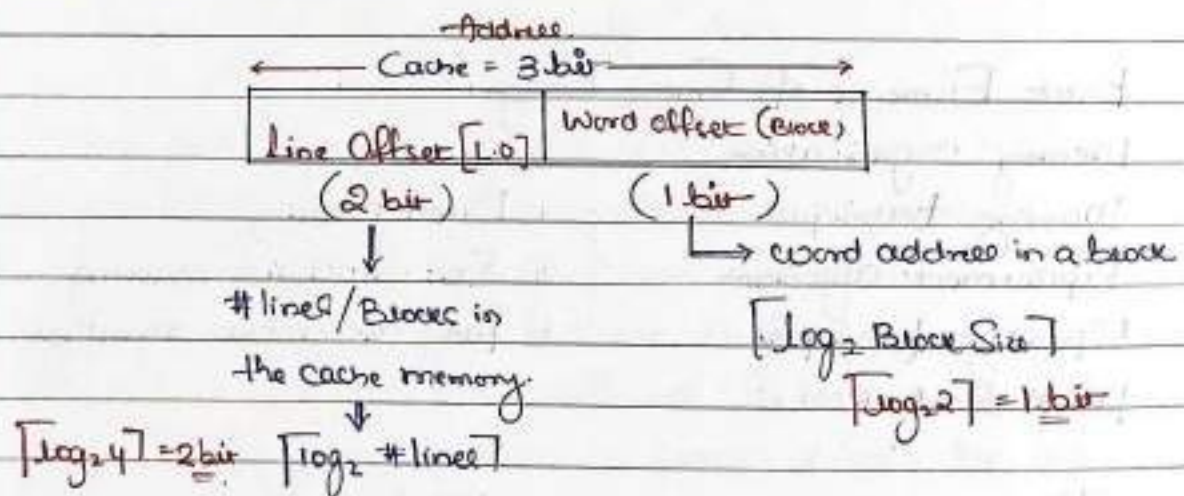
line word offset

$$\# \text{CM Lines} = \frac{\text{CM Size}}{\text{Block Size}}$$

$$= \frac{8B}{2B} = 4$$

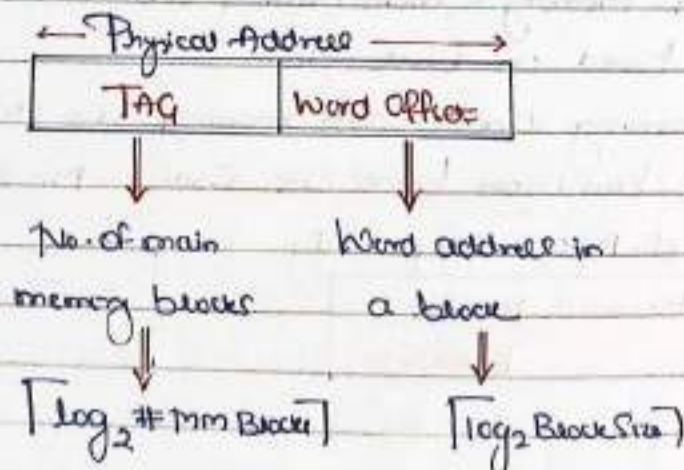
$$\text{CM Size} = 4 \times 2 \text{ Byte} = 8 \text{ Byte (After Organisation)}$$

Before and after the organisation, Cache memory Capacity is same but internal structure is different.



Physical Address is interpreted as

MM Size > CM Size



Q1. Consider a Cache memory which is indexed with 16 bit address, organised into 32 byte block size and physical address is 32 bits.

- # Cm Block
- # mm Block
- Cm Address format
- Mm Address format.

Ans. Cache Index = 16 bit $\rightarrow$ Cm Size = 2^{16} Byte = 64 KByte

Block Size = 32 Byte

Block/word offset = $\lceil \log_2 32 \rceil$
= 5 bits

Physical address = 32 bit
Mm Size = 2^{32} Byte = $2^2 \cdot 2^{30}$ Byte
= 4 GByte

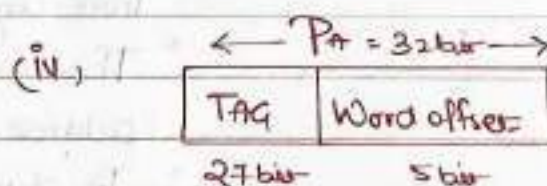
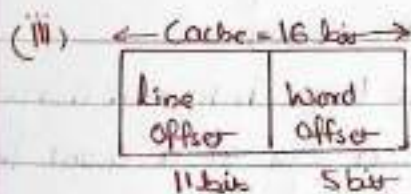
$$(i) \# \text{Cm Block} = \frac{\text{Cm Size}}{\text{Block Size}} = \frac{64 \text{ KB}}{32 \text{ B}}$$

$$= \frac{2^{16} \text{ B}}{2^5 \text{ B}} = 2^{11} = 2 \text{ K} \quad \text{L.O} = 11 \text{ bit}$$

$$(ii) \# \text{mm Block} = \frac{\text{Mm Size}}{\text{Block Size}} = \frac{4 \text{ GB}}{32 \text{ B}} = \frac{2^{32} \text{ B}}{2^5 \text{ B}}$$

$$= 2^{27}$$

= 128 M Blocks



Q2. Consider a System Mm = 256 MB, Cache = 128 KB, and Block size = 512 Byte then

1. #line

3. #mm Block

2. L.O and W.O format

4. TAG and W.O format

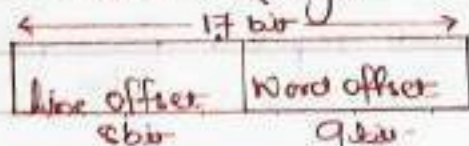
Ans. Mm Size = 256 MB = 2^{28} Byte

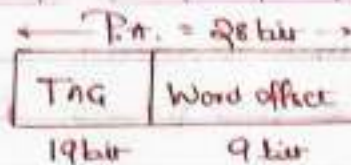
Cache Size = 128 KB = 2^{17} Byte

Block Size = 512 B = 2^9 Byte

W.O = 9 bit

Lines = $2^8 = 256$ lines





$$\# \text{ min. Block} = 2^9 = 512 \text{ K Bytes}$$

Mapping: The process of transferring the data from main memory to cache memory is called mapping. There are 3 types of mapping:

1. Direct mapping
2. Set Associative mapping
3. Fully associative mapping.

Mapping Function: * Because there are fewer cache lines than main memory blocks, an algorithm is needed for mapping main memory blocks into cache lines.

* Three techniques can be used:

① Direct: * The simplest technique

* Maps each block of main memory into only one possible cache line.

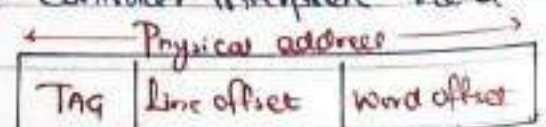
② Associative: * Permits each main memory block to be loaded into any line of the cache.

* The cache control logic interprets a memory address simply as a tag & a word field.

* To determine whether a block is in the cache, the cache control logic must simultaneously examine every line's tag for a match.

③ Set Associative: * A compromise that exhibits the strengths of both the direct and associative approaches while reducing their disadvantages.

Direct Mapping: In this direct cache controller interprets the CPU generated request:



$$\# \text{Line} = \frac{\text{Cm Size}}{\text{Block Size}}$$

$$\text{Word offset} = \log_2 (\text{Block Size})$$

$$\text{Line offset} = \log_2 (\text{Line})$$

$$\rightarrow \text{TAG} = \text{Physical Address} - (\text{Line offset} + \text{Word offset})$$

$$\rightarrow \text{TAG memory Size} = \# \text{Lines} \times \text{Tag bits} \quad (\text{depend on the mapping technique})$$

Q3. Consider a direct mapping if the size of the cache memory is 512 KB and main memory 512 MB and cache line size (Block) is 64 KB. Calculate the bit required for

(i) PA

(iii) L.O

(v) # lines

(ii) TAG

(iv) W.O

(vi) Tag memory size

Ans. Cache size = 512 KB $\Rightarrow 2^{19}$ Byte

MM size = 512 MB $\Rightarrow 2^{29}$ Byte

Block size = 64 KB = 2^{16} Byte

PA = 29 bit

W.O = 16 bit

$\leftarrow \text{PA} = 29 \text{ bit} \rightarrow$

TAG	L.O	W.O
-----	-----	-----

10 bit 3 bit 16 bit

$$\# \text{Line} = \frac{\text{Cm Size}}{\text{Block Size}} = \frac{512 \text{ KB}}{64 \text{ KB}} = \frac{2^{19} \text{ Byte}}{2^{16} \text{ Byte}} = 8 \text{ Line}$$

$$\begin{aligned} \text{Tag memory size} &= \# \text{Line} \times \text{Tag bit} \\ &= 8 \times 10 \Rightarrow \underline{80 \text{ bits}} \end{aligned}$$

Line offset = 3 bit

Q4. Consider a direct mapping, Cache size = 64 Byte, Line size = 8 Byte, MM = 256 Byte then # bits for PA, TAG, L.O, W.O, Tag memory size?

Ans. $\leftarrow \text{PA} = 8 \text{ bit} \rightarrow$

TAG	L.O	W.O
-----	-----	-----

2 bit 2 bit 3 bit

$$\# \text{Line} = \frac{\text{Cm Size}}{\text{Block Size}} = \frac{64 \text{ B}}{8 \text{ B}} = 8 \text{ Line}$$

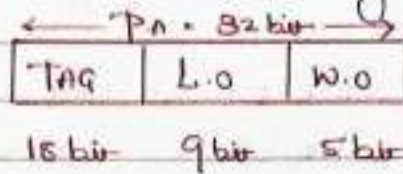
L.O = 3 bit

$$\begin{aligned} \text{Tag memory size} &= \# \text{Line} \times \text{Tag bit} \\ &= 8 \times 2 \Rightarrow \underline{16 \text{ bits}} \end{aligned}$$

Q5. Consider a machine with a byte addressable main memory of 2^{32} bytes, divided into blocks of size 32 bytes. Assume that a direct

mapped Cache having 512 Cache lines is used with this machine. The size of the tag field in bits is 18.

Ans



$$\text{TAG} = 32 - (9 + 5) \\ \Rightarrow \underline{18 \text{ bits}} \text{ (Ans)}$$

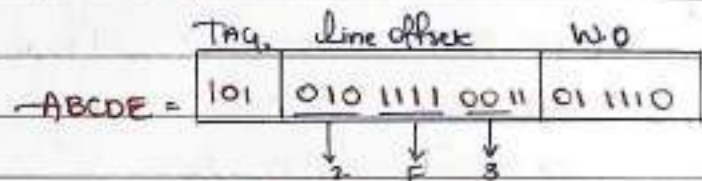
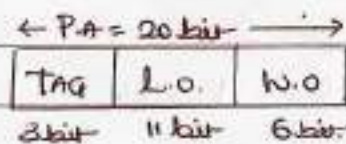
Q6. Consider a direct mapping, Cache Size = 128KB, Line Size = 64B. Main memory is 1MB then what is the line number of physical address (ABCDE)₁₆?

Ans

$$\text{Mem} = 1\text{MB} = 2^{20}\text{B} \rightarrow \text{PA} = 20 \text{ bit}$$

$$\text{Line size} = 64\text{B} = 2^6\text{B} \rightarrow \text{W.O} = 6 \text{ bit}$$

$$\# \text{ Lines} = \frac{\text{Cache Size}}{\text{Block Size}} = \frac{128\text{KB}}{64\text{B}} = \frac{2^{17}\text{B}}{2^6\text{B}} = 2^{11} \text{ lines}$$

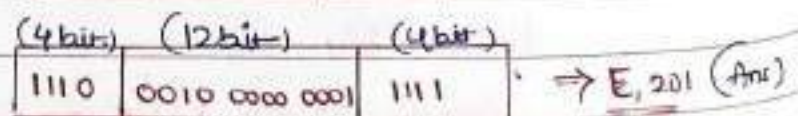
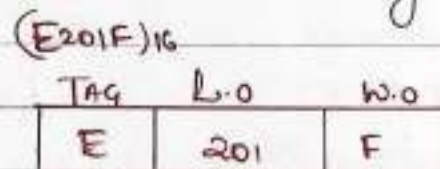
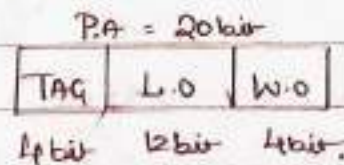


$$\text{TAG} = 20 - (11 + 6) \\ = \underline{3 \text{ bit}}$$

$$= \underline{(2F3)}_{16} \text{ (Ans)}$$

Q7. Consider a machine with a byte addressable main memory of 2^{22} bytes, block size of 16 bytes and a direct mapped Cache having 2^{12} Cache lines. Let the addresses of two consecutive bytes in main memory be (E201F)₁₆ and (E2020)₁₆. What are the tag and Cache line address (in hex) for main memory address (E201F)₁₆?

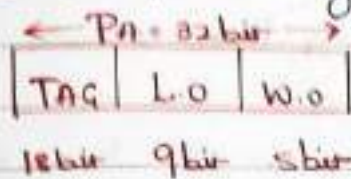
Ans



Q8. Consider a machine with a byte addressable main memory of 2^{32} bytes divided by into blocks of size 32 bytes. Assume that a direct mapped Cache having 512 Cache lines is used with this machine.

The size of the tag field in bits is _____

Ans



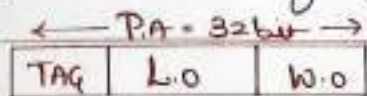
$$TAG = 32 - (9 + 5)$$

$$\Rightarrow \underline{18 \text{ bit}} \text{ (Ans)}$$

Q9. Consider a Computer System with a byte-addressable primary memory of size 2^{32} bytes. Assume the computer system has a direct mapped cache of size 32 KB ($1 \text{ KB} = 2^{10}$ bytes) and each cache block of size 64 bytes.

The size of tag field is 17 bits.

Ans



17 bit 9 bit 6 bit

$$TAG = 32 - (9 + 6)$$

$$= \underline{17 \text{ bit}} \text{ (Ans)}$$

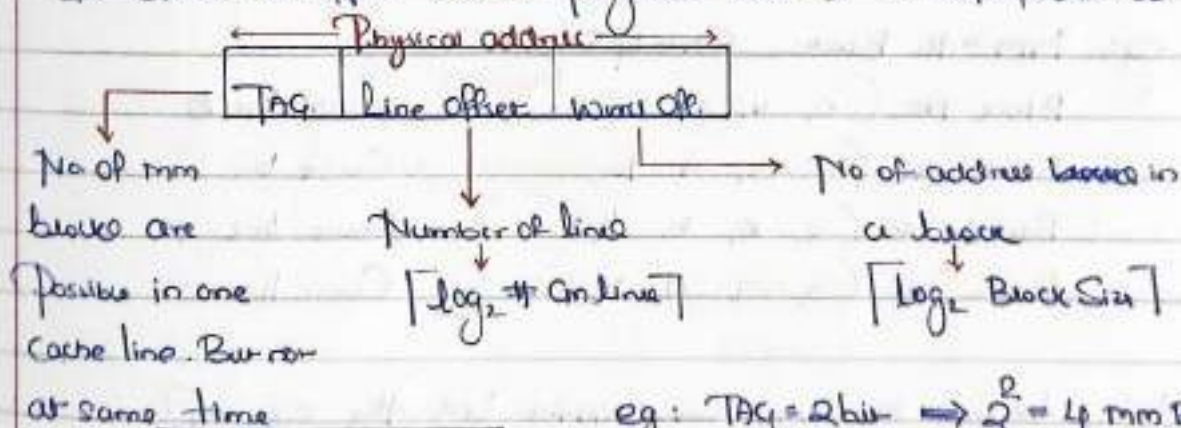
$$\# \text{ Lines} = \frac{\text{Cm Size}}{\text{Block Size}} = \frac{32 \text{ KB}}{64 \text{ B}} = \frac{2^{15} \text{ B}}{2^6 \text{ B}}$$

$$= 2^9 \text{ Lines}$$

$$L.O = 9 \text{ bit}$$

Direct Mapping (Continued)

In direct mapped cache physical address is interpreted as



$$TAG = \frac{\text{mm Size}}{\text{Cm Size}}$$

eg: TAG = 2 bit $\Rightarrow 2^2 = 4$ mm Block are mapped to one Cache line

In direct mapping an mapping function is used to transfer (copying) the data from main memory to Cache memory.

Mapping function: $K \text{ mod } N = i$

K: mm Block no./mm request

i: Cache line number (0 to N-1)

N: Number of: cm Lines

$[K \bmod N + 1]$ means 'K' main memory block are transferred into $K \bmod N$ cache memory line number.

- Assume we have 4 cache lines in Cm and 11 main memory blocks no. 11 (B_{11}).
- (i) Then mm block no 11 (B_{11}) are mapped into $11 \bmod 4 \rightarrow$ Cm line no '3'.
- (ii) If mm block = 7 (B_7) then they are placed into $7 \bmod 4 \rightarrow$ Cm line no '3'.
- i.e here block no B_{11} and Block no 7 are mapped into cache line number 3. But any one block present in cache line no 3.

eg: mm = 50 Blocks, Cm = 10 lines

$$TAG = \frac{mm}{Cm} = \frac{50}{10} \rightarrow 5:1$$

$\rightarrow$ 5 mm Blocks are fighting/mapped to one cache line. But out of 5 only any one mm block present in cache at a time.

eg: mm = 16 Blocks, Cm = 4 lines

Block No (0, 4, 8, 12) $\xrightarrow{\text{Mapped to}}$ Cache line '0'

Block No (1, 5, 9, 13) $\rightarrow$ Cache line '1'

Block No (2, 6, 10, 14) $\rightarrow$ Cache line '2'

Block No (3, 7, 11, 15) $\rightarrow$ Cache line '3'

Q10 Why before mapping we organise both the memory?

eg: mm Size = 16 B, Cache Memory = 8 Byte, Block Size = 2 Byte.

Ans Before Mapping:

← Cache = 3 bit →	
Lo	Wo
2 bit	1 bit

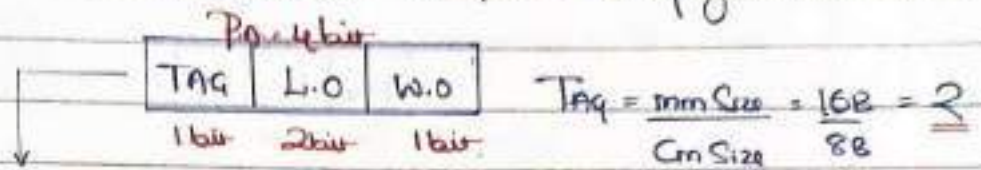
Cache Mem:

← PA = 4 bit →	
TAG	Wo
3 bit	1 bit

Main Memory:

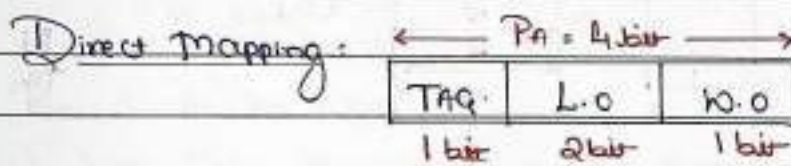
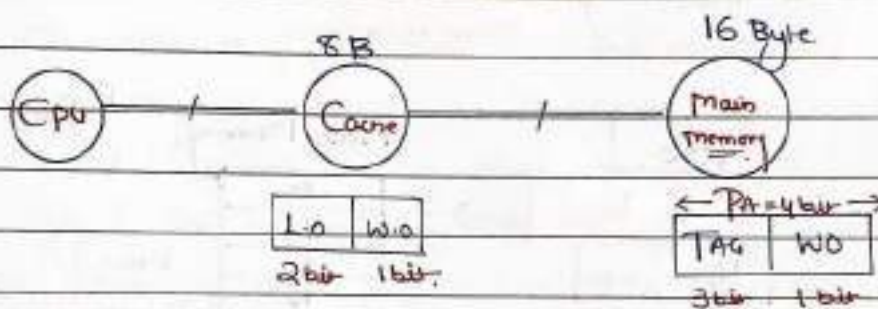
Here TAG bit tells the number of main memory blocks.

After Mapping: Cache Controller interpreted the physical address as:



→ Here TAG bit is 1 → 2' → 2 mem blocks are mapped for one cache line but only one mem block can be present in any one cache line {2:1}

→ This TAG bit is used to check (Identify) where there is Cache hit or Cache miss.



Mapping function:
 $K \bmod 4 = i$

L.O = 2 bit

lines = $2^2 = 4$

Consider the following program:

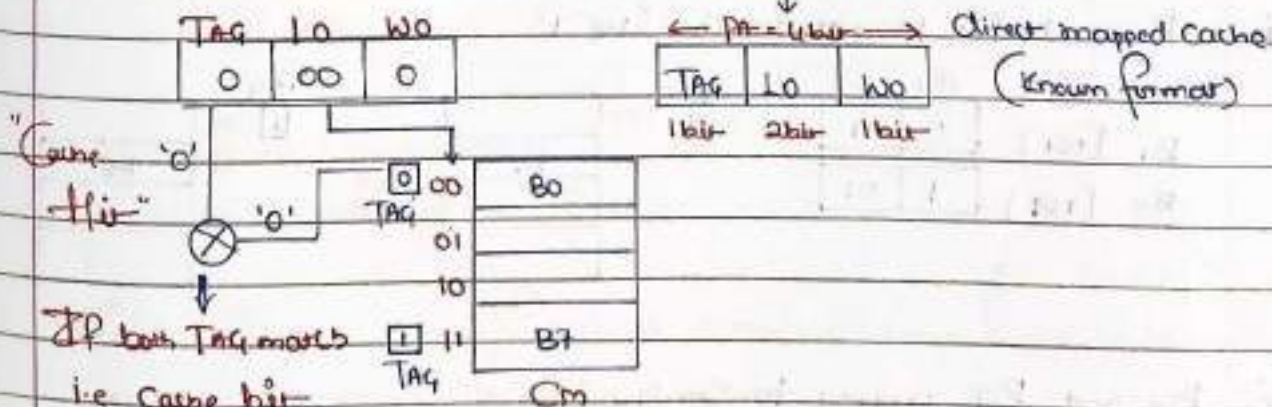
I1: mov r0 [0000]

I2: mov r1 [1000]

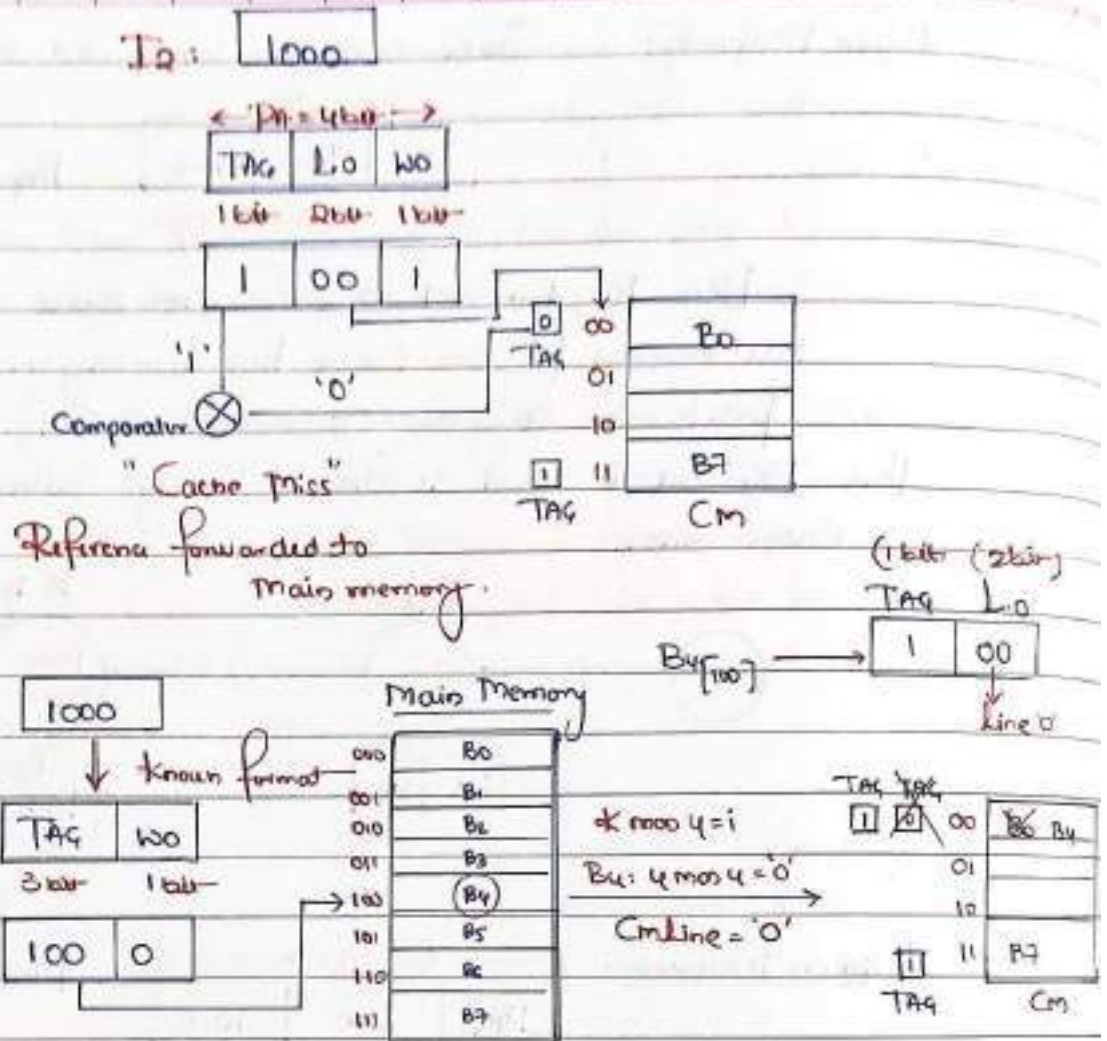
I3: add r0 r1

Cpu Generated Request Refer to Cache

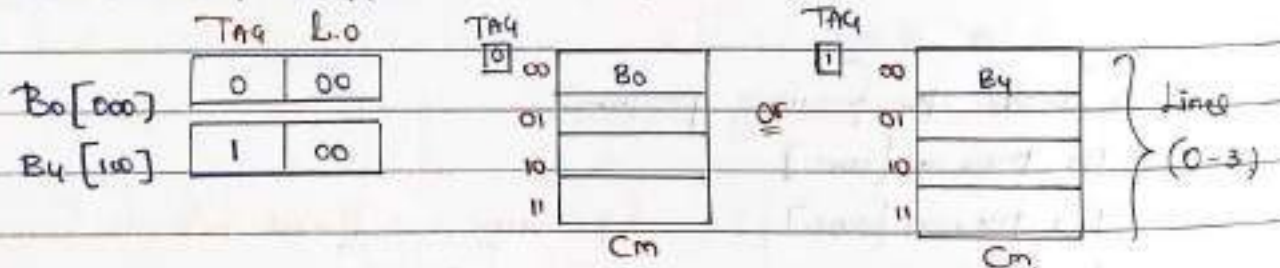
I1: 0000



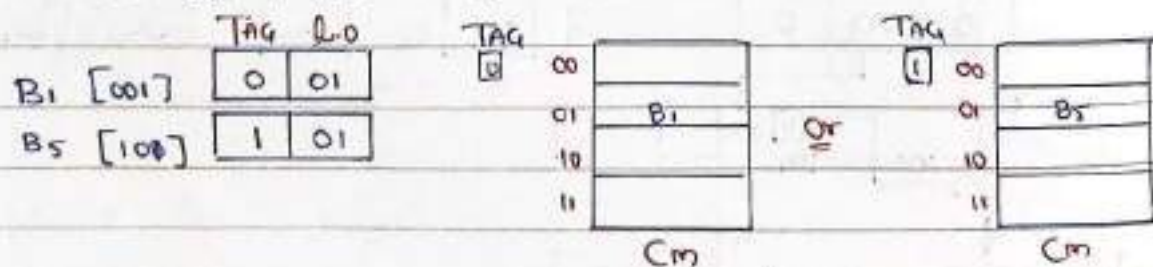
then data is given from Cache to Cpu directly.



(i) B0 and B4 mapped to Cm line '0'



(ii) B1 and B5 mapped to Cm line '1'



(iii) B3 and B7 mapped to Cm line '3'

(iv) B2 and B6 mapped to Cm line '2'

$$\boxed{\text{Tag Memory Size} = \# \text{ Lines} \times \text{Tag Bits} \left\{ \begin{array}{l} \text{Depends upon the} \\ \text{mapping technique} \end{array} \right\}}$$

Complete Working:

1. Cpu Generated request firstly refer to the cache memory.
2. (Directly Mapped Cache) Cache Controller interpreted the Physical Address as:

Physical Address		
TAG	Line Offset	Word Offset
3. Line offset field is directly connected to the address logic of cache memory so respective cache line will be enabled.
4. The existing tag is enabled in the cache line is compared with the CPU Generated tag with the help of Tag comparison.
5. If both Tag (CPU Generated tag and **existing tag on enabled cache line Tag**) are matching then it is called Cache hit (i.e. data is present in the cache). So based on word offset (**0th Byte of block or 1st byte of block**) respective data, respective word is transferred from cache to CPU.
6. If both Tag are not matching then that operation is called Cache miss, then reference is forwarded to main memory.
7. According to mm format, respective mm block is enabled and transfer from MM to CM with the help of direct mapping function [K mod N] into particular cache line the cache to CPU.
8. Here Cache Controller maintains some extra bits for each cache line and that stored in Tag memory / memory.

$$\boxed{\text{Tag Memory} = \text{No. of CM line} \times \text{Tag bits}}$$

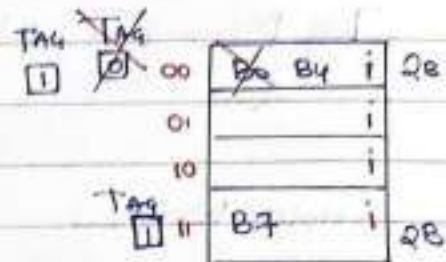
- Number of Comparator = 1.
- Size of comparator = Tag bits.

Note: * In a mapping process complete 1mm block is transferred from main memory to cache memory.

* But respective word/byte of that particular block which is demanded by the CPU is given by from cache to CPU.

Consider the following Program:

I_1 : Mov [0000] 0th Byte of B0
 I_2 : Mov [0001] 1st Byte of B0
 I_3 : Mov [1000] 0th Byte of B4
 I_4 : Mov [1001] 1st Byte of B4.



I_1 : [0000]: Block B0 (0th Byte) $\Rightarrow$ present in cache: Cache hit

I_2 : [0001]: Block B0 (Cache Byte) $\Rightarrow$ present in cache: Due to locality of reference

I_3 : [1000]: B4 $\Rightarrow$ Cache miss so bring B4 from main memory to Cache memory

$\hookrightarrow$ then 0th Byte of given to CPU (from Cache [B4]) then cache to CPU 0th Byte of B4)

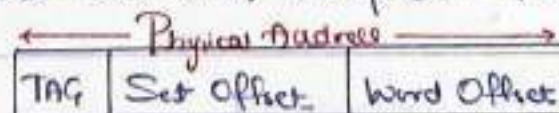
I_4 : [1001]: B4 (1st byte of B4): Cache hit: Due to locality of reference.

Disadvantages of Direct Mapping:

- * In direct mapping each cache line is able to hold only 1 main memory block at a time so number of conflict miss increase.
- * If CPU is referring the multiple blocks which are mapped into the same cache line number (eg. B0, B4, B0, B4 $\Rightarrow$ Conflict No. 0) then the number of Conflict miss will be very high. Even other cache line are empty. {Thrashing}.

Set Associative Mapping:

Set Associative Cache Controller, interprets the CPU generated request as follows:



$\rightarrow$ Word Offset = $\log_2$ Block Size.

$$\# \text{ Lines} = \frac{\text{Cm Size}}{\text{Block Size}}$$

$\rightarrow$ No. of Sets = $\frac{\# \text{ Lines}}{N\text{-way}}$

$\rightarrow$ Set offset = $\log_2$ Sets

$\rightarrow$ TAG = Physical address - (S.O + W.O)

Q1. Consider a Direct Mapped Cache if the Size of the Cache memory is 512KB and main memory 512MB and Cache line is 64KB then Calculate the number of bit required for:

- (i) PA (ii) TAG (iii) L.O (iv) W.O.
(v) #Line

Ans.

$$PA = 29 \text{ bit}$$

TAG	L.O	W.O
-----	-----	-----

$$10 \text{ bit} \quad 3 \text{ bit} \quad 16 \text{ bit}$$

$$PA = 512 \text{ MB} = 2^{29} \text{ B} \rightarrow PA = 29 \text{ bit}$$

$$\text{Line Size} = 64 \text{ KB} = 2^{16} \text{ B} \rightarrow W.O = 16 \text{ bit}$$

$$\# \text{ Line} = \frac{\text{Cm Size}}{\text{Block Size}} = \frac{512 \text{ KB}}{64 \text{ KB}} = \frac{2^{19} \text{ B}}{2^{16} \text{ B}}$$

$$\text{Tag memory Size} = \# \text{ Line} \times \text{Tag bit} \quad \# \text{ Line} = 8$$

$$= 8 \times 10$$

$$= \underline{80 \text{ bit}}$$

$$L.O = 3 \text{ bit}$$

$$\text{TAG} = PA - (L.O + W.O)$$

$$= 29 - (3 + 16)$$

$$= \underline{10 \text{ bit}}$$

Above Question with 2-way Set associative mapping:

PA = 29 bit		
TAG	Set offset	Word offset

$$11 \text{ bit} \quad 2 \text{ bit} \quad 16 \text{ bit}$$

$$\# \text{ Line} = \frac{\text{Cm Size}}{\text{Block Size}} = \frac{512 \text{ KB}}{64 \text{ KB}} = \frac{2^{19}}{2^6}$$

$$= 2^3 = \underline{8 \text{ lines}}$$

$$\text{Tag memory Size} = \# \text{ Line} \times \text{Tag bit}$$

$$= 8 \times 11$$

$$= \underline{88 \text{ bits}}$$

$$\# \text{ SET} = \frac{\# \text{ Line}}{N\text{-way}} = \frac{8}{2} = 4$$

Or

$$\text{Set offset} = 2 \text{ bit}$$

$$\text{Tag memory Size} = \# \text{ Set} \times \text{Block per set} \times \text{Tag bit}$$

$$= 4 \times 2 \times 11 \text{ bit}$$

$$= \underline{88 \text{ bits}}$$

$$\text{TAG} = PA - (S.O + W.O)$$

$$= 29 - (2 + 16)$$

$$= \underline{11 \text{ bit}}$$

Q2. Consider a 2-way Set associative Cache Size = 256KB, Line Size = 32B, Mm = 1MB, then what is the Set number of physical address (ABCDE)₁₆?

Ans.

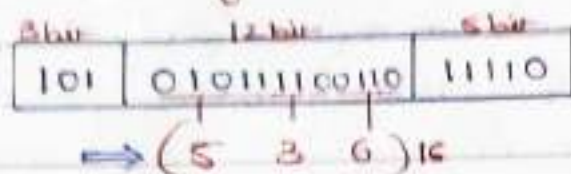
20 bit		
TAG	SO	WO

$$8 \text{ bit} \quad 12 \text{ bit} \quad 5 \text{ bit}$$

$$\# \text{ Line} = \frac{256 \text{ KB}}{32 \text{ B}} = \frac{2^{18} \text{ B}}{2^5 \text{ B}} = 2^{13} \text{ line}$$

$$\# \text{ Set} = \# \text{ Lines} = \frac{2^{12}}{2^1} = 2^{11} \quad \text{So} = 12 \text{ bit}$$

N-way



Direct Mapping
Example.



$\leftarrow P_n = 4 \text{ bit} \rightarrow$

TAG	Lo	Wo				
1 bit	2 bit	1 bit				

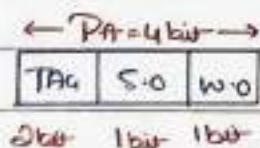
TAG	Lo	Wo				
[0]	00	B0	i	26	000	B0
					001	B1
	01		i	28	010	B2
					011	B3
TAG	10		i	22		
[1]	11	B7	i	23	100	B4
					101	B5
					110	B6
					111	B7

Cache

Mem

Q3: Mem = 16 B, Cm = 8 B, Block Size = 2 B, 2 way set associative mapping then calculate bits for Pn, TAG, S.O, W.O and Tag memory size.

Ans



$$\# \text{ Line} = \frac{\text{Cm Size}}{\text{Block Size}} = \frac{8 \text{ B}}{2 \text{ B}} = 4 \text{ Lines}$$

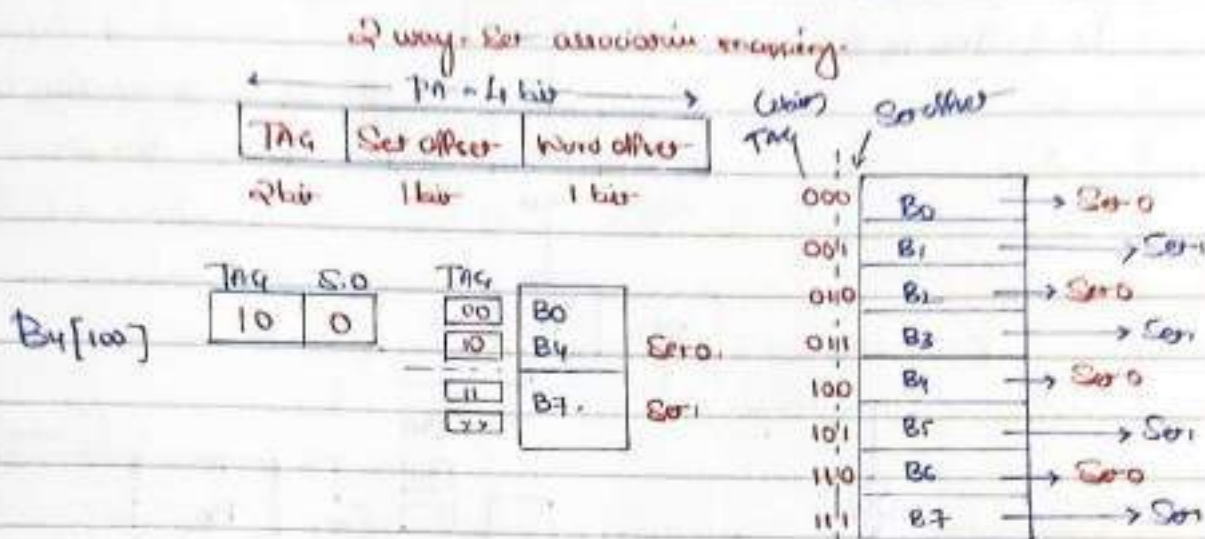
$$\# \text{ Set} = \frac{\# \text{ Line}}{\text{N-way}} = \frac{4}{2} = 2 \text{ set}$$

$$\text{Tag memory} = 4 \times 2$$

$$\text{Size} = 8 \text{ bit. (Ans)}$$

$$\text{Set offset} = 1 \text{ bit}$$

* Here Tag = 2 bit i.e. $2^2 \Rightarrow 4$ mem blocks are fighting (mapped) to any one Cache Set.



Advantage of Association Mapping

I₁: Mov r0 [0000] 0th Byte of Block B0 I₁: Hit (Cache hit)
 I₂: Mov r1 [1000] 0th " " B4 I₂: miss (Cache miss)
 I₃: Mov r2 [0001] 1st " " B0 I₃: Hit
 I₄: Mov r3 [1001] 1st " " B4 I₄: Hit
 I₅: Mov r4 [0000] 0th " " B0 I₅: Hit
 I₆: Mov r5 [1000] 0th " " B4 I₆: Hit

6 Access: Total: 1 miss
5 Hit

Note:

- 2 way Set association: In one cache memory Set we can store 2 mem block in 2 ways [2 mem Block].
- N-way Set association: In one Cm Set, we can store N mem Block by N ways.

Set association Mapping Function

Cache Set = Mem Request mod # sets in cache address

$$K \bmod S = i$$

K: Mem Block number

i: Cache Set number

S: # cache Set

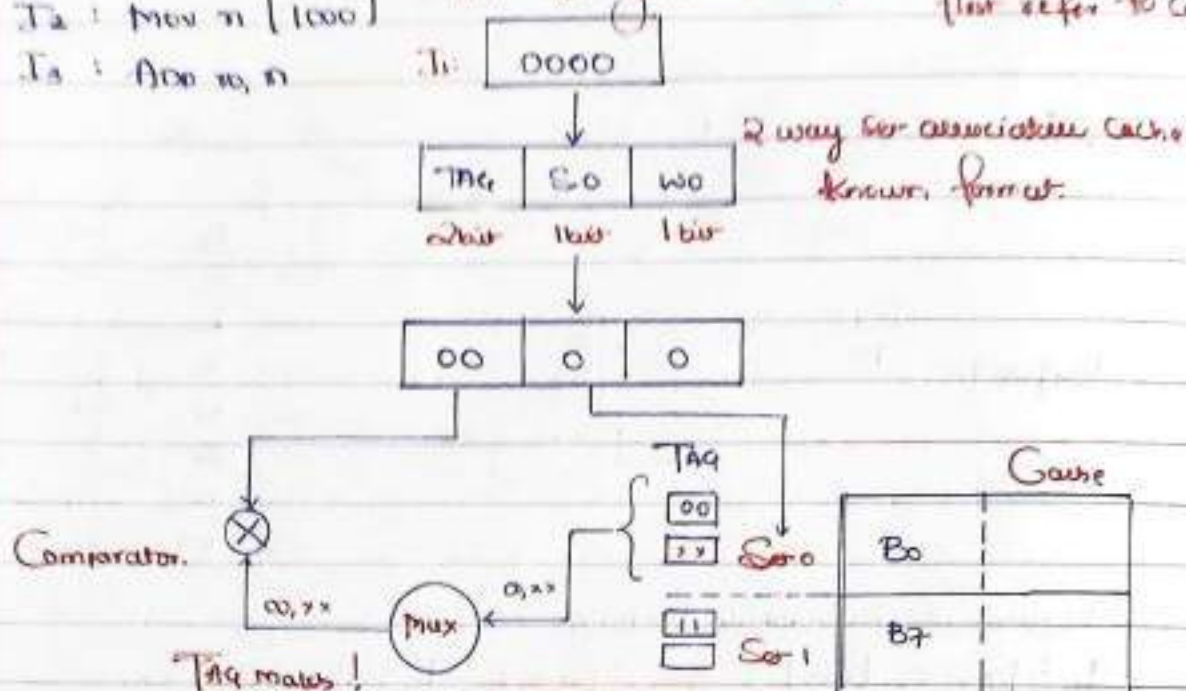
Consider the following program:

T_1 : mov r0 [1000]

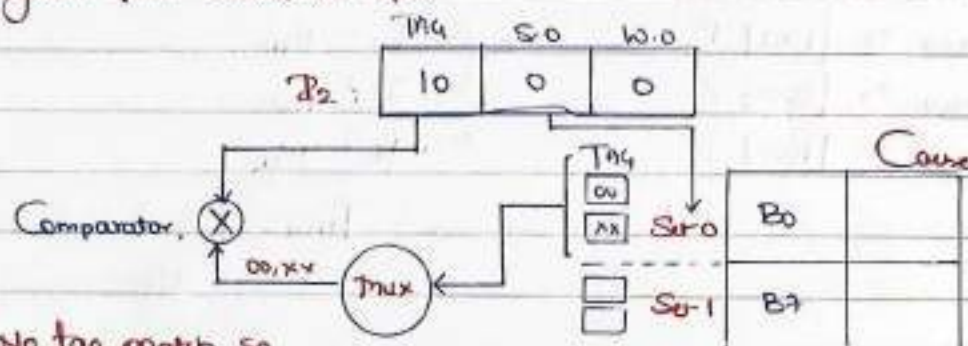
T_2 : mov r1 [1000]

T_3 : Add r0, r1

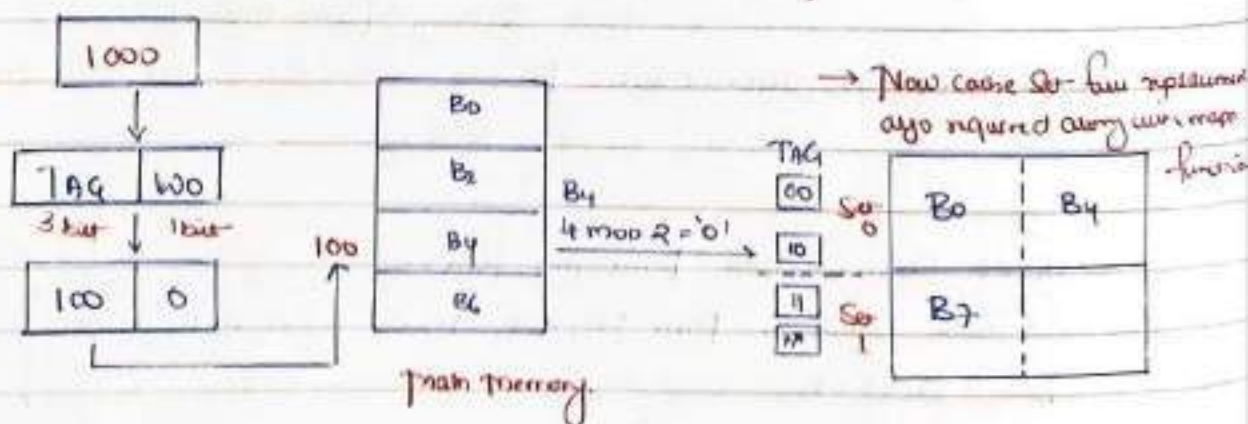
T_1 : cpu generated request first refer to cache.



Respective data cache hit given from cache to cpu.



cache miss, so reference forwarded to main memory.



→ In direct mapping:

$$\text{Hit latency} = \text{Latency of tag comparator.}$$

→ In set associative mapping:

$$\text{Hit latency} = \text{Hit latency of tag comparator} + \text{latency of multiplexer.}$$

Direct Mapping:

Number of Comparator = 1

Size of Comparator = #bits in Tag (Tag bits)

Set associative mapping:

Number of Comparator = N-way

Size of Comparator = #bits in Tag (Tag bits)

Important Points about Set-Associative:

- * #line = L & N way Set associative $\# \text{sets } [S] = \frac{L}{N}$
- * If $N=1$: Direct Mapping
- * If $N=L$: Associative Mapping; ($S=1$) i.e. only 1 set
- * Set associative mapping favours direct mapping as well as Set associative mapping Using mapping functions as:

$$K \bmod S = i$$

Within the Set MM Block Can be placed anywhere.

Note: In the Set associative mapping replacement algorithm is required along with the mapping function when cache set is full.

$$\text{Tag bits in N way set} = \text{Tag bits in direct mapping} + \log_2 N$$

eg: Consider a direct mapping. If the size of cache memory is 512KB and main memory 512MB and cache line size is 2KB then Calculate the number of bits required for Tag?

Ans.

$$Pa = 29 \text{ bit}$$

Tag	L.O	W.O
10 bit	8 bit	11 bit

$$\# \text{ Lines} = \frac{\text{Cm Size}}{\text{Line}} = \frac{512 \text{ KB}}{2 \text{ KB}} = 256$$

$$L.O = 8 \text{ bit}$$

eg: Consider a 2-way Set associative mapping, if the Size of the Cache memory is 512KB and main memory 512MB and Cache line Size is 2KB then Calculate the number of bit required for TAG?

Ans.

$$Pa = 29 \text{ bit}$$

Tag	S.O	W.O
11 bit	7 bit	11 bit

$$\# \text{ Lines} = 2^8$$

$$\# \text{ Set} = \frac{2^8}{2} = 2^7$$

$$S.O = 7 \text{ bit}$$

$$29 - (7 + 11)$$

$$\text{TAG} = 11 \text{ bit}$$

2 Way Set-Associative

$$\text{Tag bit} = \text{Tag bit in} + \lceil \log_2 N \rceil$$

direct map

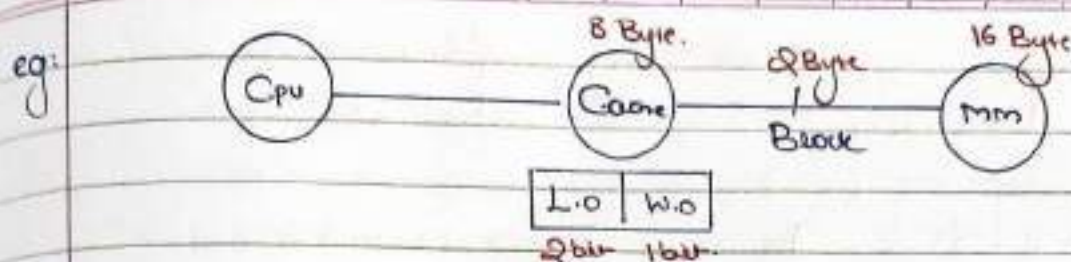
$$= 10 + \lceil \log_2 2 \rceil$$

$$= 11 \text{ bits}$$

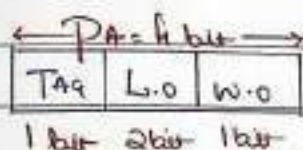
Associative Mapping:

- * In this no mapping function is used to transfer the data from mm to cm.
- * Any block of main memory can be placed anywhere in cache memory.
- * No Conflict miss.
- * This cache is designed without address called as Content Addressable memory.
- * In associative mapping more tag bits are required along with more memory size.
- * In associative mapping more hardware required, Expensive (N tag comparator, here N is number of lines, for each comparator is required)
- * In associative cache design, Counter Sequence is used to map the data, means any main memory block any cache memory line in a sequence.
- * In associative mapping Physical address is interpreted as

TAG	W.O.
-----	------



Direct Mapping



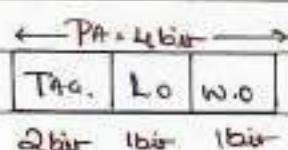
$$K \bmod N = i$$

$$K \bmod 4 = i$$

$$\text{Tag memory} = \# \text{lines} \times \text{Tag bit}$$

$$= 4 \times 1 = \underline{4 \text{ bits}}$$

Set-Associative Mapping



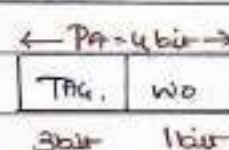
$$K \bmod S = i$$

$$K \bmod 2 = i$$

$$\text{Tag memory} = 4 \times 2$$

$$= \underline{8 \text{ bits}}$$

Associative / Fully Associative

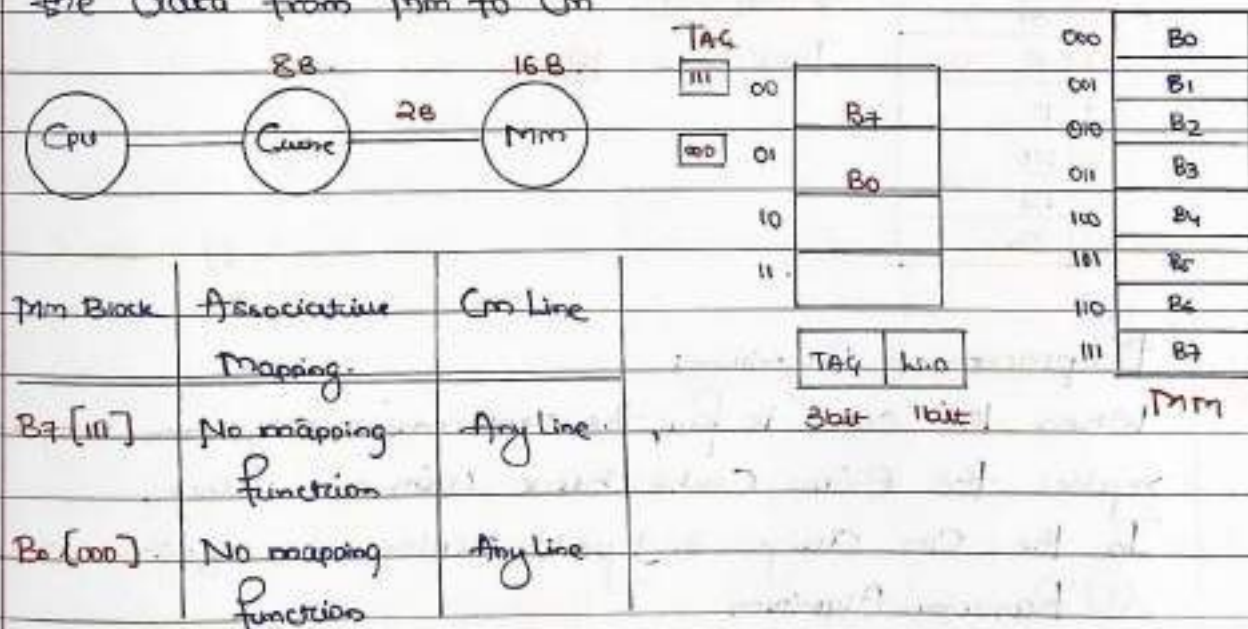


No mapping function

$$\text{Tag memory} = 4 \times 3$$

$$= \underline{12 \text{ bits}}$$

In associative mapping no mapping function is used to transfer the data from mm to Cm



eg Direct Mapped Cache

$$\text{Cache Size} = 64 \text{ KB}$$

$$\text{Block Size} = 64 \text{ word}$$

$$\text{Word length} = 32 \text{ bits}$$

$$\text{MM} = 4 \text{ Byte} = 2^{32} \text{ Byte}$$

$$\text{Block Size} = 64 \text{ word}$$

$$= 64 \times 4 \text{ Byte}$$

$$= 256 \text{ Byte}$$

$$\text{W.O} = 8 \text{ bit}$$

$$\# \text{ Lines} = \frac{\text{Cm Size}}{\text{Block Size}} = \frac{64 \text{ KB}}{256 \text{ B}} = \frac{2^{16}}{2^8} = 2^8 \text{ lines}$$

$$\text{Block Size} = 256 \text{ B} = 2^8 = 10 = 8 \text{ bits}$$



Tag entry = Tag bit + extra bit

$$= 16 + 1 + 1$$

$$= 32 - (8 + 8)$$

$$= 18 \text{ bits}$$

$$= 16 \text{ bit}$$

Tag memory Size = # Lines × Tag bit

$$= 2^8 \times 18$$

$$= 256 \times 18$$

$$= 4608 \text{ bits}$$

Q1. Consider fully associative cache consist 8 Block and Main memory contain 128 Block and request made by the CPU: 119, 84, 37, 0, 16, 0, 84, 120, 121, 93, 37, 0, 43, 39, 47, 48. Calculate the # of Compulsory and Capacity miss?

Ans.

0	119	43
1	84	39
2	37	47
3	0	48
4	16	
5	120	
6	121	
7	93	

Hit: 0, 84, 39, 0

hits = 4

Total miss = 12.

Replacement Algorithm:

When the cache is full, the replacement algorithm are required to replace the exist cache block with new block.

In the Cm design 3 type of replacement algorithm is used.

- ① Random Algorithm
- ② FIFO Algorithm
- ③ LRU replacement.

In the random algorithm, any cache block can be replaced based on the random selection.

Types of Miss:

In the C_m design 3 types of misses are present:

- ① **Compulsory miss** - (Cold start miss / first reference miss)
This miss will occur when the very first reference to the cache itself is a miss.
- ② **Capacity miss** - This will occur only when the capacity is full.
- ③ **Conflict miss** - (Collision miss / reference miss)
This miss will occur when too many blocks are placed into same cache line or same cache set.

Q2 Consider 4 block cache memory (initially empty) with the following mem blocks referenced: 7, 8, 10, 15, 7, 8, 10, 7, 8, 10. Identify the hit ratio using:

Ans.

7
8
10
15

} Compulsory miss

Mem request = 10

Cache line = 4

7 → conflict

8 → conflict

15 → compulsory miss

7
8
10

} Conflict

Direct mapped Cache:

Hit ratio = $3/10 = 0.3$

2 Way Set-Associative with LRU

#-lines = 4

Mem request = 10

Hit Ratio = $\frac{4}{10} = 0.4$

Q3. Consider a small two-way set associative cache memory, consisting of 4 blocks. For choosing the block to be replaced, use the least recently used (LRU) scheme. The number of cache misses for the following sequence of block address is 8, 12, 0, 12, 8.

Ans.

Cache lines = 4 2way set associative

Set = $4/2 = 2$

$K \bmod 2 = i$

$12 \div 2 = 0 \rightarrow \text{Set 0}$

$8 \bmod 2 = 0 \rightarrow \text{M}$

$8 \div 2 = 0 \rightarrow \text{miss (apply LRU)}$

$12 \bmod 2 = 0 \rightarrow \text{M}$

Set 0

8	8
12	

∴ Cache misses = 4

$0 \div 2 = 0 \rightarrow \text{M (cause set is full, apply LRU replac. algo.)}$

Set 1

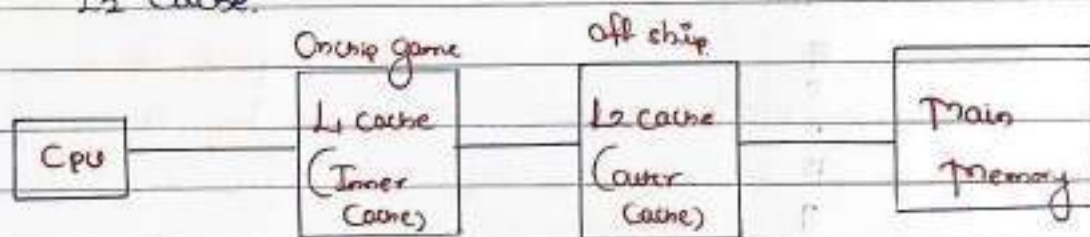
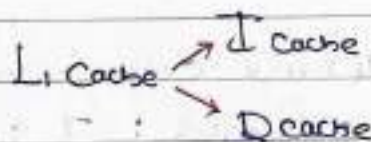
How to reduce all the miss:

- ① Compulsory miss: We can reduce the compulsory miss by increasing the block size (large block size).
- ② Capacity miss: Capacity miss occurs when cache is full. If we increase the cache size then capacity miss can be reduced.
- ③ Conflict miss: Conflict miss can be reduced by increasing the associativity.

Multilevel Cache:

2 Level Cache:

- L₁ cache
- L₂ cache



Size $L_2 > L_1$

Speed $L_1 > L_2$



I-Cache: Instruction Cache

D-Cache: Data Cache

Working Principal of multilevel cache:

- ① Principal of Inclusion: Data present in L₁ cache must be present in L₂ cache.
- ② Principal of Exclusion: Data present in L₁ cache must be different from L₂ cache.

Why Multi level Cache is used:

Level 1 Cache access = 10 nsec
time

Main memory access time = 30 nsec

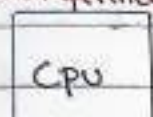
L1 Cache access time = 10 nsec

L2 cache access time = 15 nsec

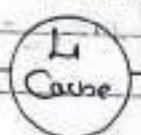
Prm access time = 30 nsec

So Tag will reduce

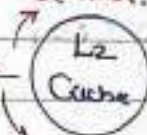
Cpu generate
100 reference



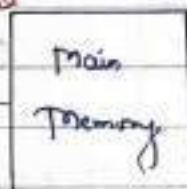
40 miss



20 miss



Here 20 times
main ref



main access time 20 times
used.

If only L1 cache then if
there is miss in level 1 cache
then directly refer to main
memory
(Here higher time use main
memory access so Tag increase)

But in 2 level cache, if miss in
level 1 cache then that miss forwarded
to level 2 cache (so some hit in
level 2) then after level 2 there is
a miss then we go to main memory.
So Tag will be reduced (because L1 and
L2 Cache access time very low).

Multi Level Cache

- To reduce the miss penalty multi level caches are used in the system design.
- The number of cycles required to transfer the data from higher levels to L1 due to miss operations is called as miss penalty.

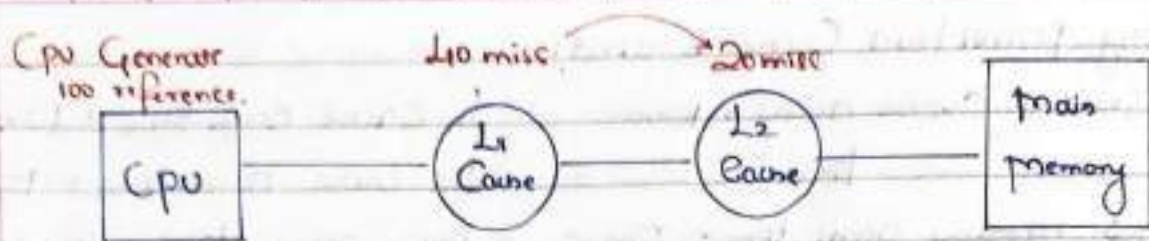
Miss Rate

Local Miss Rate (Lmr)

$$Lmr = \frac{\# \text{misses in cache}}{\# \text{access to that cache}}$$

Global Miss Rate (Gmr)

$$Gmr = \frac{\# \text{misses in the cache}}{\text{Total } \# \text{ CPU generated reference}}$$



$$LMR_{L1} = \frac{40}{100} = 0.4$$

$$LMR_{L2} = \frac{20}{40} = 0.5$$

$$GMR_{L1} = \frac{40}{100} = 0.4$$

$$LMR_{L2} = \frac{20}{100} = 0.2$$

Access time in Multilevel Cache:

Level 1:

$$T_{avg} = h_1 t_1 + (1 - h_1) (t_m + t_1)$$

or

$$T_{avg} = t_1 + (1 - h_1) (t_m)$$

Level 2:

$$T_{avg} = h_1 t_1 + (1 - h_1) h_2 (t_2 + t_1) + (1 - h_1) (1 - h_2) (t_m + t_2 + t_1)$$

or

$$T_{avg} = t_1 + (1 - h_1) \{ t_2 + (1 - h_2) t_m \}$$

hit time
in level 1

miss rate of
level 1

hit time of
level 2

miss rate of
level 2

memory access
time

Updating Technique:

- * CPU perform read or write operation on cache memory.
- * When the data (mem block) is available in cache then it is known as read hit and write hit.
- * If the data is not available in the cache then it is called read miss and write miss then we have to perform Read allocate and write allocate.

Read Allocate: When there is a read miss in the cache then transferring that data from main memory to cache memory is

Called read allocate.

Write Allocate: When there is a write miss in the cache then transferring that data (mm block) from main memory to cache memory is called write allocate.

Need of updating technique:

Assume in future there is a cache miss for any mm block, which are mapped into cm line No. 0. Then the updated data (of block 20) is replaced by new block. (As the updation gets lost).

That means updation/ work done by the CPU and load into the cache get lost. Because, that block is replaced by the new main memory block.

Now in future if we bring Block 20 then we get the old value.

Updating Techniques:

- ① Write Through: In the write through both cache memory and main memory update simultaneously so no problem of coherence (No inconsistency).

$$t_w \left[\frac{\text{word updation}}{\text{time}} \right] = \max \left\{ \frac{\text{word updation}}{\text{time in cache}}, \frac{\text{word updation}}{\text{time in mm}} \right\}$$

$$T_{\text{avg read}} = h_r t_c + (1-h_r) (t_m + t_o)$$

Read hit
Read data
Read miss
Read allocate
Read data from cm.

$$T_{\text{avg write}} = h_w t_c + (1-h_w) (t_m + t_w)$$

Write hit
Word updation
Write miss
Write allocate
Word updation

$$T_{\text{avg wr}} = \% \text{ of read Operation} \times T_{\text{avg read}} + \% \text{ of write Operation} \times T_{\text{avg write}}$$

$$\eta_{wr} = \frac{1}{T_{avg wr}}$$

Disadvantage in WR: If there is a frequent updation in cache, then WR give worst performance because every updation, we have to update pm and cm block.

Simultaneous Access:

$$T_{avg read} = h r t c + (1-h r) t m$$

$$T_{avg write} = t w$$

- ② Write Block Update Technique: In the WB updating technique main memory update later, only cache memory update. So in WB cache memory update and pm update later so coherence is present.

But this coherence does not create any problem

- * Because in WB, each cache line maintain some extra bit (like Valid/invalid, dirty bit (modified bit)) to maintain the status of cm block.
- * If the cm block is updated then modified bit (dirty bit) is set to 1. If block is not updated then clean bit is set (dirty bit = 0).
- * Before replacing the cm block we check the status of the block.
- * If a block is a updated block (modified bit set) then firstly this updated data write back into main memory (higher memory) then replace the cm block with any new (request) block.
- * If block is not updated block then directly replace with new main memory (requested block).

$$T_{avg_read} = h r t_c + (1-hr) \left\{ \frac{1}{\text{modified}} (t_x + t_m + t_c) + \frac{1}{\text{clean hit}} (t_m + t_c) \right\}$$

$$T_{avg_write} = h w t_c + (1-hw) \left\{ \frac{1}{\text{modified}} (t_y + t_m + t_c) + \frac{1}{\text{clean hit}} (t_m + t_c) \right\}$$

$$T_{avg_wb} = F_r \times T_{avg_read} + F_w \times T_{avg_write}$$

$$\eta_{wb} = \frac{1}{T_{avg_wb}}$$

F_r = % of read operation

F_w = Frequency(%) of write operation

Q1. Let WB and WT be two Set associative Cache organisations that use LRU algorithm for cache block replacement. WB is a write back Cache and WT is a write through Cache. Which of the following statements is/are true or false?

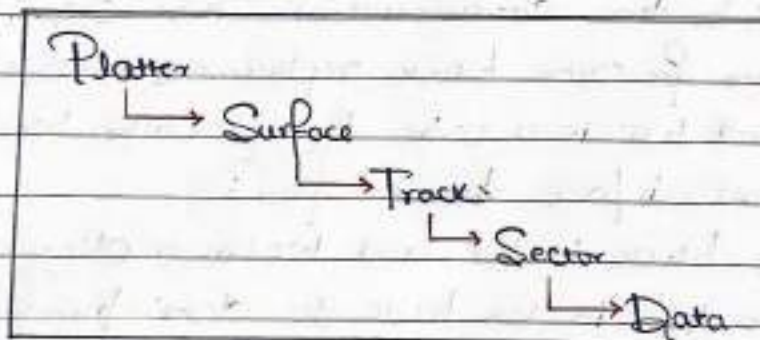
- Ans.
- A. Each cache block in WB and WT has a dirty bit. ☒
 - B. Every write hit in WB leads to data transfer from Cache to main memory. ☒
 - C. Eviction of a block from WT will not lead to data transfer from Cache to main memory. ☒
 - D. A read miss in WB will never lead to eviction of a dirty block from WB. ☒

While Cache Accessing:

1. Physical Index Cache : PM address is used in address mapping.
2. Virtual Index Cache : VM address is used in address mapping.
(VIRT) $\rightarrow$ Virtual Index Virtual Tag.

SECONDARY MEMORY & I/O INTERFACE

- * Magnetic disks provide the bulk of Secondary Storage for modern Computer Systems. Conceptually, disks are relatively simple.
- * Each disk platter has a flat circular shape, like a CD. Common platter diameters range from 1.85 to 3.5 inches.
- * The two surfaces of a platter are covered with magnetic heads are attached to disk arm that moves all the heads at a unit.
- * The surface of a platter is logically divided into circular tracks, which are subdivided into sectors.
- * The set of tracks that are at one arm position makes up a cylinder. There may be thousands of concentric cylinders in a disk drive, and each track may contain hundreds of sectors. The storage capacity of common disk drives is measured in disk drives.



Moving-Head Disk Mechanism

- * Each platter has 2 surfaces
- * Read/write head with each surface to read/write data
- * These read/write heads are all connected with the arm.
- * All these arms are connected to arm assembly and rotate at the same time.

eg: If there are 20 surfaces then:

* Read/write head : 20

* Arm : 20

* Arm Assembly : 1

Note: - All read/write head (Arm) rotating/revolving at the same time but only 1 R/W head performing read/write operation.

Q1. Consider a disk which has 16 platters. Each platter has two surfaces. Every surface has 1K Tracks is further divided into 512 Sector and every Sector can store the 8KB data, then calculate.

(i) What is the capacity of the disk?

Ans. 16 platters, each platter has 2 surfaces.

Total number of surfaces = $32 [16 \times 2]$ surfaces

Tracks per surface = 1K, total = 32K tracks

Sectors/track = 512

Total # sectors = $32K \times 512$ Sectors

Each Sector Capacity = 8KB

Disk Capacity = $32K \times 512 \times 8KB$
 $= \underline{128GB}$

(ii) How many bits are required to identify any particular Sector of disk?

Ans. # bits to identify = $16 \times 2 \times 1K \times 512$

Particular Sector = $2^4 \times 2^7 \times 2^{10} \times 2^9$

= 2^{24}

= 24 bits

Q2. Consider a disk pack with 16 surfaces, 128 tracks per surface and 256 Sectors per track. 512 bytes of data are stored in a bit-serial manner in a sector. The Capacity of the disk pack and the number of bits required to specify a particular Sector in the disk are respectively

Ans. Disk Capacity = $16 \times 128 \times 256 \times 512$ Bytes
 $= 2^4 \times 2^7 \times 2^8 \times 2^9$ Bytes
 $= 2^{28}$ Bytes
 $= \underline{256MB}$

bits required = $16 \times 128 \times 256$

to identify particular = $2^4 \times 2^7 \times 2^8$

Sector = 2^{19}

= 19 bits

Disk I/O Operation:

- ① Seek time: The amount of time taken to move the read/write head from its current position to the desired track is called as Seek time.
- ② Rotating Latency: The amount of time taken to rotate the disk when the read/write head comes out to exact position.
Rotational Latency is considered as $\frac{1}{2}$ Rotation Time
- ③ Transfer time: The amount of time taken to transfer the required data.
- ④ Transfer rate: The number of bytes found for unique line is called as transfer rate of disk.

- * The read/write head can never be outside the track it will be pointing to any particular track of the surface.
- * Read/write head will move in forward and backward direction and disk in one direction (either clockwise or anticlockwise).

eg: 600 Rpm. (600 rotation per minutes)

$$1 \text{ Rotation} = \frac{60}{600} = \frac{1}{10} \text{ sec} \times \frac{1000}{1000} = \frac{1000 \times 10^{-3} \text{ sec}}{1000} = \underline{100 \text{ msec}}$$

$$1 \text{ Rotation time} = \frac{1}{10} \text{ sec or } 100 \text{ msec}$$

$$\text{Avg rotational latency} = \frac{1}{2} \times 100$$

$$= \underline{50 \text{ msec}}$$

eg: Assume 1 Rotation time $\frac{1}{6}$ sec, 1 Track Capacity = 4KB then Calculate data transfer time for 2 Byte and data transfer rate?

- (i) Data Transfer time:
- | | | |
|--------|---|--|
| 4KB | → | $\frac{1}{6}$ sec |
| 1 Byte | → | $\frac{1}{6 \times 1024}$ sec |
| 2 Byte | → | $\frac{2 \text{ Byte}}{6 \times 1024} \text{ sec}$ |

(ii) Data Transfer Rate:

$$\begin{aligned}
 4 \text{ KB} &\longrightarrow \frac{1}{6} \text{ sec} \\
 \frac{1}{6} \text{ sec} &\longrightarrow 4 \text{ KB} \\
 1 \text{ sec} &\longrightarrow 4 \text{ KB} \times 6 \text{ sec} \\
 &= \underline{24 \text{ KB/sec}}
 \end{aligned}$$

Disk Access Time = S.T + R.L + D.T.T + Overhead (D.A.T) (if any)

- Q1. Consider a disk system, which has an average seek time of 30 ms and rotational rate of the disk is 360 Rpm. Each track of the disk has 512 sectors, each of the size 512 Byte, then calculate.
- Average seek time
 - Average rotational latency
 - Data transfer time for 4 sectors (continuous)?
 - Data transfer rate?

Ans 360 Rpm

$$\begin{aligned}
 1 \text{ Rotation} &= \frac{60}{360} = \frac{1}{6} \text{ sec} \\
 \text{Aug. R.L} &= \frac{1}{2} \times \frac{1}{6} = \frac{1}{12} \text{ sec} \\
 &= \underline{0.083 \text{ sec}}
 \end{aligned}$$

Aug Seek time = 30 msec

1. Track Capacity = # sectors/track $\times$ each sector Capacity

$$\begin{aligned}
 &= 512 \times 512 \text{ Byte} = 2^9 \times 2^9 \text{ B} = 2^{18} \text{ B} \\
 &= \underline{256 \text{ KB}}
 \end{aligned}$$

Data Transfer Amount = 4 sectors $\rightarrow 4 \times 512 \text{ B}$

$$\begin{aligned}
 &= 2^2 \times 2^9 \text{ B} \\
 &= \underline{2 \text{ KB}}
 \end{aligned}$$

Data Transfer Time:

$$256 \text{ KB} \longrightarrow \frac{1}{6} \text{ sec}$$

$$2 \text{ KB} \longrightarrow \frac{2 \text{ KB}}{6 \times 256 \text{ KB}} \text{ sec}$$

$$2 \text{ KB} = \frac{2 \text{ KB}}{6 \times 256 \text{ KB}} \text{ sec} \Rightarrow \frac{1}{768} \text{ sec}$$

Data Transfer Rate

$$256 \text{ KB} \longrightarrow \frac{1}{6} \text{ sec}$$

$$\frac{1}{6} \text{ sec} \longrightarrow 256 \text{ KB}$$

In 1 sec $\rightarrow 6 \times 256 \text{ KBps}$

$$= \underline{1536 \text{ KBps}}$$

Fixed Sector Capacity: Each Sector (Innermost and Outermost Sector) has fixed storage. (Storage density is variable)

Variable Sector Capacity: Each sector has variable storage. Innermost and outermost Sector Capacity is variable, (Storage density is fixed).

"In Today's time we generally use 'fixed sector capacity'."

Need of Cylinder:

Assume that we have a very large file size (in GB) and 1 track size is 32 MB.

If we store a file on a disk then we can store 32 MB on one track then we have to go next track of the same surface so on storing data. So we have to suffer unnecessary very high seek latency (time).

So we store the data cylinder wise as we know each surface r/w head point to same address at a time.

So first we store

{ - track '0' of surface '0' - track '0' of surface '1' " " " " - track '0' of surface '200' }	1 Seek time (latency)
---	--------------------------

With the help of cylinder we can store 200 Track data in one seek time.

"Same track number in all the surface will form a cylinder"



Number of planes = 2

Number of Surface = 4

Number of tracks per Surface = 5

Number of Cylinders = 5 (0-4)

Number of Sectors per track = 8

Number of Sectors per Cylinder = $4 \times 8 = 32$



8 Sectors
per track

In this example each track contains 8 Sectors. This is for 1st Cylinder (Cylinder-0)

First we traverse (cross) all sectors of track 0 (1st track) of surface 0

Then we traverse (cross) " " " " 0 of surface 1

" " " " 0 " " 2

" " " " 0 " " 3

I Cylinder : Cylinder Number 0

Step 1 : Surface number 0 (Ist surface) track no 0 (1st track) all 8 Sectors are traversed (crossed).

Step 2 : Surface number 1 (IInd surface) track no 0 (1st track) all 8 Sectors are traversed (crossed).

Step 3 : Surface number 2 (IIIrd surface) track no 0 (1st track) all 8 Sectors are traversed (crossed).

Step 4 : Surface number 3 (IVth surface) track no 0 (1st track) all 8 Sectors are traversed (crossed).

Now move to next Cylinder, and so on.

I Cylinder : Cylinder Number 0

$\langle C, h, s \rangle$

0th Surface { $\langle 0, 0, 0 \rangle$ 1st Sector
 $\langle 0, 0, 1 \rangle$ 2nd Sector
 $\vdots$
 $\langle 0, 0, 7 \rangle$ 7th Sector

1st Surface { $\langle 0, 1, 0 \rangle$ 8th Sector

Sectors/track : 8

Surface : 4

track per surface = 5

Cylinder = 5 (0-4)

Surface = 4 (0-3)

And so on, upto 31st Sectors.

- * All 32 Sector Stored in 4 Surface of Cylinder no 0.
- * Now we move to the next cylinder 2nd cylinder (Cylinder number 1). And so on to all cylinders.
- * In this manner we store total 160 ~~store~~ Sector which stored in (32 Sector per cylinder $\times$ 5 cylinder).

eg: What is the Sector no. of $\langle 4, 3, 7 \rangle$

(i) To cross 4 cylinder = $4 \times 4 \times 8 = 128$
(0, 1, 2, 3)

(ii) To cross 3 surface = $3 \times 8 = 24$

(iii) To cross 7th sector = 7

159 (Ans)

Q1. A hard disk has 63 sectors per track, 10 platters each with 2 recording surfaces and 1000 cylinders. The address of a sector is given as a triple $\langle c, h, s \rangle$ where c is the cylinder number, h is the surface number and s is the sector number. Thus the 0th sector is addressed as $\langle 0, 0, 0 \rangle$ the 1st sector as $\langle 0, 0, 1 \rangle$ and so on.

Surface = $10 \times 2 = 20$ Surface

Sector/track = 63

Sector per cylinder = 20×63

1260 Sector/cylinder

(i) The address $\langle 400, 16, 29 \rangle$ corresponds to Sector number:
 $\langle 400, 16, 29 \rangle$

(i) To cross 400 cylinder = $400 \times 20 \times 63 = 504,000$

(ii) To cross 16 Surface = $16 \times 63 = 1008$

(iii) To cross 29 Sector = 29

505037

Alternate method

Sector Number = $S + ST \times h + ST \times TC \times C$

ST: # Sector per track

TC: # track per cylinder

SC: # Sector per cylinder

$$\rightarrow \boxed{ST \times TC}$$

$$\text{Sector Number} = 29 + 63 \times 16 + 63 \times 20 \times 400 \\ = \underline{\underline{505037}} \text{ (Ans)}$$

(ii) The address of 1039th Sector is:

$$\langle 0 + 15 \times 63 + 31 \rangle = 976$$

$$\langle 0 + 16 \times 63 + 30 \rangle = 1038$$

$$\langle 0 + 16 \times 63 + 31 \rangle = 1039$$

$$\langle 0 + 17 \times 63 + 31 \rangle = 1102$$

$$\therefore \underline{\langle 0, 16, 31 \rangle} \text{ (Ans)}$$

$$C = \left\lfloor \frac{\text{Given Sector No}}{\text{Total \# Sector/cylinder}} \right\rfloor \\ = \left\lfloor \frac{1039}{1260} \right\rfloor = 0 \text{ (1st cylinder)}$$

$$h = (1039 - 1260) / 63 = \left\lfloor \frac{1039}{63} \right\rfloor$$

How much Sector remaining

$$= 1039 - 16 \times 63$$

$$= 1039 - 1008$$

$$= \underline{31}$$

$$= \underline{16 \text{ Surface}}$$

Note: $\langle C, h, s \rangle$

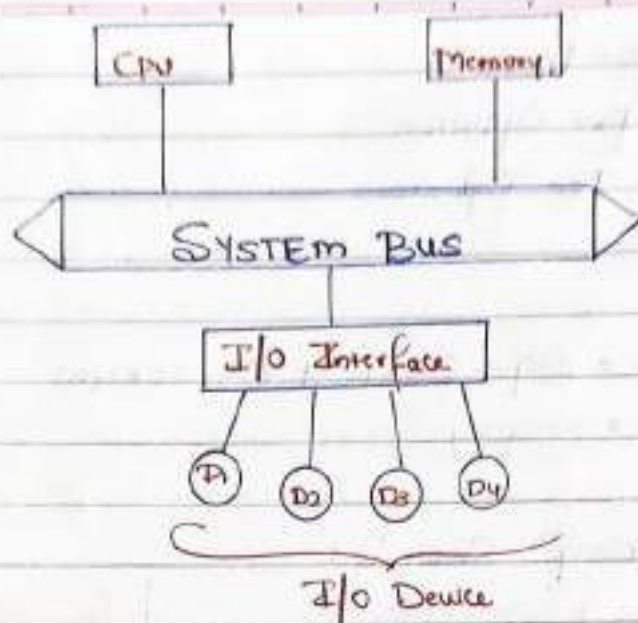
$$\boxed{C = \left\lfloor \frac{\text{Given Sector}}{\text{\# Sector/cylinder}} \right\rfloor}$$

(Cylinder Number)

$$\boxed{h = \left\lfloor \frac{(\text{Given Sector} \% \text{\# Sector per cylinder})}{\text{Total Sector per track}} \right\rfloor}$$

(Surface Number)

$$\boxed{\text{Sector Number } y = x * \text{\# Sector}}$$



Input - Output Interface

Input-output interface provides a method for transferring information between internal storage and external I/O devices.

Peripherals connected to a computer need special communication links for interfacing them with the central processing unit. The purpose of the communication link is to resolve the differences that exist between the central computer and each peripheral. The major differences are:

- ① Peripherals are electrochemical and electromagnetic devices and their manner of operation is different from the operation of the CPU and memory, which are electronic devices. Therefore, a conversion of signal values may be required.
- ② The data transfer rate of peripherals is usually slower than the transfer rate of the CPU and consequently, a synchronization mechanism may be needed.
- ③ Data codes and formats in peripherals differ from the word format in the CPU and memory.
- ④ The operating modes of peripherals are different from each other and each must be controlled so as not to disturb the operation of other peripherals connected to the CPU.

I/O Organisation

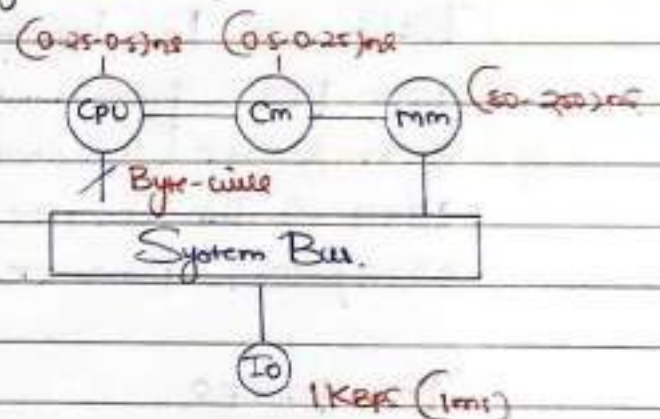
- ① I/O devices are electro-magnetic components and CPU is an electronic component. So, there is a difference that exists in the terms of operating modes, data transfer rate and word formats.
- ② To synchronise the I/O speed with a CPU, high speed interface chip is used named as I/O interface or I/O module.
- ③ I/O interface chip is responsible for I/O operations so, the computer system design I/O devices are connected to system bus via I/O interface chip.

System without I/O-interface (Programmed-I/O).

1 KB - 1 sec

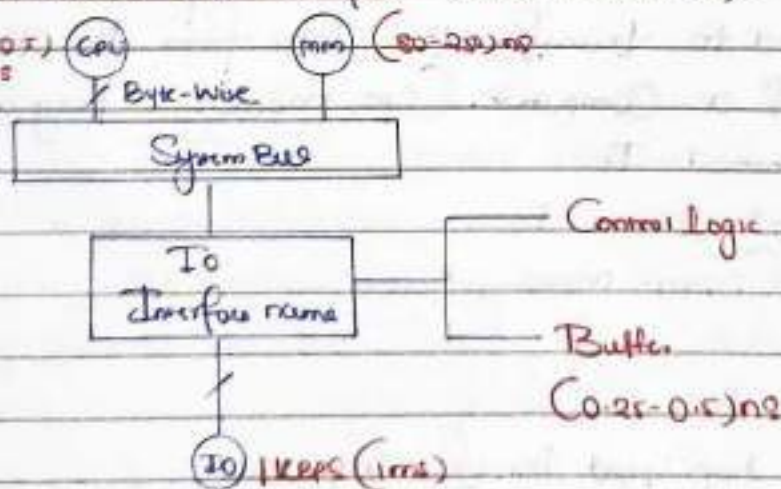
1 B - ?

$$ETIO = \frac{1 \text{ B}}{1 \text{ KB}} \text{ sec} = 10^{-3} \text{ sec} = \underline{1 \text{ ms}}$$



System with I/O-Interface (Interrupt-Driven I/O).

(0.25-0.5) ns CPU (50-250) ns mm



Access Sequence/Working Process

- ① CPU initializes the I/O interface chip along with an I/O command and then CPU will go and perform other useful task.
- ② I/O interface Control Logic interprets the I/O Commands and

Accordingly I/O port will be enabled for the operation.

- ③ Based on the speed of a I/O device and amount of data to be transfer consume the time to prepare the data, then data is transferred from I/O device to a interface buffer.
- ④ When the data is available in the buffer I/O interface generates the interrupt signal and send to the CPU and waiting for ack signal.
- ⑤ After receiving the ack signal, buffer content will be transferred to CPU. In this process, CPU will be accessing the I/O data from interface buffer therefore speed gap is synchronized & time is saved.

Different I/O interface chip used in the computer design is:

8255 PPI (Programmable Peripheral Interface)

8251 USART (Universal Sync asynchronous receiver transmitter).

8259 A Int. Controller

8237/8257 DMA etc.

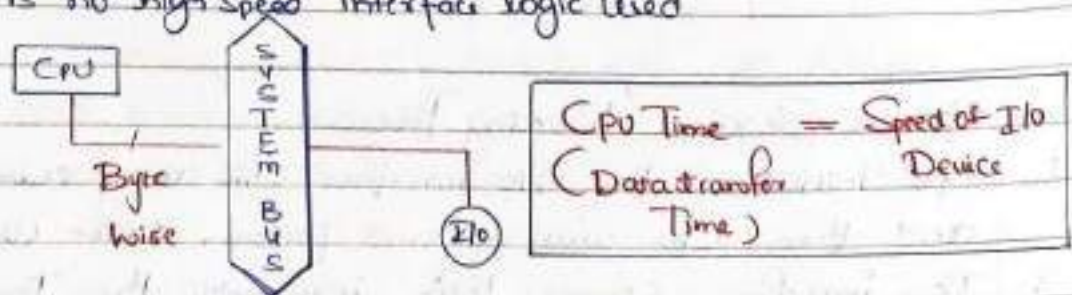
I/O - MODES

Three types of I/O transfer modes are present in the computer system, used to transfer the data from the I/O to other component of a computer (CPU, memory). They are:

1. Programmed - I/O.
2. Interrupt driven I/O
3. DMA (Direct Memory Access).

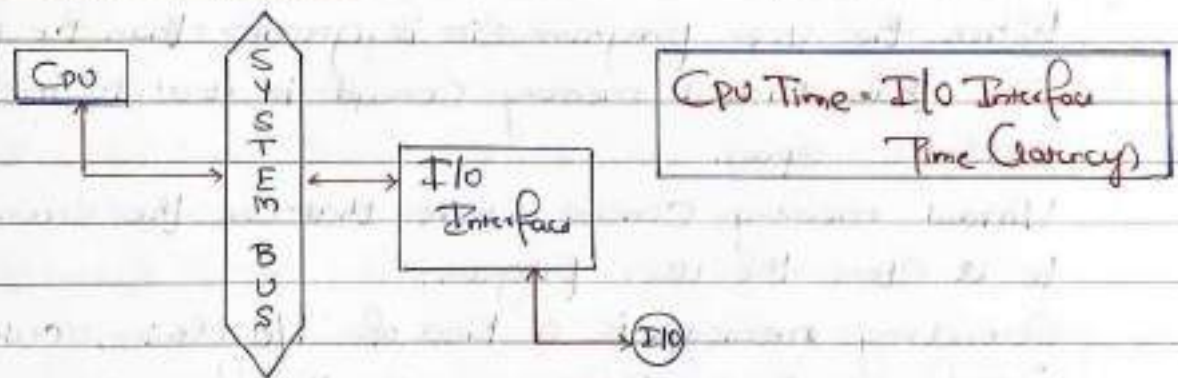
① Programmed - I/O

There is no high speed interface logic used



- * In this mode, I/O devices are directly connected to CPU without I/O interface chip.
- * In this mode, CPU takes the responsibility to complete the I/O operation, so CPU will be blocked (waiting) until the I/O operation is completed.
- * In this mode CPU utilization is inefficient.
CPU time depends on the speed of a I/O device and the size of a data unit to be transferred.
- * This mode is suitable in the system centric applications where the I/O time is important than CPU time.

② Interrupt Driven I/O



- * In this mode, I/O operations are controlled based on the interrupt signal.
- * I/O devices are connected to a system bus via I/O interface chip so, I/O interface takes the responsibility of a I/O operation.
- * In this mode processor utilization is more efficient so CPU executing the other useful task during the I/O operation.
- * CPU depends on the latency of an interface chip rather than speed of an I/O device.

Drawback: More complexity in interrupt implementation.

Solution: Use priority sense highest priority interrupt.

Who will assign:

1. Daisy Chain Method (Static Approach): Fixed: Starvation Occurs
2. Polling Method (Dynamic Approach): Changing: No Starvation

- ① Programmed I/O
- ② Interrupt Driven I/O

* In these two we cannot access the memory directly (we have to access memory with the involvement of CPU).

* These are used for small amount of data transfer.

③ Direct Memory Access:

* In this mode, bulk amount of data will be transferred from the I/O to main memory without involvement of a CPU. Here Processor and DMA Controller use the System bus in Master-Slave mode.

* When the User program size is greater than the main memory size then Virtual memory Concept is used to increase the address space.

* Virtual memory Concept states that use the Secondary memory to store the user program.

* Secondary memory is a kind of I/O device, which is interfaced to a DMA module, therefore during the program execution data will be transferred from I/O to main memory via DMA controller without involvement of a CPU.

Access Sequence (DMA)

- ① CPU initializes the DMA module along with a I/O command later busy with other useful task.
- ② I/O Command contains the information about Port address, memory address, Control Signal and Count value.
- ③ DMA module Control logic interprets the Command and enables the respective Port for the operation.
- ④ Based on the Speed of a device consumed the time to transfer the data later enables the "DMA Req" signal.
- ⑤ After receiving this signal DMA module enables the hold signal to CPU to gain the control of a System bus and

waiting for HMA signal.

- ⑥ After receiving the HMA signal; Dma module enables the "DmaAck" signal.
- ⑦ After receiving the signal, Io device transfer the data to main memory via Dma to main memory via Dma until Count becomes "0".
- ⑧ After the Dma Operation Bus Connection will be re-established to Cpu.

Data Count: The number of Bytes/words transfer from I/O to memory until the Count value becomes '0'.

Note: In the Dma operation, Cpu is in two States:

- ① Busy State: Cpu is in busy state until prepare the data so busy state depends on preparation time, preparation time depends on speed of I/O speed and amount of data.
- ② Blocked State: Cpu is in Blocked state until transferring the data so blocked time depends on transfer time. Transfer time depends on pin latency (pin access time).

→ Let x is preparation time and y refers to transfer time

% time Cpu Busy = $\left(\frac{x}{x+y} \right) 100$

% time Cpu Blocked = $\left(\frac{y}{x+y} \right) 100$

Dma module is operating in three mode:

1. Burst mode
2. Cycle Stealing mode
3. Interleaving mode

Q1. A device with data transfer rate 10Kb/sec is connected to a system. Data is transferred byte-wise. Let the interrupt overhead be 4 μ sec. The byte transfer time between the device interface register and CPU to memory is negligible. What is the performance gain of operating the device under "interrupt mode" over operating it under "Program Controlled mode".

Ans. Programmed I/O:
 10 Kbps
 10 KB $\rightarrow$ 1 sec
 1 Byte $\rightarrow \frac{1}{10 \times 10^3}$ sec $\rightarrow 10^4$ sec $\times \frac{100}{100} = 100 \times 10^{-6}$
 $= 100 \mu$ sec
 $E_{\text{prog. I/O}} = 100 \mu$ sec
 $E_{\text{interrupt}} = 4 \mu$ sec
 $\therefore = \frac{E_{\text{prog. I/O}}}{E_{\text{interrupt}}} = \frac{100}{4} = 25 \text{ (times)}$

Q2. The size of the data count register of a DMA Controller is 16 bit. The processor needs to transfer a file of 29,154 Kbytes from disk to main memory. The memory is byte addressable. The minimum number of times the DMA Controller needs to get the control of the system bus from the processor to transfer the file from the disk to main memory is _____

Ans. Count register = 16 bit
 * Data transfer in one cycle = $2^{16} - 1 = 64 \text{ KB}$
 Size of data to be transferred = 29,154 Kbytes
 * Number of times DMA need to get control of system bus = $\left\lceil \frac{29154}{64} \right\rceil = \left\lceil 455.5 \right\rceil = 456 \text{ (times)}$

Q3. Consider a disk with data transfer rate 50 MBps. It is operated with cycle stealing mode of DMA. Here whenever 64 bit information is available it is transferred in 40ns. What is the % of time CPU blocked due to DMA _____

Disk speed = 50 MBps
 64 bit data transfer (2 bytes)
 $x : [\text{Preparation time}]$
 Data transfer time = 40ns
 (y)

$$50 \text{ MB} \rightarrow 1 \text{ sec}$$

$$8 \text{ Byte} \rightarrow \frac{8 \text{ Byte}}{50 \text{ MB}} = 160 \text{ nsec} \\ = 16 \text{ usec}$$

$$\therefore \text{time Cpu Recv} = \frac{y}{x} \times 100$$

$$= \frac{40}{160+40} \times 100 = \frac{40^2}{200} \times 100$$

$$= \underline{20\%} \text{ (Ans)}$$